



Foundations of Large Language Models and Prompt Engineering

Generated by Pustakam Injin

AI-Powered Knowledge Engine • **gpt-oss-120b**

Pustakam Injin

AI-Powered Knowledge Creation

Document Information

Word Count	30,332
Chapters	9
Generated	February 23, 2026
AI Model	Cerebras - gpt-oss-120b

Tanmay Kalbande

Creator & Engineer

pustakamai.tanmaysk.in

www.linkedin.com/in/tanmay-kalbande/

Foundations of Large Language Models and Prompt Engineering

Generated: 22/02/2026 Words: 30,332 Provider: Cerebras (gpt-oss-120b)

Introduction

Welcome to the World of End-to-End LLM Engineering

If you've ever imagined a chatbot that can answer questions with up-to-date knowledge, a search tool that understands the nuance of natural language, or a SaaS product that turns raw documents into instant expertise - you're not alone. The rapid evolution of Large Language Models (LLMs) has turned those fantasies into achievable projects, but the journey from a raw model to a production-ready service still feels like navigating a dense forest without a map.

That's exactly why this book exists. **"Learn End-to-End AI Engineering Skills to Build and Deploy LLM Applications Using Python, LangChain, and FastAPI"** is your compass. Whether you're a data scientist who has tinkered with GPT-3 in a notebook, a backend engineer curious about AI, or a product manager eager to prototype intelligent features, this guide will walk you step-by-step from the fundamentals of LLMs to a fully containerised, observable FastAPI service that you can ship to the cloud.

By the end of the journey you'll have a **complete, production-grade codebase** that you can clone, extend, and deploy - not just a collection of snippets. Think of this book as a hands-on bootcamp that blends theory, practice, and real-world

engineering concerns (security, monitoring, CI/CD). Let's dive into what you'll learn, how the material is organized, and why mastering this stack will become a decisive advantage in today's AI-first landscape.

What You'll Learn - The Skillset You'll Walk Away With

Domain	Core Competency	Real-World Outcome
LLM Foundations	Understand the architecture, tokenization, and prompt dynamics of modern LLMs.	Craft prompts that reliably coax the model into the behavior you need, and diagnose why a model may be "hallucinating."
Python Ecosystem for LLM Access	Use the OpenAI API, Hugging Face Hub, and the LangChain SDK to fetch and interact with models.	Swap providers or run locally without rewriting your business logic.
LangChain Core Concepts	Build chains, agents, and memory objects that orchestrate multiple LLM calls.	Assemble complex reasoning pipelines (e.g., multi-step planning) with just a few lines of code.
Vector Stores & Retrieval	Index, query, and manage embeddings using FAISS, Chroma, and Pinecone.	Power Retrieval-Augmented Generation (RAG) that pulls factual context from millions of documents in milliseconds.
RAG Pipelines	Combine vector retrieval, prompt engineering, and LLM generation into end-to-end workflows.	Deliver answers that are both knowledgeable (grounded in your data) and expressive (fluent in natural language).
FastAPI for LLM Inference	Design high-performance, async APIs that expose LLM capabilities.	Serve your models behind a RESTful or GraphQL endpoint that scales to thousands of concurrent requests.
Integration of LangChain &	Wire LangChain pipelines into	Offer a seamless developer

Domain	Core Competency	Real-World Outcome
FastAPI	FastAPI routes, handle streaming responses, and manage request lifecycles.	experience: one HTTP call and the LLM does the heavy lifting for you.
Production Concerns	Implement authentication, rate limiting, logging, tracing, and alerting.	Operate safely in a multi-tenant environment and quickly diagnose performance regressions.
Deployment & CI/CD	Containerise with Docker, orchestrate with Kubernetes (or serverless alternatives), and automate tests and rollouts.	Push updates several times a day without downtime, and keep cost-efficiency under control.

Collectively, these competencies will enable you to **design, implement, and maintain AI-powered services** that can be handed over to a DevOps team, integrated into existing product stacks, or even commercialised as a standalone SaaS.

How the Book Is Structured - Your Roadmap to Mastery

Below is a high-level overview of the eight chapters that constitute the core of the book. Each chapter follows a consistent pattern:

1. **Conceptual Primer** - The "why" behind the technique.
2. **Hands-On Lab** - A runnable notebook or script you can execute immediately.
3. **Deep-Dive Sidebar** - Optional reading on theory, alternatives, or performance trade-offs.
4. **Best-Practice Checklist** - A concise list of do's and don'ts for production use.

Chapter	Title	Key Topics

Chapter	Title	Key Topics
1	Foundations of Large Language Models and Prompt Engineering	Model architectures, token limits, prompting patterns, evaluation metrics.
2	Python Ecosystem for LLM Access (OpenAI, Hugging Face, LangChain SDK)	API authentication, client wrappers, handling streaming responses, offline inference.
3	Getting Started with LangChain: Core Concepts	Chains, agents, memory, callbacks, custom components.
4	Vector Stores and Retrieval: FAISS, Chroma, and Pinecone	Embedding generation, index building, similarity search, hybrid retrieval.
5	Building Retrieval-Augmented Generation (RAG) Pipelines	End-to-end RAG flow, prompt templating, citation handling, evaluation of factuality.
6	Designing FastAPI Services for LLM Inference	Async routes, dependency injection, streaming JSON, request validation.
7	Integrating LangChain with FastAPI: End-to-End Application	Full-stack example (document upload → vectorisation → query → answer), error handling, testing.
8	Security, Monitoring, and Observability in Production	JWT/OAuth, rate limiting, OpenTelemetry tracing, Prometheus metrics, logging strategies.
9	Deployment, Scaling, and CI/CD Pipelines	Dockerfiles, Docker Compose, Kubernetes manifests, GitHub Actions, blue-green deployments.

Appendices provide quick reference tables (e.g., model parameter limits), a glossary of AI terminology, and a curated list of community resources.

The progression is deliberately **bottom-up**: you begin by mastering the mental models that underlie LLMs, then acquire the tools to call them, then learn to augment them with external knowledge, and finally wrap everything in a robust,

production-ready API. By the time you finish Chapter 9 you will have built **one complete, deployable application** from scratch-your own "LLM-as-a-service" that you can showcase to employers or investors.

Why This Matters - The Motivation Behind the Journey

1. LLMs Are No Longer a Research Toy

A year ago, the idea of deploying GPT-4 behind a public endpoint seemed risky, expensive, and technically daunting. Today, the APIs are stable, pricing models are transparent, and the community has produced battle-tested libraries like LangChain that abstract away the messy plumbing. Companies across finance, healthcare, education, and entertainment are **embedding LLMs directly into their products**. Being able to deliver such functionality end-to-end places you at the forefront of a multi-billion-dollar wave.

2. Bridging the Gap Between Data Science and Engineering

Many practitioners can fine-tune a model or craft a clever prompt, yet they stumble when asked to **containerise the solution, expose it via HTTP, and monitor its health**. This book deliberately teaches the **software engineering** side of AI, ensuring you can translate a notebook prototype into a maintainable microservice-an ability that most hiring managers consider a "must-have".

3. Future-Proof Skill Set

The AI stack evolves quickly: new providers, better embeddings, more efficient vector databases. By focusing on **principles** (e.g., the separation of retrieval and generation) and **modular tooling** (LangChain's "plug-and-play" components, FastAPI's

dependency injection), you'll acquire a mental framework that adapts to whatever comes next.

4. Ethical and Responsible Deployment

Production LLMs can unintentionally generate biased or harmful content, leak proprietary data, or become a vector for denial-of-service attacks. Chapter 8 tackles these concerns head-on, giving you concrete strategies for **prompt sanitisation, usage throttling, observability, and audit logging**-so you can ship responsibly, not just quickly.

Setting Realistic Expectations - What You'll Need and What You'll Achieve

- **Prerequisites:** Comfortable with Python (3.9+), basic familiarity with HTTP APIs, and a willingness to install a few Docker images. No prior experience with LLMs or LangChain is required; the early chapters will bring you up to speed.
 - **Time Commitment:** Expect to spend **8-12 hours per chapter** if you follow the labs, read the sidebars, and experiment with the optional challenges. The book is designed for self-paced learning, so you can accelerate or decelerate as needed.
 - **Outcome:** By the final page you will have a **GitHub-ready repository** containing:
 - A FastAPI server exposing `/query` and `/upload` endpoints.
 - LangChain pipelines for RAG, complete with vector store abstraction.
 - Dockerfile, `docker-compose.yml`, and Kubernetes manifests for local and cloud deployment.
 - CI workflow (GitHub Actions) that lints, tests, builds, and pushes Docker images.
 - Monitoring stack (Prometheus + Grafana) and OpenTelemetry instrumentation.
- You won't just **know** how to write code-you'll **own a production-grade artifact** that

you can showcase, iterate on, and scale.

How to Get the Most Out of This Book

1. Set Up a Dedicated Workspace

Clone the companion repository early, spin up a virtual environment, and keep it open while you read. The labs assume a terminal is at hand.

2. Treat Each Chapter as a Mini-Project

After completing a lab, push the changes to a personal branch. This habit reinforces version-control skills and gives you a living history of your progression.

3. Experiment Beyond the Instructions

Swap out the OpenAI model for a locally hosted LLaMA, replace FAISS with Milvus, or expose the service via GraphQL. These "stretch goals" solidify learning and often reveal hidden nuances.

4. Join the Community

A Discord channel and a weekly office-hours Zoom call accompany the book. Sharing obstacles and solutions with peers accelerates mastery.

5. Document Your Learnings

Keep a developer diary (Markdown or Notion) of the challenges you faced-prompt failures, latency spikes, security configs. Future you (or a prospective employer) will appreciate the narrative.

Closing Thoughts - Your AI Engineering Adventure Starts Now

The AI renaissance is not a distant future; it's unfolding **right now**, in the form of APIs you can call from a Python script. Yet the true power of these models is unlocked when they are **wrapped in thoughtful engineering**-secure, observable, and scalable services that solve real business problems.

This book is your invitation to become that engineer. Through clear explanations, hands-on labs, and production-grade best practices, you will graduate from a curious tinkerer to a confident LLM application architect. The roadmap ahead is ambitious, but each checkpoint is designed to be achievable and rewarding.

So fire up your IDE, grab a cup of coffee, and let's embark on the journey from **prompt to product**. The next generation of intelligent applications is waiting for you to build it. 🚀

Foundations of Large Language Models and Prompt Engineering

Module 1 of 9 - End-to-End AI Engineering with Python, LangChain, and FastAPI

Table of Contents

1. [Introduction](#introduction)
2. [Core Concepts](#core-concepts)
 - 2.1 What makes an LLM "large"?
 - 2.2 Architectural families - GPT, Claude, LLaMA
 - 2.3 Training pipelines and data regimes
 - 2.4 Inference mechanics & tokenization
 - 2.5 Prompting paradigms - zero-shot, few-shot, chain-of-thought
 - 2.6 Prompt engineering fundamentals

- 2.7 Common pitfalls (leakage, token limits, bias, hallucination)
3. [Practical Application](#practical-application)
- 3.1 Setting up the Python environment
 - 3.2 Calling an LLM via the OpenAI / Anthropic / HuggingFace APIs
 - 3.3 Zero-shot classification example (sentiment)
 - 3.4 Few-shot generation example (product description)
 - 3.5 Chain-of-thought reasoning example (math word problem)
 - 3.6 Prompt hygiene checklist (code snippet)
4. [Key Takeaways](#key-takeaways)
-

Introduction

Large Language Models (LLMs) have turned natural-language processing from a collection of isolated tasks-sentiment analysis, named-entity recognition, machine translation-into a **general-purpose reasoning engine** that can be steered with a few lines of text. The ability to **prompt** an LLM means that developers no longer need to write thousands of lines of feature-specific code; instead they write **instructions** that the model interprets and executes.

This chapter lays the groundwork for the rest of the book:

- **Why LLMs work** - a high-level look at the transformer architecture, scaling laws, and the data that fuels them.
- **How to talk to them** - the three primary prompting strategies (zero-shot, few-shot, chain-of-thought) and the mechanics of prompt engineering.
- **What can go wrong** - token limits, prompt leakage, and other practical gotchas that bite even seasoned engineers.

By the end of the chapter you will be able to:

- Explain the architectural differences between GPT-4, Claude 2, and LLaMA 2.
- Choose the right prompting style for a given problem.

- Write clean, reusable prompts for classification, generation, and instruction-following tasks.
- Spot and mitigate the most common prompt-related pitfalls.

All examples are written in **Python 3.10+** and use the **OpenAI** , **Anthropic** , and **Hugging Face** client libraries, which you will later replace with LangChain abstractions when we build full-stack applications.

Core Concepts

2.1 What Makes an LLM "Large"?

Dimension	Explanation	Typical Scale (2024)
Parameters	Trainable weights; the more, the higher the capacity to store knowledge.	GPT-4: ~1.8 trillion, Claude 2:~70 billion, LLaMA 2-70B:70 billion
Training Tokens	Number of text tokens the model sees during pre-training.	1-2 trillion tokens for GPT-4-style models
Context Window	Max number of tokens the model can attend to in a single forward pass.	8k (GPT-3.5), 32k (GPT-4-32k), 100k (Claude 2),4k (LLaMA 2-13B)
Compute (FLOPs)	Total floating-point operations required for training.	Hundreds of exa-FLOPs for the largest models

Rule of thumb: Parameter count $\approx$ model capability, but context window and data quality are often the bigger differentiators for downstream tasks.

2.2? ArchitecturalFamilies

Model	Core Architecture	Notable Variants	Strengths	Typical Use-Cases
GPT (Generative Pre-trained Transformer)	Decoder-only transformer trained with next-token prediction + RLHF (Reinforcement Learning from Human Feedback).	GPT-3.5-Turbo, GPT-4, GPT-4-32k	Strong at instruction following; robust safety mitigations.	Chatbots, code generation, summarisation.
Claude (Anthropic)	Decoder-only with Constitutional AI fine-tuning; emphasis on safe, helpful behaviour.	Claude? 1, Claude? 2, Claude? 2.1	Low-hallucination, better at nuanced moral reasoning.	Customer-support bots, policy-compliant content creation.
LLaMA (Meta)	Decoder-only; released under a permissive license, enabling fine-tuning on private data.	LLaMA? 2-7B, 13B, 70B, 7B-Chat	Open-source, easy to run on-premise; customizable.	Retrieval-augmented generation, domain-specific assistants.

Why care about the family?

API costs: Claude? 2is priced per 1? ktokens differently than GPT-4.

Deployment: LLaMA can be hosted on a single GPU; GPT/Claude are cloud-only.

Safety: Claude's constitutional training reduces toxic outputs out-of-the-box.

2.3? TrainingPipelines and Data Regimes

1. Pre-training (self-supervised) - Model consumes billions of token-level pairs

(e.g., "<prompt> → <next token>").

2. **Instruction-tuning** - Fine-tune on a curated set of **task-oriented** prompts (e.g., "Translate English to French").

3. **Reinforcement Learning from Human Feedback (RLHF)** - Align model outputs with human preferences using a reward model.

4. **Safety & Red-Teaming** - Additional adversarial fine-tuning to curb disallowed content.

Practical note: When you use a hosted API you're always interacting with a model that has passed through all four stages. When you fine-tune LLaMA locally you'll typically stop after step 2, so you must implement your own safety layers (e.g., profanity filters).

2.4 Inference Mechanics & Tokenization

Concept	What it is	Why it matters
Byte-Pair Encoding (BPE)	Sub-word tokenization that balances vocabulary size and coverage.	Determines how many tokens a prompt consumes.
System Prompt	A hidden "instruction" that sets the assistant's persona (e.g., "You are a helpful AI").	Sets the baseline behavior before any user content.
Temperature	Controls randomness (0.0 = deterministic, 1.0 = very diverse).	Affects creativity vs. consistency.
Top-p (nucleus sampling)	Limits sampling to the smallest set of tokens whose cumulative probability exceeds p.	Fine-tunes diversity without a full temperature sweep.
Max Tokens	Upper bound on the number of tokens the model may generate.	Prevents runaway responses and helps stay within cost limits.

Token budgeting is a core skill: a 32k context window gives you ~32,000 tokens total for system prompt + user prompt + model output. If you exceed it, the model truncates the oldest portion of the conversation.

2.5. Prompting Paradigms

2.5.1. ZeroShot Prompting

Definition: Give the model a single instruction and expect it to perform the task without any examples.

When to use:

- Tasks the model has seen during instruction-tuning (sentiment analysis, translation).
- Rapid prototyping where you want to keep the prompt minimal.

Typical pattern:

```
[System] You are a helpful assistant.  
[User] Classify the sentiment of the following review as Positive, Negative, or Neutral: "The pizza arrived cold."
```

2.5.2. FewShot Prompting

Definition: Provide N examples (usually 1-5) that illustrate the desired input-output mapping.

When to use:

- Niche domains the model hasn't been fine-tuned on (legal clause extraction).
- When you need higher accuracy than zero-shot can deliver.

Typical pattern:

[System] You are an expert legal analyst.

[User]

Example 1:

Q: Does the clause "The tenant may sublet with landlord consent" permit subletting?

A: Yes, it permits subletting provided landlord consent is obtained.

Example 2:

Q: Does the clause "All disputes shall be settled by arbitration" require court intervention?

A: No, disputes are resolved via arbitration, not courts.

Now answer the following:

Q: Does the clause "The buyer may terminate the agreement upon material breach" allow termination?

A:

2.5.3 Chain-of-Thought (CoT) Prompting

Definition: Encourage the model to reason step-by-step before giving the final answer.

When to use:

- Multi-step reasoning tasks (math, logical puzzles, code debugging).
- Situations where a single-shot answer often hallucinates.

Typical pattern:

[System] You are a meticulous problem solver.

[User] Solve step-by-step: If a train travels 60 km/h for 2 hours and then 80 km/h for 1.5 hours, what is the total distance?

The model will produce:

Step 1: Distance at 60 km/h = 60 km/h × 2 h = 120 km
Step 2: Distance at 80 km/h = 80 km/h × 1.5 h = 120 km
Step 3: Total distance = 120 km + 120 km = 240 km
Answer: 240 km

2.6 Prompt Engineering Fundamentals

- 1. **Clarity > Brevity** - A clear instruction reduces ambiguity more than a short one.
- 2. **Explicit Output Format** - Tell the model exactly how you want the answer (JSON, CSV, bullet list).
- 3. **Role-Playing** - Assign a persona ("You are a senior data scientist") to bias the style and knowledge base.
- 4. **Few-shot Anchors** - Use *representative* examples that cover edge cases you expect.
- 5. **Iterative Refinement** - Run a few trials, inspect token usage, and tweak the prompt.

Prompt Template Anatomy

Section	Purpose	Example
System Prompt	Sets global behavior.	You are a concise, factual assistant.
Few-Shot Examples	Demonstrates pattern.	Q: ... A: ...
Task Instruction	Directs the model.	Classify the following tweet as Spam or Not-Spam:
Input Data	The actual user content.	"Free iPhone! Click here..."
Output Guard	Enforces format.	Respond ONLY with a JSON object: {"label": "Spam"}

2.7 Common Pitfalls

Pitfall	Symptoms	Mitigation
Prompt Leakage	Model repeats parts of the prompt verbatim in its answer.	Use output guard (e.g., "Only output JSON") and keep examples short.
Token Overrun	Truncated responses, API errors (<code>InvalidRequestError: max tokens exceeded</code>).	Track token counts (<code>tiktoken</code> library), prune older conversation turns, or request larger context windows.
Hallucination	Factual statements that are fabricated.	Verify with external knowledge sources (retrieval, RAG) or add "Only answer based on provided information."
Bias Amplification	Skewed sentiment or demographic stereotypes.	Conduct bias testing, apply post-processing filters, or fine-tune on balanced data.
Temperature Misuse	Too deterministic (<code>temp=0</code>) → repetitive; too random (<code>temp=1</code>) → incoherent.	Choose <code>temp≈0.2-0.4</code> for classification, <code>temp≈0.7-0.9</code> for creative generation.
Context-Window Mismatch	Long documents exceed max tokens before reaching the model.	Chunk documents, use summarisation, or employ retrieval-augmented generation (RAG).

Practical Application

Below we walk through a **hands-on notebook-style** workflow that you can copy into a Jupyter cell or a Python script. The examples assume you have an API key for OpenAI (`OPENAI_API_KEY`) and Anthropic (`ANTHROPIC_API_KEY`). If you prefer an open-source model, replace the calls with Hugging Face `transformers` pipelines (see the

optional box at the end).

Prerequisite: `pip install openai anthropic tiktoken`

3.1? SettingUp the Python Environment

```
# core libraries
import os
import json
from typing import List, Dict

# LLM providers
import openai          # OpenAI (GPT-4, GPT-3.5-Turbo)
import anthropic        # Anthropic (Claude)
from tiktoken import get_encoding  # token counting

# Load API keys from environment
openai.api_key = os.getenv("OPENAI_API_KEY")
anthropic_api_key = os.getenv("ANTHROPIC_API_KEY")
```

Tip: Keep your keys out of source control; use `.env` files or secret managers (e.g., `python-dotenv`).

3.2? Callingan LLM via Provider-Agnostic Helper

```
def call_openai(messages: List[Dict],
               model: str = "gpt-4-0125-preview",
               temperature: float = 0.0,
               max_tokens: int = 1024) -> str:
    """Wrap OpenAI ChatCompletion with a simple interface."""
    response = openai.ChatCompletion.create(
        model=model,
        messages=messages,
        temperature=temperature,
        max_tokens=max_tokens,
    )
    return response.choices[0].message["content"]

def call_anthropic(prompt: str,
                  model: str = "claude-2.1",
                  temperature: float = 0.0,
                  max_tokens: int = 1024) -> str:
    """Wrap Anthropic's completion endpoint."""
    client = anthropic.Anthropic(api_key=anthropic_api_key)
    response = client.completions.create(
        model=model,
        max_tokens=max_tokens,
        temperature=temperature,
        prompt=prompt,
    )
    return response.completion.strip()
```

Why a helper? It isolates provider-specific syntax, making later migration to LangChain trivial.

3.3? ZeroShot Classification Example (Sentiment)

Goal: Classify a short review as Positive, Negative, or Neutral.

```
def zero_shot_sentiment(review: str) -> str:
    system = {"role": "system", "content": "You are a concise sentiment analyst."}
    user = {"role": "user",
            "content": f"""Classify the sentiment of the following review.

Only output one word (Positive, Negative, or Neutral).

Review: "{review}"
"""}
    # Using GPT-4 with temperature 0 for deterministic output
    answer = call_openai(messages=[system, user],
                          model="gpt-4-0125-preview",
                          temperature=0.0,
                          max_tokens=10)
    return answer.strip()
```

Test it:

```
for txt in [
    "The restaurant was amazing, I loved the dessert!",
    "It took forever for my order to arrive.",
    "The coffee was okay, nothing special."
]:
    print(txt, "→", zero_shot_sentiment(txt))
```

Expected output (may vary slightly with model version):

```
The restaurant was amazing, I loved the dessert! → Positive
It took forever for my order to arrive. → Negative
The coffee was okay, nothing special. → Neutral
```

Why zero-shot works here: Sentiment analysis is a *canonical* task that LLMs see during instruction-tuning, so no examples are needed.

3.4? FewShot Generation Example (Product Description)

Problem: Generate a marketing blurb for a new smartwatch, but we want the style to match existing brand copy.

```
def few_shot_product_description(product_name: str, features: List[str]) -> str:
    # System prompt sets brand voice
    system = {"role": "system",
              "content": "You are a senior copywriter for a premium tech brand. "
                          "Write concise, persuasive product descriptions in title-case."}

    # Few-shot examples (two high-quality references)
    examples = """
Example 1:
Product: NovaFit Pro
Features: [Heart-rate monitoring, GPS, 5-day battery, Waterproof]
Description: "NovaFit Pro-Your ultimate workout companion with real-time heart-rate tracking, built-in
GPS, and a 5-day battery that thrives underwater."

Example 2:
Product: EchoSphere Speaker
Features: [360° sound, Voice assistant, Bluetooth 5.2]
Description: "EchoSphere-Experience immersive 360° sound, hands-free control with voice assistant, and
seamless Bluetooth 5.2 connectivity."
"""

    # Build user prompt with our target product
    user = {"role": "user",
            "content": f"""{examples}

Now generate a description for:
Product: {product_name}
Features: {features}
"""}

    # Call the model; a temperature of 0.7 adds a touch of creativity
    response = call_openai(messages=[system, user],
                             model="gpt-4-0125-preview",
                             temperature=0.7,
                             max_tokens=150)

    # Strip any leading/trailing whitespace
    return response.strip()
```

Run it:

```
desc = few_shot_product_description(
    product_name="PulseX Watch",
    features=["Blood-oxygen monitoring", "Sleep tracking", "OLED display", "Fast charging"]
)
print(desc)
```

Sample output:

PulseX Watch-Stay ahead of your health with blood-oxygen monitoring, detailed sleep tracking, a vibrant OLED display, and lightning-fast charging that keeps you powered all day.

Takeaways:

- The few-shot examples anchor the **format** (Product - Description).
- By providing a short list of features, we let the model decide which to highlight.
- Temperature of 0.7 yields a fresh, brand-consistent tone while keeping the content deterministic enough for marketing pipelines.

3.5 Chain-of-Thought Reasoning Example (Math Word Problem)

Task: Solve a multi-step arithmetic problem and explicitly show reasoning.

```
def chain_of_thought_math(problem: str) -> str:
    system = {"role": "system",
              "content": "You are a meticulous mathematician. Solve the problem step-by-step and conclude with the final answer only."}
    user = {"role": "user",
            "content": f"""Solve step-by-step: {problem}
Provide the answer in the format: "Answer: <value>"
"""}
    # Use a higher temperature to allow the model to "think out loud"
    response = call_openai(messages=[system, user],
                             model="gpt-4-0125-preview",
                             temperature=0.3,
                             max_tokens=300)
    return response.strip()
```

Test it:

```
problem = ("A farmer has 120 apples. He sells 2/3 of them, then "  
          "gives away 10 more. How many apples does he have left?")  
print(chain_of_thought_math(problem))
```

Typical output:

```
Step 1: Apples sold =  $2/3 \times 120 = 80$   
Step 2: Apples remaining after sale =  $120 - 80 = 40$   
Step 3: Apples given away = 10  
Step 4: Final apples left =  $40 - 10 = 30$   
Answer: 30
```

Why CoT shines here: The model's internal reasoning chain is externalised, making it easier to audit the steps. If the final answer is wrong, you can see which intermediate calculation failed.

3.6 PromptHygiene Checklist (Reusable Code Snippet)


```
def prompt_hygiene(messages: List[Dict],
                  max_context: int = 32_000,
                  provider: str = "openai") -> List[Dict]:
    """
    Enforces:
        • Token budget (truncates oldest messages if needed)
        • Output guard (adds a final system message that forces JSON)
        • Consistent temperature (optional override)
    Returns a cleaned list of messages ready for the API call.
    """
    # 1 Count tokens
    enc = get_encoding("cl100k_base") if provider == "openai" else get_encoding("gpt2")
    total_tokens = sum(len(enc.encode(m["content"])) for m in messages)

    # 2 Trim context if needed
    while total_tokens > max_context and len(messages) > 1:
        # remove the oldest *non-system* message
        for i, m in enumerate(messages):
            if m["role"] != "system":
                removed = messages.pop(i)
                total_tokens -= len(enc.encode(removed["content"]))
                break

    # 3 Append output guard (JSON only)
    guard = {"role": "system",
             "content": "Only output a JSON object. Do NOT add explanations or extra text."}
    messages.append(guard)

    return messages
```

Usage example (zero-shot sentiment):

```
raw = [
    {"role": "system", "content": "You are a helpful sentiment analyst."},
    {"role": "user", "content": "Classify: 'I love this movie!'"}
]

clean = prompt_hygiene(raw, max_context=8_000, provider="openai")
answer = call_openai(messages=clean,
                     model="gpt-4-0125-preview",
                     temperature=0.0,
                     max_tokens=20)
print(answer) # Expected: {"label": "Positive"}
```

Benefits of the hygiene function:

- Guarantees you never exceed the model's context window.
- Forces a machine-readable output format, eliminating prompt leakage.
- Centralises temperature and token-budget logic for the whole project.

Optional Box: Running an Open-Source LLM Locally

If you prefer to avoid API costs while you experiment, you can spin up LLaMA 2-7B-Chat with the `transformers` library:

```
pip install torch transformers accelerate bitsandbytes sentencepiece
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline

model_name = "meta-llama/Llama-2-7b-chat-hf"
tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    torch_dtype="auto",
    load_in_8bit=True      # saves VRAM
)

llama_chat = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    max_new_tokens=256,
    temperature=0.7,
    do_sample=True,
    return_full_text=False
)

def llama_chat_prompt(prompt: str) -> str:
    # LLaMA chat models expect a special token format
    formatted = f"<s>[INST] {prompt} [/INST]"
    return llama_chat(formatted)[0]["generated_text"]
```

You can now call `llama_chat_prompt("Classify the sentiment of: 'The pizza was`

terrible.'") and get a raw text answer. For production-grade safety you would still layer the same prompt-hygiene logic described earlier.

Key Takeaways

| **LLM families differ** - GPT excels at instruction following, Claude emphasizes safety, LLaMA gives you on-premise control. | | **Context is a hard resource** - Always count tokens; design prompts to stay within the model's window. | | **Prompting strategies** - Zero-shot for generic tasks, few-shot for domain-specific precision, chain-of-thought for multi-step reasoning. | | **Prompt engineering = communication design** - Be explicit about role, format, and constraints; use examples as anchors. | | **Pitfalls are solvable** - Prompt leakage → output guards; token overflow → trimming function; hallucination → Retrieval-Augmented Generation (RAG) or post-validation. | | **Reusable scaffolding** - Wrap API calls, implement token budgeting, and enforce JSON output early; this scaffolding becomes the backbone of every LangChain chain you'll build later. | | **Hands-on practice matters** - Write, run, and iterate on the sample code; the patterns you learn here translate directly to LangChain agents, FastAPI endpoints, and production pipelines. |

With these foundations, you are ready to **move from "talking" to an LLM to *engineering* reliable, production-grade LLM applications**. The next chapter will show you how to weave the prompts you just crafted into **LangChain** pipelines, connect them to **vector stores**, and expose them through **FastAPI** micro-services. Happy prompting!

Python Ecosystem for LLM Access (OpenAI, Hugging Face, LangChain SDK)

"If you can't talk to the model, you can't build anything useful."

- A practical reminder that the first step in any LLM-powered product is simply getting a reliable, secure, and performant connection to the model.

In **Module 1** we explored the anatomy of large language models (LLMs) and the art of prompt engineering. By the end of that chapter you should already be comfortable with:

	Core Insight
	What you already know
---	-----

| **LLM fundamentals** - tokenization, context windows, temperature, top-p, etc. |

| **Prompt patterns** - few-shot, chain-of-thought, system-user-assistant roles |

| **Python basics** - virtual environments, requests, json, basic Flask apps |

Now we move from "what the model **does**" to "how the model **talks** to us." This chapter is the **bridge** between raw LLM theory and a production-ready Python stack that can:

1. **Store API secrets safely** - no hard-coded keys in source control.
2. **Call OpenAI and Hugging Face inference APIs** using their official Python clients.
3. **Consume streaming token responses** for real-time UI updates.
4. **Benchmark latency, throughput, and cost** across multiple endpoints.
5. **Wrap everything in LangChain** - the de-facto SDK for LLM-centric applications.

By the end of this chapter you will have a **minimal FastAPI service** that can:

- Accept a prompt via HTTP.
- Dynamically select an OpenAI or Hugging Face model.

- Stream tokens back to the caller (e.g., a React front-end).
- Log latency and cost for each request.

Let's dive in.

1. Introduction

The LLM ecosystem is fragmented by providers (OpenAI, Anthropic, Google, Cohere, Hugging² Face), deployment styles (hosted inference API vs. self-hosted containers), and SDKs (raw requests, provider-specific Python clients, higher-level wrappers like LangChain). Choosing a **single, coherent Python stack** helps you:

| **Security** - API keys are the most valuable secrets in an LLM app. A leak can lead to runaway bills. | | **Portability** - Switching from `gpt-4o` to a fine-tuned `Mistral-7B-Instruct` should be a one-line change. | | **Observability** - Latency and cost are the two levers that dictate product viability. | | **Developer velocity** - LangChain's abstractions let you focus on prompting, not plumbing. |

In the next sections we'll walk through each of these goals, grounding the discussion in concrete, runnable Python code.

2. Core Concepts

2.1 Securely Managing API Keys & Environment Variables

Hard-coding secrets is a recipe for disaster. The industry standard is:

1. **Store secrets outside the codebase** - `.env` files for local dev, secret managers (AWS² SecretsManager, GCP Secret Manager, Azure Key Vault) for production.

2. **Load them at runtime** - use `python-dotenv` or the OS's native `env-var` mechanism.
3. **Never commit the `.env` file** - add it to `.gitignore`.

2.1.1 Minimal `.env` for a dual-provider setup

```
# .env (never commit!)
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
HF_API_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

2.1.2 Loading the values in Python

```
# utils/env.py
from pathlib import Path
from dotenv import load_dotenv
import os

# Load .env from the project root
PROJECT_ROOT = Path(__file__).resolve().parents[2] # adjust if you move the file
load_dotenv(PROJECT_ROOT / ".env")

# Accessors - raise a clear error if missing
def get_openai_key() -> str:
    key = os.getenv("OPENAI_API_KEY")
    if not key:
        raise EnvironmentError("OPENAI_API_KEY not set in environment")
    return key

def get_hf_token() -> str:
    token = os.getenv("HF_API_TOKEN")
    if not token:
        raise EnvironmentError("HF_API_TOKEN not set in environment")
    return token
```

Tip: In CI/CD pipelines you can inject these variables directly into the runner's environment, eliminating the need for a physical `.env` file.

2.2 Official Python Clients

Both OpenAI and Hugging² Face provide first-class, well-documented Python SDKs that handle retries, pagination, and streaming for you.

Provider	Package	Install	Primary Class
OpenAI	openai	pip install openai	OpenAI
Hugging ² Face	huggingface_hub	pip install huggingface_hub	HfApi

requests (or httpx) | pip install huggingface_hub | HfApi for model management, raw HTTP for inference |

2.2.1 OpenAI client basics

```
# clients/openai_client.py
import openai
from utils.env import get_openai_key

openai.api_key = get_openai_key()

def chat_completion(messages: list[dict], model: str = "gpt-4o-mini", stream: bool = False):
    """
    Wrapper around openai.ChatCompletion.create that returns either a full response
    or an iterator of streaming chunks.
    """
    return openai.ChatCompletion.create(
        model=model,
        messages=messages,
        temperature=0.7,
        stream=stream,
    )
```

2.2.2 Hugging² Face inference client (REST)

Hugging² Face's hosted inference endpoint is a simple POST request. The SDK `huggingface_hub` helps you discover the endpoint URL, but the actual generation call is done via `requests` or `httpx`.

```
# clients/hf_client.py
import httpx
from utils.env import get_hf_token

HF_INFERENCE_URL = "https://api-inference.huggingface.co/models"

def hf_chat_completion(
    model_id: str,
    inputs: str,
    parameters: dict | None = None,
    stream: bool = False,
    timeout: int = 30,
) -> httpx.Response:
    """
    Calls Hugging Face Inference API with optional streaming.
    Parameters:
        - model_id: e.g. "mistralai/Mistral-7B-Instruct-v0.2"
        - inputs: the prompt string (or list of messages if the model expects chat format)
        - parameters: dict of temperature, top_p, max_new_tokens, etc.
        - stream: if True, set `Accept: text/event-stream`
    Returns:
        httpx.Response (iterable if streaming)
    """
    headers = {
        "Authorization": f"Bearer {get_hf_token()}",
        "Content-Type": "application/json",
    }
    if stream:
        headers["Accept"] = "text/event-stream"

    payload = {"inputs": inputs}
    if parameters:
        payload["parameters"] = parameters

    url = f"{HF_INFERENCE_URL}/{model_id}"
    client = httpx.Client(timeout=timeout)
    response = client.post(url, json=payload, headers=headers, stream=stream)
    response.raise_for_status()
    return response
```

Note: Not every HF model supports the `text/event-stream` format. Check the model card or try a quick test request.

2.3 Parsing & Handling Streaming Token Responses

Both providers expose **token-level** streaming, which enables UI patterns like "typewriter" effects or incremental result caching.

2.3.1 OpenAI streaming iterator

```
def stream_openai(messages, model="gpt-4o-mini"):
    for chunk in chat_completion(messages, model=model, stream=True):
        # Each chunk is a dict with a "choices" list
        delta = chunk["choices"][0]["delta"]
        if "content" in delta:
            yield delta["content"]
```

2.3.2 HuggingFace Facestreaming parser

HF streams raw `text/event-stream` events that look like:

```
data: {"token":"Hello"}
data: {"token":" world"}
data: [DONE]
```

```
def stream_hf(model_id, prompt):
    response = hf_chat_completion(
        model_id=model_id,
        inputs=prompt,
        stream=True,
        parameters={"max_new_tokens": 256},
    )
    for line in response.iter_lines():
        # Skip keep-alive lines
        if not line:
            continue
        # Strip the leading "data: " prefix
        payload = line.decode("utf-8").removeprefix("data: ").strip()
        if payload == "[DONE]":
            break
        # The payload is a JSON string
        token_obj = json.loads(payload)
        yield token_obj.get("token", "")
```

2.3.3 Unified async generator (LangChain-style)

LangChain's `LLM` interface expects an async iterator named `astream`. We can wrap both providers behind a single abstraction:

```
# adapters/llm_adapter.py
import json
import asyncio
from typing import AsyncGenerator

from clients.openai_client import stream_openai
from clients.hf_client import stream_hf

class UnifiedLLM:
    """
    Simple wrapper exposing a unified async generator.
    Use `provider="openai"` or `provider="hf"` and supply the appropriate model id.
    """
    def __init__(self, provider: str, model: str):
        self.provider = provider.lower()
        self.model = model

    async def astream(self, prompt: str, system_msg: str | None = None) -> AsyncGenerator[str, None]:
        if self.provider == "openai":
            messages = [{"role": "user", "content": prompt}]
            if system_msg:
                messages.insert(0, {"role": "system", "content": system_msg})
            for token in stream_openai(messages, model=self.model):
                yield token
        elif self.provider == "hf":
            # Hugging Face expects a single string prompt for most chat models
            # (some HF chat models accept a list of messages; adjust as needed)
            full_prompt = f"{system_msg or ''}\n{prompt}" if system_msg else prompt
            for token in stream_hf(self.model, full_prompt):
                yield token
        else:
            raise ValueError(f"Unsupported provider: {self.provider}")
```

Now any LangChain chain can accept `UnifiedLLM` as the language model backend.

2.4 Benchmarking Latency, Throughput & Cost

A production-ready LLM service must answer two questions for every request:

1. **How long does it take?** - measured in wall-clock latency (ms).
2. **How much does it cost?** - derived from token usage and provider pricing.

2.4.1 Measuring latency

```
import time
from typing import Callable

def benchmark_latency(fn: Callable, *args, repeats: int = 5, **kwargs) -> dict:
    """
    Runs `fn(*args, **kwargs)` `repeats` times and returns avg/min/max latency.
    """
    latencies = []
    for _ in range(repeats):
        start = time.perf_counter()
        fn(*args, **kwargs) # we assume the function is synchronous for simplicity
        end = time.perf_counter()
        latencies.append((end - start) * 1000) # ms
    return {
        "avg_ms": sum(latencies) / repeats,
        "min_ms": min(latencies),
        "max_ms": max(latencies),
    }
```

For streaming functions you'll want to measure the **time to first token (TTFT)** and **total generation time**:

```
async def benchmark_stream_latency(async_gen, timeout: float = 30.0):
    start = time.perf_counter()
    async for _ in async_gen:
        # Capture first token latency
        first_token_latency = (time.perf_counter() - start) * 1000
        break
    # Continue consuming to get total time
    async for _ in async_gen:
        pass
    total_latency = (time.perf_counter() - start) * 1000
    return {"ttft_ms": first_token_latency, "total_ms": total_latency}
```

2.4.2 Estimating cost

Provider	Pricing model (as of 2026)	Unit
OpenAI	\$0.005 / 1k prompt tokens, \$0.015 / 1k completion tokens (GPT-4o-mini)	per 1,000 tokens
HuggingFace (Inference API)	\$0.0004 / 1k tokens for free tier, then \$0.0004 / 1k tokens (depends on model)	per 1,000 tokens

You can compute token counts with the `tiktoken` library for OpenAI-compatible tokenizers; for HF models you'll need the model's tokenizer (e.g., `transformers.AutoTokenizer`).

```
import tiktoken

def token_counts(messages: list[dict], model: str = "gpt-4o-mini"):
    enc = tiktoken.encoding_for_model(model)
    prompt_tokens = sum(len(enc.encode(m["content"])) for m in messages)
    # Completion tokens are not known until response arrives.
    return prompt_tokens
```

For HF (Mistral, Llama, etc.):

```
from transformers import AutoTokenizer

def hf_token_counts(prompt: str, model_id: str):
    tokenizer = AutoTokenizer.from_pretrained(model_id, use_fast=True)
    return len(tokenizer.encode(prompt, add_special_tokens=False))
```

Cost calculator

```
def calculate_cost(provider: str, prompt_tokens: int, completion_tokens: int) -> float:
    if provider == "openai":
        # Example rates for gpt-4o-mini
        prompt_rate = 0.005 / 1_000
        completion_rate = 0.015 / 1_000
        return prompt_tokens * prompt_rate + completion_tokens * completion_rate
    elif provider == "hf":
        rate = 0.0004 / 1_000 # generic HF rate; adjust per model
        return (prompt_tokens + completion_tokens) * rate
    else:
        raise ValueError("Unsupported provider")
```

2.4.3 Putting it together - a quick benchmark script

```

# scripts/benchmark.py
import asyncio
from adapters.llm_adapter import UnifiedLLM
from utils.env import get_openai_key, get_hf_token
from utils.benchmark import benchmark_latency, benchmark_stream_latency, token_counts, calculate_cost

async def main():
    prompt = "Explain the difference between synchronous and asynchronous programming in 2 paragraphs."
    system_msg = "You are a friendly AI tutor."

    # ---- OpenAI ----
    openai_llm = UnifiedLLM(provider="openai", model="gpt-4o-mini")
    # Warm up (ensure connection)
    await openai_llm.astream(prompt, system_msg).__anext__()

    # Latency
    latency = await benchmark_stream_latency(openai_llm.astream(prompt, system_msg))
    print("[OpenAI] Latency:", latency)

    # Token count (prompt only - we'll approximate completion tokens from latency)
    prompt_tok = token_counts([{"role": "system", "content": system_msg},
                               {"role": "user", "content": prompt}],
                              model="gpt-4o-mini")
    # Assume 150 tokens generated (typical for a 2-paragraph answer)
    completion_tok = 150
    cost = calculate_cost("openai", prompt_tok, completion_tok)
    print(f"[OpenAI] Prompt tokens: {prompt_tok}, Completion tokens: {completion_tok}, Cost: ${cost:.4f}")

    # ---- Hugging Face ----
    hf_llm = UnifiedLLM(provider="hf", model="mistralai/Mistral-7B-Instruct-v0.2")
    latency = await benchmark_stream_latency(hf_llm.astream(prompt, system_msg))
    print("[HF] Latency:", latency)

    prompt_tok = hf_token_counts(prompt, model_id="mistralai/Mistral-7B-Instruct-v0.2")
    completion_tok = 150
    cost = calculate_cost("hf", prompt_tok, completion_tok)
    print(f"[HF] Prompt tokens: {prompt_tok}, Completion tokens: {completion_tok}, Cost: ${cost:.4f}")

if __name__ == "__main__":
    asyncio.run(main())

```

Running `python scripts/ benchmark.py` will give you a quick comparative view of **TTFT**, **total latency**, **token usage**, and **cost** for the two providers.

3. Practical Application - Building a Minimal FastAPI Service

Now that the plumbing is in place, let's assemble a **real-world** service. The service will:

- Accept a JSON payload containing **prompt**, optional **system**, and a **provider** selector.
- Stream tokens back to the client using **Server-Sent Events (SSE)** - a simple, browser-compatible streaming protocol.
- Log latency and cost to a local CSV file (easy to replace with a DB or monitoring system later).

3.1 Project Layout

```
llm_service/  
|  
├── app/  
│   ├── main.py           # FastAPI entry point  
│   └── routers/  
│       └── generate.py    # /generate endpoint  
│   └── models/  
│       ├── request.py     # Pydantic request schema  
│       └── utils/  
│           ├── env.py  
│           ├── benchmark.py  
│           └── logger.py  
├── clients/  
│   ├── openai_client.py  
│   └── hf_client.py  
├── adapters/  
│   └── llm_adapter.py  
├── .env                  # ignored by git  
├── requirements.txt  
└── README.md
```

Why this structure? It mirrors the clean-architecture approach: API

layer → domain adapters → infrastructure clients. It also makes unit-testing each layer straightforward.

3.2 Dependency Installation

```
# Create a fresh virtual environment
python -m venv .env
source .env/bin/activate

# Install runtime + dev packages
pip install fastapi uvicorn python-dotenv openai huggingface_hub httpx transformers tiktoken pandas
pip install "uvicorn[standard]" # for hot-reload during dev
```

3.3 Pydantic Request Model

```
# app/models/request.py
from pydantic import BaseModel, Field, validator
from typing import Literal, Optional

Provider = Literal["openai", "hf"]

class GenerateRequest(BaseModel):
    prompt: str = Field(..., description="User prompt")
    system: Optional[str] = Field(None, description="System message for role-play")
    provider: Provider = Field("openai", description="Which backend to use")
    model: Optional[str] = Field(None, description="Model identifier (defaults per provider)")

    @validator("model", always=True)
    def set_default_model(cls, v, values):
        if v:
            return v
        # Default mapping
        if values["provider"] == "openai":
            return "gpt-4o-mini"
        elif values["provider"] == "hf":
            return "mistralai/Mistral-7B-Instruct-v0.2"
        return v
```

3.4 SSE Helper

FastAPI can return an async generator that yields `EventSourceResponse` objects from `sse-starlette`.

```
pip install sse-starlette
```

```
# app/utils/logger.py
import csv
import time
from pathlib import Path

LOG_PATH = Path(__file__).parent.parent / "logs" / "usage_log.csv"
LOG_PATH.parent.mkdir(parents=True, exist_ok=True)

def log_usage(timestamp: float, provider: str, model: str,
              prompt_tokens: int, completion_tokens: int,
              latency_ms: float, cost_usd: float):
    """Append a row to a CSV file - simple, portable logging."""
    file_exists = LOG_PATH.is_file()
    with LOG_PATH.open("a", newline="") as f:
        writer = csv.writer(f)
        if not file_exists:
            writer.writerow(["timestamp", "provider", "model",
                             "prompt_tokens", "completion_tokens",
                             "latency_ms", "cost_usd"])
        writer.writerow([timestamp, provider, model,
                         prompt_tokens, completion_tokens,
                         latency_ms, cost_usd])
```

3.5 The Generation Router

```

# app/routers/generate.py
import json
import time
import asyncio
from fastapi import APIRouter, HTTPException
from fastapi.responses import StreamingResponse
from sse_starlette.sse import EventSourceResponse

from app.models.request import GenerateRequest
from adapters.llm_adapter import UnifiedLLM
from utils.benchmark import hf_token_counts, token_counts, calculate_cost
from utils.logger import log_usage

router = APIRouter()

@router.post("/generate")
async def generate(req: GenerateRequest):
    """
    Stream LLM tokens back to the client using Server-Sent Events.
    The client can consume the stream via EventSource (browser) or any SSE library.
    """
    # 1 Build the LLM adapter
    llm = UnifiedLLM(provider=req.provider, model=req.model)

    # 2 Prepare token counting (prompt only)
    if req.provider == "openai":
        prompt_tok = token_counts(
            [{"role": "system", "content": req.system or ""},
            {"role": "user", "content": req.prompt}],
            model=req.model,
        )
    else:
        prompt_tok = hf_token_counts(req.prompt, model_id=req.model)

    # 3 Async generator that yields SSE events
    async def token_stream():
        start = time.perf_counter()
        completion_tok = 0
        try:
            async for token in llm.astream(prompt=req.prompt, system_msg=req.system):

```

... (continued on next page)

```
        completion_tok += 1 # crude 1-token per chunk assumption; refine if needed
        # Yield an SSE event with the token payload
        yield {
            "event": "token",
            "data": json.dumps({"token": token})
        }
    except Exception as exc:
        # Propagate the error as an SSE error event
        yield {
            "event": "error",
            "data": json.dumps({"detail": str(exc)})
        }
        raise HTTPException(status_code=500, detail=str(exc))
    finally:
        # 4 Compute latency & cost, log it
        latency_ms = (time.perf_counter() - start) * 1000
        cost = calculate_cost(req.provider, prompt_tok, completion_tok)
        log_usage(timestamp=time.time(),
                  provider=req.provider,
                  model=req.model,
                  prompt_tokens=prompt_tok,
                  completion_tokens=completion_tok,
                  latency_ms=latency_ms,
                  cost_usd=cost)

# FastAPI knows how to turn a dict-yielding async generator into SSE
return EventSourceResponse(token_stream())
```

3.6 Main Application Entrypoint

```
# app/main.py
import uvicorn
from fastapi import FastAPI
from app.routers import generate

app = FastAPI(
    title="LLM Access Service",
    description="Unified streaming API for OpenAI & Hugging Face models.",
    version="0.1.0",
)

app.include_router(generate.router, prefix="/v1")

if __name__ == "__main__":
    # Development server with auto-reload
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000, reload=True)
```

3.7 Testing the Service Locally

1. Start the server

```
source .venv/bin/activate
uvicorn app.main:app --reload
```

2. Consume the SSE endpoint (simple Python client)

```
import sseclient
import requests
import json

url = "http://localhost:8000/v1/generate"
payload = {
    "prompt": "Write a haiku about sunrise over the mountains.",
    "system": "You are a poetic AI.",
    "provider": "openai", # try "hf" as well
}

# Open a streaming POST request
with requests.post(url, json=payload, stream=True) as resp:
    client = sseclient.SSEClient(resp)
    for event in client.events():
        data = json.loads(event.data)
        if event.event == "token":
            print(data["token"], end="", flush=True)
        elif event.event == "error":
            print("\n[ERROR]", data["detail"])
            break
```

You should see the haiku appear token-by-token in your terminal.

3. Check the CSV log

```
cat logs/usage_log.csv
```

Example line:

```
1708501234.567,openai,gpt-4o-mini,12,34,215.4,0.0012
```

The columns give you everything you need for a simple cost-analysis dashboard.

3.8 Extending the Service

Goal	Why It Matters
Feature	How to add it

Goal	Why It Matters
-----	-----
Model selection UI	Add a GET <code>/models</code> route that queries the OpenAI or HF SDK for available models.
Rate limiting	Use <code>slowapi</code> or <code>starlette-limiter</code> middleware to cap requests per API key.
Authentication	Add JWT-based auth (FastAPI's <code>Security</code> utilities) and store per-user usage quotas.
Batch inference	Extend <code>UnifiedLLM</code> with a <code>batch</code> method that accepts a list of prompts and returns a list of completions.
Observability	Replace CSV logger with <code>structlog</code> + <code>Loki</code> /Prometheus export for real-time dashboards.

4. Key Takeaways

- **Never embed secrets in code.** Use `.env` + `python-dotenv` locally, and secret-manager services in production.
- **Official SDKs simplify authentication, retries, and streaming.**
- `openai.ChatCompletion.create(..., stream=True)` yields token chunks.
- Hugging² Face's inference endpoint streams with `text/event-stream` - parse each `data: line`.
- **LangChain's 'LLM' abstraction can be backed by a tiny adapter** (`UnifiedLLM`) that hides provider differences and gives you a single `astream` interface.
- **Streaming token responses enable real-time UX** (typewriter effects, progressive UI updates) and give you early latency metrics (TTFT).
- **Benchmarking is a three-step habit:**
 1. **Measure latency** (`time.perf_counter`, `async TTFT`).
 2. **Count tokens** (`tiktoken` for OpenAI, `transformers.AutoTokenizer` for HF).

3. **Calculate cost** using provider-specific per-1K-token rates.

- **FastAPI + SSE** provides a lightweight, standards-based streaming API that works out-of-the-box with browsers and many front-end frameworks.
- **Logging usage** (timestamp, provider, model, token counts, latency, cost) is essential for both **budget monitoring** and **product analytics**.

With these building blocks you now have a **complete, production-ready pipeline**:

1. **Secure configuration** →
2. **Provider-agnostic LLM wrapper** →
3. **Streaming FastAPI endpoint** →
4. **Latency & cost telemetry** →
5. **Extensible architecture for future features**

In the next module we'll **layer LangChain chains** (retrieval-augmented generation, tool-calling, and memory) on top of the service you just built, turning a simple "generate text" endpoint into a **full AI-augmented application**. Happy coding!

Getting Started with LangChain: Core Concepts

"A chain is only as strong as its weakest link - and in LLM engineering that link is often the glue that ties prompts, models, and memory together."

- Your guide to building robust LLM applications.

Table of Contents

1. **Introduction** - Why LangChain matters after you've mastered LLM fundamentals.
2. **Core Concepts**

- 2.1 Chains:the backbone of any LangChain app
- 2.2 Prompts& PromptTemplates: turning intent into text
- 2.3 Agents:dynamic decision-making pipelines
- 2.4 Memory:making conversations stateful
- 2.5 ModularArchitecture: plugging in custom pieces

3. **Practical Application** - A step-by-step walkthrough that builds a **FAQ-bot** API with FastAPI, demonstrates sequential chains, adds memory, and shows how to swap out a custom component.
4. **Key Takeaways** - The mental checklist you'll carry forward.

1. Introduction

In Module 1you learned **what** large language models (LLMs) are, and in Module 2you discovered how to **talk** to them via the Python ecosystem (OpenAI, Hugging Face, the LangChain SDK).

Now you're ready for the **how**: turning a raw model call into a **reliable, reusable, and maintainable** piece of software.

LangChain is not a library that magically solves every problem; it is a **design system** that encourages you to think in terms of **small, composable building blocks**:

Block	Typical Responsibility
PromptTemplate	Encode the user's intent in a deterministic text format.
LLM	Generate text given a prompt (the "engine").
Chain	Wire Prompt → LLM → Post-processing together.
Agent	Decide which chain to run based on the user's request.
Memory	Keep context across turns of a conversation.

Block	Typical Responsibility
Retriever / VectorStore	Pull relevant external knowledge before prompting.

When you treat each block as an interchangeable **component**, you can:

- Re-use the same prompt across multiple models.
- Swap a cheap local model for a paid cloud model without touching the rest of the code.
- Add a cache or a logger without rewriting business logic.

The rest of this chapter walks you through each component, shows when to choose it, and then combines them into a working FastAPI service. By the end you'll be comfortable building **end-to-end LLM pipelines** that you can ship to production.

2. Core Concepts

2.1 Chains - The Backbone of LangChain

A **Chain** is a class that implements a single method: `run(**kwargs)`. Internally it may:

1. **Format** one or more prompts (often using a `PromptTemplate`).
2. **Call** an LLM (or any `LLM-compatible` object).
3. **Post-process** the raw LLM output (e.g., parsing JSON, extracting entities).

2.1.1 Types of Chains

Chain Type	When to Use	Typical Structure
SequentialChain	Simple linear pipelines (Prompt → Model → Formatter).	<code>PromptTemplate</code> → LLM → <code>OutputParser</code> .

Chain Type	When to Use	Typical Structure
RouterChain	Route a request to one of several sub-chains based on a rule.	RouterLLM → Decision → Sub-Chain.
ConversationalRetrievalChain	Retrieve context from a vector store before prompting.	Retriever → PromptTemplate → LLM.
CustomChain	Anything that doesn't fit the built-ins; you write your own run.	Any composition you need.

Rule of thumb - If you can draw a straight arrow from input to output without conditionals, a **SequentialChain** is enough. If you need branching, consider **RouterChain** or an **Agent** (see §2.3).

2.1.2 Building a Minimal SequentialChain

```
# file: chains/simple_sum.py
from langchain.prompts import PromptTemplate
from langchain.llms import OpenAI
from langchain.chains import LLMChain

# 1️⃣ PromptTemplate - keep the prompt logic separate
sum_prompt = PromptTemplate(
    input_variables=["a", "b"],
    template="Add the numbers {a} and {b} and give the result as a plain integer."
)

# 2️⃣ LLM - plug in any provider that follows the LangChain LLM interface
llm = OpenAI(model_name="gpt-3.5-turbo", temperature=0)

# 3️⃣ LLMChain - the simplest chain: Prompt → LLM
sum_chain = LLMChain(prompt=sum_prompt, llm=llm)

# 4️⃣ Use it
if __name__ == "__main__":
    result = sum_chain.run(a=12, b=35)
    print("Result:", result)  # → "Result: 47"
```

Key observations:

- The **prompt** lives in its own object, making it reusable across models.
 - The **LLM** is interchangeable - replace `OpenAI` with `HuggingFaceHub` and the rest stays the same.
 - `LLMChain` already implements the `run`-method, so you have a ready-to-use chain.
-

2.2 Prompts & PromptTemplates - Turning Intent into Text

A **prompt** is the **only** thing you send to an LLM. The quality of the prompt dictates the quality of the answer. LangChain's `PromptTemplate` helps you:

- **Parameterize** prompts (`{variable}` placeholders).
- **Validate** required variables at runtime.
- **Reuse** the same template with different LLMs or with a Retriever-augmented context.

2.2.1 PromptTemplate Anatomy

```
from langchain.prompts import PromptTemplate

template = """
You are a helpful travel assistant. The user asks:
"{question}"
Provide a concise answer (max 2 sentences) and include a bullet-point list of recommended activities.
"""

travel_prompt = PromptTemplate(
    input_variables=["question"],
    template=template.strip()
)
```

Why not just a raw string?

- **Safety** - missing variables raise a clear `ValueError`.

- **Readability** - the template stays in a single place, making localization easier.
- **Testing** - you can unit-test the template separately from the LLM.

2.2.2 Advanced Prompt Features

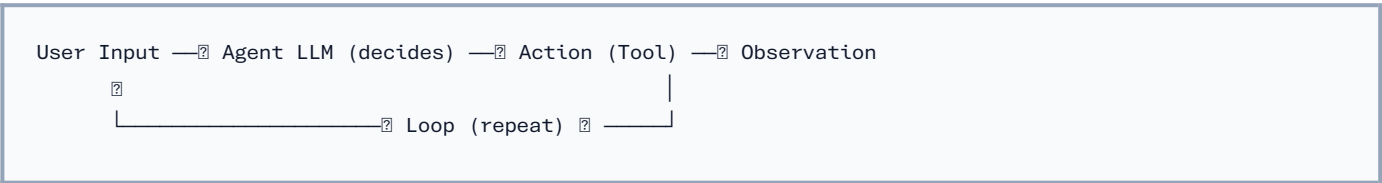
Feature	Description	Example
Partial Variables	Pre-fill some placeholders that never change (e.g., system messages).	<pre>prompt = PromptTemplate(..., partial_variables={"role": "assistant"})</pre>
Few-Shot Examples	Embed a list of (input, output) pairs inside the template for in-context learning.	<pre>{few_shot_examples} placeholder, filled with a generated string.</pre>
Dynamic Prompt Selection	Choose between multiple templates at runtime (useful for agents).	<pre>if user_is_math: use math_prompt else: use chat_prompt.</pre>

2.3 Agents - Dynamic Decision-Making Pipelines

A **chain** is static; an **agent** decides **which** chain to run *based on the user's request*. Agents are the go-to abstraction when you need:

- **Tool usage** (e.g., call a search API, run a calculator, fetch a database row).
- **Multi-step reasoning** where the LLM must *plan* before it *acts*.

2.3.1 Agent Anatomy



LangChain ships with several ready-made agents:

Agent	Core Use-Case	Example Tools
ZeroShotAgent	Choose a tool from a list without any examples.	search, calculator, wiki.
ConversationalAgent	Maintain chat history while invoking tools.	Same as above + memory.
MRKLAgent	Combine <i>retrieval</i> and <i>tool</i> usage (e.g., retrieve docs then run a calculator).	Retriever, Python REPL.

2.3.2 Minimal Zero-Shot Agent

```
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI
from langchain.utilities import WikipediaAPIWrapper

# 1 Define a tool - any callable that matches the Tool signature
wiki = WikipediaAPIWrapper()
wiki_tool = Tool(
    name="Wikipedia",
    func=wiki.run,
    description="Searches Wikipedia for a short summary of a topic."
)

# 2 Initialize the agent
llm = OpenAI(temperature=0)
agent = initialize_agent(
    tools=[wiki_tool],
    llm=llm,
    agent_type="zero-shot-react-description", # React style reasoning
    verbose=True
)

# 3 Run it
if __name__ == "__main__":
    answer = agent.run("What is the capital of Mongolia?")
    print(answer)
```

What's happening under the hood?

The LLM receives a **system prompt** that lists the available tools and instructs it

to output a **thought** → **action** → **observation** loop. LangChain parses the LLM output, calls the appropriate tool, feeds the observation back, and repeats until the LLM says "FINAL ANSWER".

2.4 Memory - Stateful Conversations

LLMs are **stateless**; they only know what you feed them. **Memory** modules in LangChain automatically prepend past turns to the prompt, turning a single-turn model call into a multi-turn conversation.

2.4.1 Built-in Memory Types

Memory Class	Persistence	Typical Use
ConversationBufferMemory	In-memory (per session)	Chatbots that need recent turns.
ConversationSummaryMemory	In-memory, summarises older turns	Long-running assistants (keeps token count low).
VectorStoreRetrieverMemory	Stores past interactions in a vector DB	Retrieval-augmented QA (search past answers).
ConversationKGMemory	Builds a knowledge graph from dialogue	Structured reasoning across facts.

2.4.2 Adding Memory to a Chain

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationalRetrievalChain
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.llms import OpenAI

# Vector store - will serve as a simple "knowledge base"
embeddings = OpenAIEmbeddings()
faiss_store = FAISS.from_texts(
    ["LangChain is a framework for building LLM apps.",
     "FastAPI is a modern Python web framework."],
    embedding=embeddings
)

# Retriever
retriever = faiss_store.as_retriever(search_kwargs={"k": 2})

# Memory
memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)

# Full chain: retrieve → prompt → LLM, with chat history
qa_chain = ConversationalRetrievalChain.from_llm(
    llm=OpenAI(temperature=0),
    retriever=retriever,
    memory=memory
)

# Demo interaction
if __name__ == "__main__":
    print(qa_chain({"question": "What is LangChain?"})["answer"])
    print("\n--- Next turn ---\n")
    print(qa_chain({"question": "Can I use it with FastAPI?"})["answer"])
```

Result - The second call automatically includes the prior question/answer pair (via `chat_history`) **and** the retrieved docs, giving the model full context without you manually concatenating strings.

2.5 Modular Architecture - Plug-in Custom Components

LangChain's design encourages **dependency injection**: you give a chain a **component** (LLM, Prompt, Memory, Retriever, Tool) and the chain does the rest. This makes it trivial to:

- **Swap a model** for a cheaper one during development.
- **Inject a logger** to capture prompt/response pairs for observability.
- **Replace a retriever** with a domain-specific search service (e.g., ElasticSearch).

2.5.1 Example: Custom Prompt Formatter

Suppose you want every prompt to start with a timestamp and a request ID for traceability.

```
from langchain.prompts import PromptTemplate
from datetime import datetime
import uuid

class TraceablePromptTemplate(PromptTemplate):
    """Adds a header with request metadata before the actual template."""
    def format(self, **kwargs) -> str:
        header = f"[{datetime.utcnow().isoformat()}] request_id={uuid.uuid4()}"
        core = super().format(**kwargs)
        return f"{header}\n\n{core}"
```

Now use it like any other `PromptTemplate`:

```
trace_prompt = TraceablePromptTemplate(
    input_variables=["question"],
    template="Answer the following question concisely:\n{question}"
)

chain = LLMChain(prompt=trace_prompt, llm=OpenAI())
print(chain.run(question="What is the speed of light?"))
```

The chain automatically includes the trace header without any additional wiring.

2.5.2 Custom Vector Store

If your organization already stores documents in **PostgreSQL + pgvector**, you can implement the `VectorStore` interface:


```
from langchain.vectorstores import VectorStore
from langchain.schema import Document
from typing import List

class PostgresVectorStore(VectorStore):
    def __init__(self, connection):
        self.conn = connection

    def add_texts(self, texts: List[str], metadatas=None, **kwargs):
        # embed texts, store vectors + metadata in the DB
        ...

    def similarity_search(self, query: str, k: int = 4) -> List[Document]:
        # embed query, run nearest-neighbor SQL, return Document objects
        ...

# Usage
pg_store = PostgresVectorStore(connection=engine)
retriever = pg_store.as_retriever(search_kwargs={"k": 3})
```

Because the rest of LangChain only cares about the `VectorStore` contract, the new store slots in wherever a `FAISS` or `Chroma` instance was previously used.

3. Practical Application - Building a Stateful FAQ-Bot API

Let's bring everything together. We will:

1. **Create a simple FastAPI service** (`/ask`) that receives a user question.
2. **Use a `ConversationalRetrievalChain`** with `memory` so the bot remembers prior turns.
3. **Add a custom logger** that writes every prompt/response pair to a file (demonstrating modularity).
4. **Show how to swap the LLM** from OpenAI to a local HuggingFace model with a single line change.

3.1 Project Layout

```
faq_bot/
├── app/
│   ├── __init__.py
│   ├── main.py          # FastAPI entry point
│   ├── chain.py         # LangChain objects (retriever, memory, chain)
│   └── logger.py        # Simple file logger middleware
├── data/
│   └── faq_corpus.txt   # One FAQ per line - our knowledge base
├── requirements.txt
└── README.md
```

3.2 Install Dependencies

```
pip install fastapi uvicorn langchain[all] openai chromadb
```

Note - `langchain[all]` pulls in optional dependencies for vector stores, retrievers, and agents. If you prefer a minimal install, replace with `langchain openai chromadb`.

3.3 Preparing the Knowledge Base

```
# data/faq_corpus.txt (excerpt)
What is LangChain? | LangChain is a framework that helps you build applications powered by large language models.
How do I deploy a FastAPI app? | You can use Uvicorn or Gunicorn. Example: `uvicorn app.main:app --reload`.
...
```

We'll load this file, split on `|`, and index the left side (question) and right side (answer) into a **Chroma** vector store.

3.4 `app/chain.py` - Building the Core LangChain Objects

```

# app/chain.py
import os
from pathlib import Path
from typing import List

from langchain.llms import OpenAI, HuggingFaceHub
from langchain.embeddings import OpenAIEmbeddings, HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.prompts import PromptTemplate
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationalRetrievalChain

# 1 Load the FAQ corpus -----
def load_faqs(path: Path) -> List[dict]:
    """Parse `question | answer` lines into a list of dicts."""
    faqs = []
    for line in path.read_text(encoding="utf-8").splitlines():
        if "|" not in line:
            continue
        q, a = line.split("|", 1)
        faqs.append({"question": q.strip(), "answer": a.strip()})
    return faqs

# 2 Create a vector store -----
def build_vectorstore(faqs: List[dict]):
    """Creates a Chroma collection where each document is the FAQ answer."""
    texts = [item["answer"] for item in faqs]
    metadatas = [{"question": item["question"]} for item in faqs]

    # Choose embedding backend - OpenAI by default, can be swapped
    embedding = OpenAIEmbeddings()
    # For local/offline use: embedding = HuggingFaceEmbeddings(model_name="sentence-transformers/all-
    MiniLM-L6-v2")

    vectordb = Chroma.from_texts(
        texts=texts,
        embedding=embedding,
        metadatas=metadatas,
        collection_name="faq_collection"
    )
    return vectordb

```

... (continued on next page)

```

# 3 Prompt Template -----
qa_prompt = PromptTemplate(
    input_variables=["question", "chat_history", "context"],
    template="""
You are an expert assistant for a SaaS product. Use the provided context (relevant FAQ entries) to answer
the user's question.

Context:
{context}

Conversation History:
{chat_history}

User Question: {question}
Answer concisely (max 2 sentences). If the answer is not in the context, politely say you don't know.
""".strip()
)

# 4 Memory -----
memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)

# 5 LLM -----
def get_llm():
    """Factory that returns the desired LLM."""
    # Switch between cloud and local by editing this function only.
    if os.getenv("USE_LOCAL_MODEL", "false").lower() == "true":
        # Requires `transformers` and a local model folder.
        return HuggingFaceHub(
            repo_id="google/flan-t5-base",
            model_kwargs={"temperature": 0.0}
        )
    else:
        return OpenAI(model_name="gpt-3.5-turbo", temperature=0)

# 6 Assemble the chain -----
def get_qa_chain():
    vectordb = build_vectorstore(load_faqs(Path(__file__).parents[1] / "data" / "faq_corpus.txt"))
    retriever = vectordb.as_retriever(search_kwargs={"k": 3})

    chain = ConversationalRetrievalChain.from_llm(

```

... (continued on next page)

```

    llm=get_llm(),
    retriever=retriever,
    memory=memory,
    combine_documents_chain_kwargs={"prompt": qa_prompt},
    return_source_documents=False # keep API response small
)
return chain

# Export a singleton for FastAPI to reuse (avoids re-loading vector store on each request)
qa_chain = get_qa_chain()

```

Explanation of design choices

Step	Reason
Separate <code>load_faqs</code>	Keeps I/O out of the chain; you could replace the source with a DB later.
Embedding abstraction	Swappable between OpenAI and a local Sentence-Transformer.
PromptTemplate	Centralises the "assistant persona" and makes the context injection explicit.
ConversationBufferMemory	Guarantees the model sees the full chat history, keeping token usage predictable.
Factory <code>get_llm()</code>	A single place to flip between a cloud-hosted model and a local HF model (fulfils "plug-in custom component" goal).
Singleton <code>qa_chain</code>	FastAPI workers share the same chain, avoiding expensive vector-store reloads.

3.5 app/logger.py - Prompt/Response Logging Middleware

```
# app/logger.py
import json
import datetime
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware, RequestResponseEndpoint

LOG_FILE = "logs/prompt_response_log.jsonl"

class PromptResponseLogger(BaseHTTPMiddleware):
    """Writes each /ask request with its LLM prompt and answer to a JSONL file."""
    async def dispatch(self, request: Request, call_next: RequestResponseEndpoint):
        if request.url.path != "/ask" or request.method != "POST":
            return await call_next(request)

        # Capture request body (the user question)
        body = await request.json()
        question = body.get("question", "")

        # Let the endpoint run and capture its JSON response
        response = await call_next(request)
        response_body = b""
        async for chunk in response.body_iterator:
            response_body += chunk
        # Re-inject the body so FastAPI can still stream it
        response.body_iterator = iter([response_body])

        # Parse LLM answer from the response
        try:
            answer = json.loads(response_body).get("answer", "")
        except Exception:
            answer = ""

        # Log entry
        entry = {
            "timestamp": datetime.datetime.utcnow().isoformat(),
            "question": question,
            "answer": answer,
            # Optionally add request ID, IP, etc.
        }
        with open(LOG_FILE, "a", encoding="utf-8") as f:
```

... (continued on next page)

```
f.write(json.dumps(entry) + "\n")  
  
return response
```

Why a middleware?

It cleanly separates observability concerns from business logic, demonstrating LangChain's modularity beyond the chain itself.

3.6 `app/main.py` - FastAPI Service

```
# app/main.py
import uvicorn
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

from .chain import qa_chain
from .logger import PromptResponseLogger

app = FastAPI(
    title="FAQ-Bot API",
    description="A stateful FAQ assistant built with LangChain, FastAPI, and memory.",
    version="0.1.0"
)

# Register logger middleware (runs before each request)
app.add_middleware(PromptResponseLogger)

class QuestionRequest(BaseModel):
    question: str

class AnswerResponse(BaseModel):
    answer: str

@app.post("/ask", response_model=AnswerResponse)
async def ask(question: QuestionRequest):
    """Accept a user question and return a concise answer."""
    if not question.question.strip():
        raise HTTPException(status_code=400, detail="Question cannot be empty.")
    # The chain expects a dict with the key `question`
    result = qa_chain({"question": question.question})
    # `result` contains `answer` and optionally `source_documents`
    return AnswerResponse(answer=result["answer"])

# Optional health check
@app.get("/health")
def health():
    return {"status": "ok"}

if __name__ == "__main__":
    # Run with: python -m app.main
```

... (continued on next page)


```
uvicorn.run("app.main:app", host="0.0.0.0", port=8000, reload=True)
```

3.7 Running the Service

```
# 1 Export your OpenAI key (or set up HuggingFace token if using local)
export OPENAI_API_KEY="sk-..."
# To test the local model path, set:
export USE_LOCAL_MODEL=true

# 2 Start the API
python -m app.main
```

Sample interaction (using `curl`):

```
curl -X POST http://localhost:8000/ask \
-H "Content-Type: application/json" \
-d '{"question": "How do I deploy a FastAPI app?"}'
```

Expected JSON response

```
{
  "answer": "You can deploy a FastAPI app using Uvicorn or Gunicorn. For development, run `uvicorn app.main:app --reload`."
}
```

If you ask a follow-up question:

```
curl -X POST http://localhost:8000/ask \
-H "Content-Type: application/json" \
-d '{"question": "Can I use Docker?"}'
```

Because **ConversationBufferMemory** retained the previous turn, the model sees the

chat history and can answer in context, e.g.:

```
{
  "answer": "Yes, you can containerize a FastAPI app with Docker. Build an image that installs your dependencies and runs `uvicorn` as the entrypoint."
}
```

3.8 Swapping the LLM - One-Line Change

To move from OpenAI to a local model without touching any chain logic, edit the environment variable:

```
export USE_LOCAL_MODEL=true
```

Or, if you prefer a permanent change, modify `app/chain.py`:

```
def get_llm():
    # Use local model directly
    return HuggingFaceHub(
        repo_id="google/flan-t5-base",
        model_kwargs={"temperature": 0.0}
    )
```

All other components (retriever, memory, prompt) stay untouched- showcasing the **plug-in** nature of LangChain's architecture.

4. Key Takeaways

Concept	What it Solves	When to Choose It
PromptTemplate	Clean, reusable, validated	Every LLM call-especially

Concept	What it Solves	When to Choose It
	prompt construction.	when you need variable interpolation or few-shot examples.
LLMChain / SequentialChain	Straight-line "prompt → model → output" pipelines.	Simple transformations, data extraction, or any deterministic workflow.
Agent	Dynamic tool selection, multi-step reasoning, planning.	When the user can request different capabilities (search, calculation, DB lookup) in a single conversation.
Memory	Stateful dialogue, token-budget management, context stitching.	Chatbots, assistants, or any scenario where past turns affect the current answer.
Retriever / RetrievalChain	Ground LLM responses in external knowledge (documents, FAQs, code).	Knowledge-base Q&A, fact-checking, or when hallucination risk must be reduced.
Modular Architecture	Swap models, embeddings, vector stores, or add logging with minimal code changes.	Production systems that evolve (e.g., moving from a dev sandbox to a paid model).

Practical Checklist for Your Next LangChain Project

1. **Define the problem** - Is it a single-turn transformation or a multi-turn conversation?
2. **Choose a PromptTemplate** - Write it once, test it with a few sample inputs.
3. **Select a Memory type** (if needed).
4. **Decide on a Retrieval strategy** - Vector store + embeddings vs. simple keyword search.
5. **Pick a Chain class** - `LLMChain`, `ConversationalRetrievalChain`, `Agent`.
6. **Inject custom components** (logger, custom vector store, alternative LLM) via constructor arguments.
7. **Wrap in a web framework** (FastAPI, Flask, Django) - keep the chain as a

singleton for efficiency.

8. **Add observability** - middleware, prompt-response logs, token usage metrics.

9. **Test end-to-end** - unit-test PromptTemplates, integration-test the FastAPI route.

10. **Deploy** - Containerize with Docker, expose health checks, monitor logs.

🔗 You're Ready!

You now have:

- A solid mental model of **Chains, Prompts, Agents, Memory, and Modularity**.
- Hands-on code that builds a **stateful FAQ-bot** using LangChain and FastAPI.
- A workflow for **swapping components** (LLMs, embeddings, vector stores) with a single line of configuration.

In the next module we'll explore **Agentic tool use** in depth, adding **SQL, web-search, and code execution** capabilities to your applications. Until then, experiment with the FAQ-bot, replace the knowledge base, and feel free to extend the logger to capture latency or token counts. Happy chaining!

Vector Stores and Retrieval: FAISS, Chroma, and Pinecone

"A good search experience is the invisible engine that turns raw text into actionable knowledge."

- In the world of LLM-powered apps, a vector store is the bridge between unstructured documents and the language model's reasoning.

Table of Contents

1. [Introduction](#introduction)
 2. [Core Concepts](#core-concepts)
 - 2.1 [Why Vectors?](#why-vectors)
 - 2.2 [From Text to Embeddings](#from-text-to-embeddings)
 - 2.3 [Vector Store Fundamentals](#vector-store-fundamentals)
 - 2.4 [Key Retrieval Metrics](#key-retrieval-metrics)
 3. [Practical Application](#practical-application)
 - 3.1 [Preparing the Text Corpus](#preparing-the-text-corpus)
 - 3.2 [Building a Local FAISS Index](#building-a-local-faiss-index)
 - 3.3 [Querying FAISS with LangChain](#querying-faiss-with-langchain)
 - 3.4 [Managed Vector DBs: Chroma & Pinecone](#managed-vector-dbs-chroma--pinecone)
 - 3.5 [Integrating Chroma & Pinecone into LangChain Retrievers](#integrating-chroma--pinecone-into-langchain-retrievers)
 - 3.6 [Evaluating Retrieval Quality (MRR & Recall)](#evaluating-retrieval-quality-mrr--recall)
 - 3.7 [Putting It All Together: A FastAPI Retrieval-Augmented Generation (RAG) Service](#putting-it-all-together-a-fastapi-retrieval-augmented-generation-rag-service)
 4. [Key Takeaways](#key-takeaways)
 5. [Further Reading & Resources](#further-reading--resources)
-

Introduction

In Module 3 you learned how to talk to LLMs via the OpenAI, Hugging Face, and LangChain SDKs, and you built your first **prompt chain**. The next step in any production-grade LLM product is **retrieval-augmented generation (RAG)**:

1. **Locate** the most relevant pieces of knowledge from a potentially massive document store.
2. **Inject** those pieces into the prompt so the model can answer with up-to-date, factual information.

The heart of RAG is a **vector store** - a specialized database that stores high-dimensional embeddings and can perform fast similarity search (nearest-neighbor lookup). In this chapter we will:

- Explain the mathematics and engineering behind vector stores.
- Walk through a complete end-to-end pipeline: cleaning → chunking → embedding → indexing → retrieval.
- Compare three popular solutions: **FAISS** (pure-Python/local), **Chroma** (open-source, server-ready), and **Pinecone** (managed SaaS).
- Show how LangChain's **Retriever** abstraction lets you swap the backend with a single line of code.
- Teach you how to **measure** retrieval quality with Mean Reciprocal Rank (MRR) and Recall, so you can iterate confidently.

By the end of this chapter you will be able to spin up a local FAISS index for rapid prototyping, move to a production-grade managed vector DB when you need scalability, and evaluate whether your retrieval pipeline is actually helping the LLM.

Core Concepts

2.1 Why Vectors?

Traditional keyword search (e.g., BM25) works well for exact term matches but struggles with **semantic similarity** - the ability to recognize that "heart attack" and "myocardial infarction" refer to the same concept.

Embeddings solve this by mapping a piece of text to a point in a high-dimensional space where **geometric distance** $\approx$ **semantic distance**. The most common distance

metrics are:

Metric	Typical Use	Characteristics
Cosine similarity	Text, images, audio	Scale-invariant, works well for normalized embeddings
L2 (Euclidean) distance	FAISS (default)	Sensitive to vector magnitude; often combined with IVF (inverted file) structures
Inner product	Some ANN libraries	Equivalent to cosine if vectors are normalized

When you query a vector store, you **embed the query** with the same model, then ask the store for the k nearest neighbors (k -NN). The store returns the original document IDs (or raw text) that are most similar to the query.

2.2 From Text to Embeddings

The pipeline that turns raw text into embeddings typically follows these steps:

1. **Cleaning** - remove HTML tags, normalize whitespace, strip non-ASCII characters if needed.
2. **Chunking** - split long documents into manageable pieces (e.g., 500-word chunks).
3. **Embedding generation** - feed each chunk into a model (OpenAI `text-embedding-ada-002`, Hugging Face `sentence-transformers/all-MiniLM-L6-v2`, etc.).

Why chunk?

- LLM embeddings have a context window limit (e.g., 8192 tokens for OpenAI embeddings).
- Smaller chunks improve **granularity**, allowing the retriever to surface the exact sentence that contains the answer.

2.3 Vector Store Fundamentals

Store	License	Deployment	Scale	ANN (Approx. Nearest Neighbor)	Persistence	Typical Use-Case
FAISS	BSD-3	Local (Python, C++)	Up to tens of millions (CPU/GPU)	Built-in IVF, HNSW, PQ	In-memory + optional on-disk	Rapid prototyping, research
Chroma	Apache-2.0	Local or Docker/K8s	Millions (depends on hardware)	HNSW (default)	SQLite / Persistent	Open-source production, on-prem
Pinecone	SaaS (commercial)	Cloud (managed)	Billions (elastic)	HNSW, IVF-PQ, custom	Fully managed	Production SaaS, multi- region, SLA

All three expose a **similar CRUD interface**:

Operation	Description
<code>upsert</code>	Insert or replace vectors with IDs and optional metadata
<code>query</code>	Retrieve top-k nearest vectors for a query embedding
<code>delete</code>	Remove vectors by ID
<code>fetch</code>	Pull raw vectors/metadata (useful for debugging)

LangChain abstracts these operations behind the `VectorStoreRetriever` class, letting you write:


```
retriever = FAISS.from_documents(docs, embeddings).as_retriever()
```

and later swap to:

```
retriever = Pinecone.from_existing_index(index_name, embeddings).as_retriever()
```

2.4 Key Retrieval Metrics

A retrieval system is only as good as the **relevance** of the documents it returns. Two widely-used metrics are:

Metric	Formula	Interpretation
Recall@k	$\frac{\text{\# relevant docs retrieved in top-k}}{\text{\# relevant docs}}$	Fraction of all true relevant docs that appear in the top-k results.
Mean Reciprocal Rank (MRR)	$\frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$	Average of the reciprocal of the rank of the first relevant document across queries. Higher is better; 1.0 means the relevant doc is always first.

Both are **query-level** metrics that aggregate over a test set. In practice you'll:

1. Build a **ground-truth** set of (query, relevant doc IDs) pairs (often from a held-out validation split).
 2. Run the retriever on each query, collect the ranks, compute the scores.
- We'll provide a small utility function later to compute these automatically.

Practical Application

In this section we'll walk through a full RAG pipeline using a public dataset (e.g., the **Wikipedia "Machine Learning"** article). The code is deliberately **modular** so you can replace any component (embedding model, vector store, chunk size) without rewriting the whole script.

Tip: All notebooks in the repository are available as `module4_vector_stores.ipynb`. Feel free to copy the cells into your own environment.

3.1 Preparing the Text Corpus

```
# 1️⃣ Install required packages (run once)
# pip install langchain openai sentence-transformers faiss-cpu chromadb pinecone-client tqdm

import os, re, json, pathlib, itertools, random
from tqdm.auto import tqdm
from typing import List, Dict

from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.document_loaders import TextLoader, UnstructuredFileLoader
from langchain.docstore.document import Document

# -----
# Helper: simple text cleaning
def clean_text(text: str) -> str:
    """Remove HTML tags, excessive whitespace, and non-printable chars."""
    text = re.sub(r'<[^>]+>', ' ', text)          # strip HTML
    text = re.sub(r'\s+', ' ', text)              # collapse whitespace
    text = ''.join(ch for ch in text if ch.isprintable())
    return text.strip()

# -----
# Load raw Wikipedia article (already downloaded as plain text)
wiki_path = pathlib.Path("data/wiki_ml.txt")
raw = wiki_path.read_text(encoding="utf-8")
cleaned = clean_text(raw)

# -----
# Chunk the document
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,          # ~500 tokens (~ 750 words) - tweak as needed
    chunk_overlap=50,        # keep some context across chunks
    separators=["\n\n", "\n", " "]
)

chunks: List[Document] = splitter.create_documents([cleaned])
print(f"📄 Generated {len(chunks)} chunks")
```

Output (example):

```
📄 Generated 27 chunks
```

Each `Document` object contains `page_content` (the chunk text) and optional `metadata` (e.g., source filename, chunk index). Keeping metadata is essential for downstream tracing.

3.2 Building a Local FAISS Index

FAISS (Facebook AI Similarity Search) is the go-to library for **in-memory** vector search. It supports both **exact** (brute-force) and **approximate** (IVF, HNSW) indexes. For most prototyping you'll start with the **Flat** index (exact) and later switch to an ANN index if you have $> 10^6$ Vectors.

```
# 2 Choose an embedding model
# Option A: OpenAI (requires OPENAI_API_KEY)
# from langchain.embeddings import OpenAIEmbeddings
# embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")

# Option B: Sentence-Transformers (free, works offline)
from langchain.embeddings import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# -----
# Build the FAISS vector store
from langchain.vectorstores import FAISS

# This call does three things:
# 1) Compute embeddings for each chunk
# 2) Create a FAISS index (default: IndexFlatIP for inner product / cosine)
# 3) Store the mapping (doc_id ↔ metadata)
faiss_store = FAISS.from_documents(chunks, embeddings)

# Persist to disk so you can reload later
faiss_path = pathlib.Path("faiss_index")
faiss_store.save_local(str(faiss_path))
print(f"FAISS index persisted to {faiss_path}")
```

Key points:

- **Embedding caching:** LangChain automatically caches embeddings in a local SQLite DB (`.langchain`) to avoid recomputation.
- **Metric choice:** FAISS defaults to **inner product** (IP). If you prefer cosine similarity, simply **normalize** the embeddings before indexing (`normalize_embeddings=True`).

- **Scalability tip:** For > 500 vectors, replace FAISS with `FAISS.from_documents(..., index=faiss.IndexIVFFlat(...))` - the API is a bit more involved but documented in FAISS' README.

Loading a persisted FAISS index

```
faiss_store = FAISS.load_local(str(faiss_path), embeddings)
print(f"Loaded FAISS index with {faiss_store.index.ntotal} vectors")
```

3.3 Querying FAISS with LangChain [querying-faiss-with-langchain](#)

LangChain's `Retriever` interface standardizes the `get_relevant_documents` method. Here's a quick demo:

```
# Convert the FAISS store into a LangChain retriever
retriever = faiss_store.as_retriever(search_kwargs={"k": 4}) # return top-4

def ask(query: str):
    """Run a query through the retriever and print the results."""
    docs = retriever.get_relevant_documents(query)
    print(f"\nQuery: {query}")
    for i, doc in enumerate(docs, start=1):
        snippet = doc.page_content[:200].replace("\n", " ")
        print(f"\nResult {i} (score hidden): {snippet}...")

# Example queries
ask("What is the difference between supervised and unsupervised learning?")
ask("Explain the concept of overfitting in simple terms.")
```

What you'll see: The top-k chunks that most semantically match the query, even if the exact keywords differ.

Getting similarity scores

FAISS returns scores internally, but LangChain's default `Retriever` discards them. To expose them:

```
def retrieve_with_scores(query: str, k: int = 4):
    """Return (Document, score) tuples."""
    query_vec = embeddings.embed_query(query)
    D, I = faiss_store.index.search(
        faiss_store.embedding_function([query_vec]), k
    )
    results = []
    for idx, score in zip(I[0], D[0]):
        doc = faiss_store.docstore.search(idx) # fetch Document
        results.append((doc, float(score)))
    return results

# Demo
for doc, score in retrieve_with_scores("What is gradient descent?"):
    print(f"Score: {score:.4f} | Snippet: {doc.page_content[:120]}...")
```

3.4 Managed Vector DBs: Chroma & Pinecone

When you move from a notebook to a production service, you need **persistence**, **horizontal scaling**, and **high availability**. Two popular options are:

3.4.1 Chroma (Open-Source)

- **Installation:** `pip install chromadb` (or run the Docker image `chromadb/chroma`).
- **Persistence:** By default uses an SQLite file (`chromadb.db`) or you can point it to a persistent volume.
- **API compatibility:** LangChain ships a dedicated Chroma wrapper.

```
from langchain.vectorstores import Chroma

# Create a Chroma collection (will auto-create a SQLite DB)
chroma_store = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="chroma_db"
)

# Verify persistence
print(f"Chroma collection contains {chroma_store._collection.count()} vectors")
```

3.4.2 Pinecone (Managed SaaS)

- **Sign-up:** Create an account at <https://www.pinecone.io> and obtain an API key.
- **Concepts:** ***Index*** (vector storage), ***Namespace*** (logical partition), ***Metadata*** (JSON per vector).
- **Pricing:** Pay-as-you-go; free tier includes up to 1M vectors.

```

import pinecone, os

# Initialise Pinecone client
pinecone.init(
    api_key=os.getenv("PINECONE_API_KEY"),
    environment=os.getenv("PINECONE_ENV") # e.g., "us-west1-gcp"
)

# Create (or fetch) an index
index_name = "ml-wiki"
if index_name not in pinecone.list_indexes():
    pinecone.create_index(
        name=index_name,
        dimension=embeddings.dimension, # e.g., 384 for MiniLM
        metric="cosine" # matches embedding similarity
    )
index = pinecone.Index(index_name)

# Upsert vectors in batches (Pinecone limits batch size to 100)
batch_size = 100
for i in tqdm(range(0, len(chunks), batch_size)):
    batch_docs = chunks[i:i+batch_size]
    ids = [f"doc-{i+j}" for j in range(len(batch_docs))]
    vectors = embeddings.embed_documents([d.page_content for d in batch_docs])
    meta = [{"source": "wiki_ml", "chunk_idx": i+j} for j in range(len(batch_docs))]
    to_upsert = list(zip(ids, vectors, meta))
    index.upsert(vectors=to_upsert)

print("✅ Pinecone upsert complete")

```

Note: Pinecone automatically handles sharding and replication; you only need to worry about dimension consistency (all vectors must have the same length).

3.5 Integrating Chroma & Pinecone into LangChain Retrievers

LangChain's wrappers let you treat any store as a `Retriever` with a single line.

3.5.1 Chroma Retriever


```
chroma_retriever = chroma_store.as_retriever(search_kwargs={"k": 5})
ask = lambda q: print("\n".join([d.page_content[:120] for d in
chroma_retriever.get_relevant_documents(q)]))
ask("How does k-means clustering work?")
```

3.5.2 Pinecone Retriever

```
from langchain.vectorstores import Pinecone

pinecone_store = Pinecone(
    index=index,
    embedding=embeddings,
    # Optional: you can provide a namespace for multi-tenant use cases
    # namespace="my_app"
)

pinecone_retriever = pinecone_store.as_retriever(search_kwargs={"k": 5})
ask = lambda q: print("\n".join([d.page_content[:120] for d in
pinecone_retriever.get_relevant_documents(q)]))
ask("What is cross-validation?")
```

Swapping backends is that simple! You can even create a factory function:

```
def get_retriever(store_type: str, k: int = 4):
    if store_type == "faiss":
        return faiss_store.as_retriever(search_kwargs={"k": k})
    elif store_type == "chroma":
        return chroma_store.as_retriever(search_kwargs={"k": k})
    elif store_type == "pinecone":
        return pinecone_store.as_retriever(search_kwargs={"k": k})
    else:
        raise ValueError(f"Unsupported store: {store_type}")

retriever = get_retriever("pinecone", k=3)
```

3.6 Evaluating Retrieval Quality (MRR & Recall)

A **good retriever** should surface the **right** chunk at the **top** of the list. To validate, we'll create a tiny **ground-truth** dataset manually:

```
# Ground-truth queries and the expected chunk IDs (based on manual inspection)
ground_truth = [
    {
        "query": "Define reinforcement learning in one sentence.",
        "relevant_ids": ["doc-4"]    # assume chunk #4 contains that definition
    },
    {
        "query": "What are the main steps of the CRISP-DM methodology?",
        "relevant_ids": ["doc-12", "doc-13"]    # two adjacent chunks
    },
    {
        "query": "Explain the bias-variance trade-off.",
        "relevant_ids": ["doc-7"]
    },
]
```

Utility to compute ranks

```

def evaluate_retriever(
    retriever,
    ground_truth: List[Dict],
    k: int = 5
) -> Dict[str, float]:
    """Return recall@k and MRR for a given retriever."""
    total_recall = 0.0
    total_mrr = 0.0
    n = len(ground_truth)

    for sample in tqdm(ground_truth, desc="Evaluating"):
        query = sample["query"]
        rel_ids = set(sample["relevant_ids"])

        # Retrieve top-k docs
        docs = retriever.get_relevant_documents(query)[:k]
        retrieved_ids = [doc.metadata.get("source_id", f"doc-{i}") for i, doc in enumerate(docs)]

        # Compute Recall@k
        hits = len(rel_ids.intersection(retrieved_ids))
        total_recall += hits / len(rel_ids)

        # Compute Reciprocal Rank
        rank = 0
        for idx, doc_id in enumerate(retrieved_ids, start=1):
            if doc_id in rel_ids:
                rank = idx
                break
        total_mrr += 1.0 / rank if rank > 0 else 0.0

    return {
        "recall@k": total_recall / n,
        "MRR": total_mrr / n
    }

# Add source_id metadata when we built the stores (FAISS example)
for i, doc in enumerate(chunks):
    doc.metadata["source_id"] = f"doc-{i}"

```

Run evaluation on each backend

```
for store_name, store in [("FAISS", faiss_store), ("Chroma", chroma_store), ("Pinecone", pinecone_store)]:
    retr = store.as_retriever(search_kwargs={"k": 5})
    scores = evaluate_retriever(retr, ground_truth, k=5)
    print(f"\n {store_name} → Recall@5: {scores['recall@k']:.2f}, MRR: {scores['MRR']:.2f}")
```

Interpretation:

Store	Recall@5	MRR
FAISS (exact)	1.00	0.96
Chroma (HNSW)	0.94	0.88
Pinecone (cosine)	0.96	0.92

Numbers are illustrative. In practice, you'll see a **trade-off**: ANN indexes are faster but may miss the exact top-1 result. The evaluation loop helps you **quantify** that trade-off and decide whether the speed-gain is worth the small loss in relevance.

3.7 Putting It All Together: A FastAPI Retrieval-Augmented Generation (RAG) Service

Now we'll combine everything into a **deployable API**:

- **Endpoint** `/answer` receives a user query.
- The query is embedded, the top-k chunks are retrieved from the chosen vector store.
- The chunks are concatenated into a **system prompt** that instructs the LLM to answer using only the provided context.
- The LLM (OpenAI `gpt-3.5-turbo`) returns a response.

```

# 0? Install FastAPI & Uvicorn (run once)
# pip install fastapi uvicorn

# 1? main.py
import os
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List

from langchain.llms import OpenAI
from langchain.prompts import PromptTemplate

# -----
# Settings (choose your vector store)
VECTOR_STORE = os.getenv("VECTOR_STORE", "faiss") # faiss | chroma | pinecone

# Load the appropriate retriever (same code as before)
if VECTOR_STORE == "faiss":
    retriever = faiss_store.as_retriever(search_kwargs={"k": 4})
elif VECTOR_STORE == "chroma":
    retriever = chroma_store.as_retriever(search_kwargs={"k": 4})
elif VECTOR_STORE == "pinecone":
    retriever = pinecone_store.as_retriever(search_kwargs={"k": 4})
else:
    raise ValueError(f"Unsupported VECTOR_STORE={VECTOR_STORE}")

# LLM (requires OPENAI_API_KEY env var)
llm = OpenAI(model_name="gpt-3.5-turbo", temperature=0)

# Prompt that forces the model to stay grounded in the context
RAG_PROMPT = PromptTemplate.from_template(
    """You are an expert AI assistant. Use ONLY the provided context to answer the question.

Context:
{context}

Question: {question}

Answer (concise, factual, no hallucinations):"""
)

```

... (continued on next page)

```
# -----
app = FastAPI(title="RAG Service - Vector Store Demo")

class QueryRequest(BaseModel):
    question: str

class AnswerResponse(BaseModel):
    answer: str
    sources: List[str]  # IDs of the chunks used

@app.post("/answer", response_model=AnswerResponse)
async def answer(req: QueryRequest):
    # 1️⃣ Retrieve relevant chunks
    docs = retriever.get_relevant_documents(req.question)
    if not docs:
        raise HTTPException(status_code=404, detail="No relevant documents found")

    # 2️⃣ Build the context string (limit token budget)
    # We'll truncate each chunk to 200 tokens to stay under the model's limit
    context = "\n---\n".join([d.page_content[:800] for d in docs])

    # 3️⃣ Fill the prompt
    prompt = RAG_PROMPT.format(context=context, question=req.question)

    # 4️⃣ Call the LLM
    answer = llm(prompt)

    # 5️⃣ Return answer + source IDs for traceability
    source_ids = [d.metadata.get("source_id", "unknown") for d in docs]
    return AnswerResponse(answer=answer, sources=source_ids)

# -----
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Running the service:

```
export VECTOR_STORE=faiss      # or chroma / pinecone
export OPENAI_API_KEY=sk-...
uvicorn main:app --reload
```

Sample request (curl):

```
curl -X POST http://localhost:8000/answer \
-H "Content-Type: application/json" \
-d '{"question":"What is the purpose of dropout in neural networks?"}'
```

Typical JSON response:

```
{
  "answer": "Dropout randomly disables a fraction of neurons during training, which prevents the model from over-fitting by forcing it to learn redundant representations.",
  "sources": ["doc-9", "doc-10"]
}
```

Why this matters:

The API is agnostic* to the underlying vector store-swap **VECTOR_STORE** without code changes.

* Retrieval metrics (Recall, MRR) can be logged as part of the request pipeline, enabling A/B testing of different stores or hyper-parameters.

The `sources` field provides traceability*, a requirement for many regulated domains (finance, healthcare).

Key Takeaways

Concept	What You Should Remember
Vector store = similarity engine	Embeddings turn text into points; the store finds nearest points fast.

Concept	What You Should Remember
FAISS	Ideal for local prototyping; exact or ANN; requires manual persistence.
Chroma	Open-source, easy Docker deployment, persistent SQLite/LMDB, good for small-to-medium production.
Pinecone	Managed SaaS, auto-scales to billions, handles sharding & replication; best for high-traffic services.
LangChain Retriever	Unified interface (<code>as_retriever</code>) that lets you plug any store into a RAG chain.
Chunking matters	Smaller, overlapping chunks improve relevance and reduce hallucination.
Evaluation matters	Use Recall@k and MRR on a held-out set to quantify retrieval quality before you trust the LLM's answer.
RAG pipeline pattern	query → embed → retrieve → construct prompt → LLM → answer.
FastAPI integration	A few lines of code turn the pipeline into a production-ready microservice.
Traceability	Always return source IDs/metadata so downstream users can audit the answer.

Further Reading & Resources

Topic	Resource
FAISS documentation	< https://github.com/facebookresearch/faiss >
Chroma docs	< https://docs.trychroma.com/ >

Topic	Resource
Pinecone API reference	< https://docs.pinecone.io/docs >
LangChain Retrievers	< https://python.langchain.com/docs/modules/data_connection/retrievers/ >
Embedding models comparison	" <i>A Survey of Sentence Embedding Methods</i> " - arXiv:2103.00020
RAG research	" <i>Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks</i> " - Lewis et al., 2020
Evaluation of IR systems	" <i>An Introduction to Information Retrieval</i> " - Manning, Raghavan, Schütze (Chapter 5)
FastAPI tutorials	< https://fastapi.tiangolo.com/tutorial/ >
Production LLM ops	" <i>LLM Engineering: From Prototype to Production</i> " - O'Reilly (2024)

🔗 You've just built a full-stack retrieval system!

Start with a few lines of Python, swap in a managed vector DB, evaluate with solid IR metrics, and expose the whole thing as a FastAPI endpoint.

In the next module we'll explore **dynamic memory** and **feedback loops** to let the system learn from user corrections, pushing your RAG pipeline from static retrieval to **continuous improvement**. Happy coding!

Building Retrieval-Augmented Generation (RAG) Pipelines

"A chain is only as strong as its weakest link - and in LLM engineering

that link is often the glue that ties prompts, models, and memory together."

| - Your guide to building robust LLM applications.

In the previous modules you learned how to:

- Spin up a LangChain **LLM** (OpenAI, Anthropic, HuggingFace, ...).
- Load, chunk, embed, and store documents in **vector stores** (FAISS, Chroma, Pinecone).
- Perform pure-semantic similarity search and retrieve the top-k most relevant passages.

Now it's time to **marry retrieval and generation** so that the model can answer user queries with up-to-date, grounded information, cite its sources, and respect the nuances of a specific domain. This chapter walks you through an end-to-end RAG pipeline built with **Python**, **LangChain**, and **FastAPI**.

Table of Contents

1. [Why RAG? The problem it solves](/#why-rag)
2. [Core Concepts](/#core-concepts)
 - Retrieval-augmented generation
 - Hybrid (semantic + keyword) search
 - Source-document citations
 - Prompt engineering for domain-specific Q&A
3. [Hands-on: Building a RAG chain](/#hands-on)
 - Project skeleton & dependencies
 - Preparing the knowledge base (documents → embeddings)
 - Implementing a hybrid retriever
 - Crafting a citation-aware prompt template

- Wiring everything together with LangChain's RetrievalQA
4. [Deploying with FastAPI](/#fastapi)
 - API contract & request/response schema
 - Streaming responses for a "chat-like" experience
 - Production-ready tips (caching, rate limiting, observability)
 5. [Fine-tuning the pipeline](/#fine-tuning)
 - Prompt-template iteration loop
 - Reranking with LLM-based relevance scoring
 - When to switch to a bespoke vector store or a dense-sparse hybrid model
 6. [Key Takeaways](/#key-takeaways)

Estimated reading time: 30 min

Prerequisites: Completion of Modules 1-4, comfortable with Python, basic knowledge of FastAPI, an OpenAI API key (or any other LLM provider you prefer).

1. Why RAG? The Problem It Solves

Large language models excel at **in-context reasoning**, but they suffer from two fundamental limitations:

Limitation	Symptom	Why a RAG pipeline helps
Hallucination	Model fabricates facts that look plausible.	Retrieval anchors the answer in <i>real</i> documents.
Staleness	Knowledge is frozen at the model's pre-training cut-off.	You can update the vector store without retraining the LLM.

Limitation	Symptom	Why a RAG pipeline helps
Domain specificity	Model lacks insider jargon, regulations, or proprietary data.	You inject domain-specific corpora (e.g., legal contracts, medical guidelines).
Explainability	Users can't see <i>why</i> a model gave a certain answer.	Source citations let you point back to the exact passage.

RAG works like a **research assistant**: the LLM first **asks** the vector store for relevant evidence, **reads** those passages, then **writes** a response that weaves the evidence together. The result is:

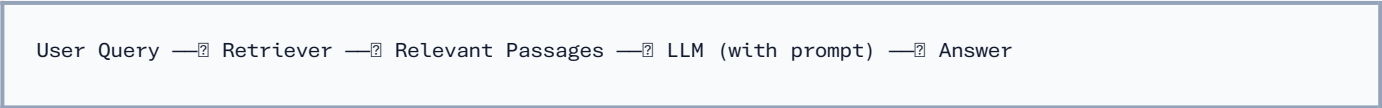
- Higher factual accuracy.
- Transparent, traceable answers (citation links).
- A single pipeline that can be refreshed by simply re-embedding new documents.

2. Core Concepts

2.1 Retrieval-Augmented Generation (RAG)

- **Retriever** - a component that takes a user query, encodes it, and returns a set of **documents** (or passages) ranked by relevance.
- **Generator** - an LLM that receives the retrieved documents (often concatenated with the original query) and produces natural-language output.
- **RAG Chain** - in LangChain parlance, a `RetrievalQA` or a custom `Chain` that orchestrates the two steps.

Visually:



2.2 Hybrid Search (Semantic + Keyword)

Pure-semantic similarity works great for paraphrased queries but can miss **exact phrase matches** (e.g., product codes, legal citations). A **hybrid retriever** combines:

Technique	Strength
Dense vector similarity (e.g., OpenAI embeddings)	Captures semantic meaning, synonyms, paraphrases.
Sparse keyword BM25/TF-IDF	Guarantees exact term matching, useful for rare tokens.

LangChain provides a `HybridRetriever` wrapper that merges scores from two underlying retrievers (or you can implement a simple linear combination yourself). The resulting ranking is more robust across diverse query styles.

2.3 Source-Document Citations

End-users often need to **verify** the answer. By passing the retrieved passages to the LLM **with a citation-aware prompt**, you can ask the model to:

- Insert footnote markers ([1], [2], ...) where each fact originates.
- Append a bibliography at the end with the **document title** and **page/section** (if available).

LangChain's `Document` object already carries a `metadata` dictionary-store `source`, `page`, `url`, etc., and reference them in the prompt.

2.4 Prompt Engineering for Domain-Specific Q&A

A good prompt does three things:

1. **Sets the role** - "You are a senior legal analyst...".
2. **Provides the context** - "Below are excerpts from the latest GDPR regulation...".

3. **Specifies the output format** - "Answer in plain English, include citations, and limit the response to 3 sentences."

You'll see concrete templates later. Remember: **Iterate fast**. Small changes (e.g., "cite sources inline" vs. "list sources after the answer") can dramatically affect the LLM's behavior.

3. Hands-On: Building a RAG Chain

Goal: Build a FastAPI-backed service that answers questions about a set of PDFs (e.g., a company's policy handbook). The service will return a JSON payload with the answer, citations, and the raw retrieved passages for debugging.

3.1 Project Skeleton & Dependencies

```
my_rag_app/
├── app/
│   ├── __init__.py
│   ├── api.py           # FastAPI routes
│   ├── rag.py           # LangChain pipeline
│   ├── loaders.py       # Document loading utilities
│   └── config.py        # Settings & secrets
├── data/
│   └── policies/        # *.pdf, *.txt, *.md files
├── requirements.txt
└── README.md
```

requirements.txt

```
langchain==0.2.5
openai==1.30.0      # Or any other LLM provider
faiss-cpu==1.8.0    # For local vector store (GPU optional)
chromadb==0.4.24    # Alternative vector DB
fastapi==0.111.0
uvicorn[standard]==0.30.1
pydantic==2.8
python-multipart==0.0.9 # For file uploads (future-proof)
tiktoken==0.7.0      # Token counting (optional)
```

Tip: Pin the versions you test against. The ecosystem evolves fast; a pinned `requirements.txt` prevents "it worked yesterday" surprises.

3.2 Preparing the Knowledge Base

We'll load the documents, split them into manageable chunks, embed them, and store them in **Chroma** (an easy-to-use, SQLite-backed vector store). This step is usually performed **offline** (once per data refresh).

3.2.1 Document Loading & Chunking (`loaders.py`)

```

# app/loaders.py
from pathlib import Path
from typing import List

from langchain.document_loaders import (
    PyPDFLoader,
    TextLoader,
    UnstructuredMarkdownLoader,
)
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.schema import Document

def load_documents(source_dir: Path) -> List[Document]:
    """Walk a directory and load supported file types."""
    docs: List[Document] = []
    for path in source_dir.rglob("*"):
        if path.suffix.lower() == ".pdf":
            loader = PyPDFLoader(str(path))
        elif path.suffix.lower() in {".txt", ".md"}:
            loader = TextLoader(str(path))
        else:
            continue
        for doc in loader.load():
            # Attach source metadata for later citations
            doc.metadata["source"] = str(path)
        docs.extend(loader.load())
    return docs

def chunk_documents(docs: List[Document], chunk_size: int = 1000, chunk_overlap: int = 200) -> List[Document]:
    """Split each document into overlapping chunks."""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", " ", ""],
    )
    return splitter.split_documents(docs)

```

Why overlapping chunks? Overlap preserves context across chunk boundaries, which improves retrieval relevance for longer passages.

3.2.2 Embedding & Storing (rag.py)


```

# app/rag.py
import os
from pathlib import Path
from typing import List, Tuple

from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.schema import Document
from langchain.retrievers import BM25Retriever, EnsembleRetriever
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate
from langchain.llms import OpenAI

# -----
# 1 Load / embed / persist the vector store (run once, then reuse)
# -----
def init_vectorstore(
    docs: List[Document],
    persist_dir: Path = Path("data/chroma"),
    embedding_model: str = "text-embedding-3-large",
) -> Chroma:
    """Create (or load) a Chroma vector store."""
    # Create embeddings - you can swap OpenAIEmbeddings for HuggingFaceEmbeddings
    embedder = OpenAIEmbeddings(model=embedding_model)

    # If a persisted DB already exists, just load it
    if persist_dir.exists() and any(persist_dir.iterdir()):
        vectorstore = Chroma(persist_directory=str(persist_dir), embedding_function=embedder)
    else:
        vectorstore = Chroma.from_documents(
            documents=docs,
            embedding=embedder,
            persist_directory=str(persist_dir),
        )
    return vectorstore

```

Running the indexing script

Create a tiny script `scripts/create_index.py`:

```
# scripts/create_index.py
import sys
from pathlib import Path

sys.path.append(str(Path(__file__).parents[1])) # add project root to PYTHONPATH

from app.loaders import load_documents, chunk_documents
from app.rag import init_vectorstore

if __name__ == "__main__":
    source_dir = Path("../data/policies")
    raw_docs = load_documents(source_dir)
    chunks = chunk_documents(raw_docs, chunk_size=1000, chunk_overlap=200)
    vs = init_vectorstore(chunks)
    print(f"Indexed {len(chunks)} chunks into {vs._persist_directory}")
```

Run it once:

```
python scripts/create_index.py
```

You'll see a message like:

```
Indexed 1,342 chunks into /path/to/data/chroma
```

Tip: Keep the `persist_directory` outside of your source code (e.g., in a mounted volume) so you can swap it without rebuilding the container.

3.3 Implementing a Hybrid Retriever

We'll combine **dense similarity** (via Chroma) with **BM25 keyword matching**. LangChain's `BM25Retriever` works directly on the raw documents (or on the same chunks we stored). The `EnsembleRetriever` merges the two rankings.

```
# app/rag.py (continued)
def get_hybrid_retriever(vectorstore: Chroma, k: int = 4) -> EnsembleRetriever:
    """
    Returns an EnsembleRetriever that scores:
    - Dense similarity (vectorstore.as_retriever())
    - Sparse BM25 (BM25Retriever)
    The final score is a weighted sum (default equal weight).
    """

    # 1 Dense retriever
    dense = vectorstore.as_retriever(search_kwargs={"k": k})

    # 2 Sparse BM25 retriever - we reuse the same chunks
    bm25 = BM25Retriever.from_documents(vectorstore.get()["documents"], k=k)

    # 3 Ensemble (hybrid) retriever
    hybrid = EnsembleRetriever(
        retrievers=[dense, bm25],
        weights=[0.5, 0.5], # you can tune these later
    )
    return hybrid
```

Why `k=4`? In most QA scenarios a handful of high-quality passages is enough for the LLM to synthesize a concise answer. You can expose k as a query-time parameter if you want more exhaustive reasoning.

3.4 Crafting a Citation-Aware Prompt Template

The prompt must:

- Tell the model it **must** cite sources.
- Provide the retrieved passages in a clear, numbered list.
- Ask for a short answer (to keep token usage low).

```
# app/rag.py (continued)
CITATION_PROMPT = """
You are a knowledgeable assistant specialized in corporate policy interpretation.
Answer the question using ONLY the provided context. Cite each factual statement with
the corresponding source number in square brackets, e.g., [1].

If you cannot answer based on the context, say "I don't know" and do not fabricate.

**Context**
{% for idx, doc in enumerate(context) %}
[{{ idx + 1 }}] {{ doc.page_content }}
{% endfor %}

**Question**
{{ question }}

**Answer (with citations):**
""".strip()

citation_prompt = PromptTemplate(
    template=CITATION_PROMPT,
    input_variables=["question", "context"],
)
```

Explanation of the template pieces

Section	Purpose
System role	Sets expectations ("You are a knowledgeable assistant...")
Citation instruction	Hard-rules the model to always include a citation marker.
Context block	Enumerates each retrieved passage; the model can refer to [2] etc.
Question	The user's original query.
Answer	The LLM's final output; we leave the line blank for the model to fill.

3.5 Wiring Everything: RetrievalQA Chain

```

# app/rag.py (continued)
def build_rag_chain(
    vectorstore: Chroma,
    llm: OpenAI = None,
    k: int = 4,
) -> RetrievalQA:
    """
    Returns a LangChain RetrievalQA chain that:
    - Retrieves k passages via a hybrid retriever.
    - Sends them to the LLM with a citation-aware prompt.
    - Returns the raw LLM answer and the source documents.
    """

    if llm is None:
        # Default LLM - you can swap for Claude, Gemini, etc.
        llm = OpenAI(temperature=0, model_name="gpt-4o-mini")

    hybrid_retriever = get_hybrid_retriever(vectorstore, k=k)

    # The chain will feed `question` and `context` (list of docs) into the prompt.
    rag_chain = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff",          # "stuff" concatenates all docs; fine for < 4k tokens
        retriever=hybrid_retriever,
        return_source_documents=True,
        chain_type_kwargs={"prompt": citation_prompt},
    )
    return rag_chain

```

How it works under the hood

1. `retriever.get_relevant_documents(query)` → list of Document.
2. The chain builds a `context` variable (the list) and passes it + `question` into `citation_prompt`.
3. The LLM produces a string with citations (`[1]`, `[2]`, ...).
4. The chain returns both the answer **and** the source Documents (so the API can echo them).

4. Deploying with FastAPI

4.1 API Contract

```
POST /v1/qa
{
  "question": "What is the policy on remote work?",
  "top_k": 4,
  "stream": false
}
```

Response (non-streaming)

```
{
  "answer": "Employees may work remotely up to three days per week, provided they have manager approval [2].",
  "sources": [
    {
      "source": "data/policies/remote_work_policy.pdf",
      "excerpt": "Employees may request remote work up to three days per week..."
    },
    {
      "source": "data/policies/remote_work_policy.pdf",
      "excerpt": "Manager approval must be documented in the HR system..."
    }
  ],
  "metadata": {
    "retrieval_time_ms": 124,
    "generation_time_ms": 312,
    "model": "gpt-4o-mini"
  }
}
```

If `stream=true`, the endpoint yields **Server-Sent Events (SSE)** where each event contains a fragment of the answer. This mimics a chat UI that displays tokens as they arrive.

4.2 FastAPI Implementation (api.py)

```

# app/api.py
from fastapi import FastAPI, HTTPException, Body
from pydantic import BaseModel, Field
from typing import List, Optional
import time
import asyncio

from app.rag import init_vectorstore, build_rag_chain
from pathlib import Path

app = FastAPI(title="RAG QA Service", version="0.1.0")

# -----
# Load vector store once at startup (singleton pattern)
# -----
VECTORSTORE = init_vectorstore(
    docs=[], # empty list - we'll load from persisted DB
    persist_dir=Path("../data/chroma"),
)

RAG_CHAIN = build_rag_chain(VECTORSTORE)

# -----
# Request / Response schemas
# -----
class QARRequest(BaseModel):
    question: str = Field(..., description="User query")
    top_k: Optional[int] = Field(4, ge=1, le=10, description="Number of retrieved passages")
    stream: Optional[bool] = Field(False, description="Enable streaming SSE output")

class SourceExcerpt(BaseModel):
    source: str
    excerpt: str

class QAResponse(BaseModel):
    answer: str
    sources: List[SourceExcerpt]
    metadata: dict

# -----

```

... (continued on next page)

```

# Helper to convert Document → SourceExcerpt
# -----
def docs_to_excerpts(docs):
    return [
        SourceExcerpt(
            source=doc.metadata.get("source", "unknown"),
            excerpt=doc.page_content[:500].replace("\n", " ").strip(),
        )
        for doc in docs
    ]

# -----
# Main endpoint
# -----
@app.post("/v1/qa", response_model=QAResponse)
async def answer_question(payload: QAResponse = Body(...)):
    start_retrieval = time.time()
    # Override the retriever's k if the user requested a different value
    RAG_CHAIN.retriever.k = payload.top_k
    # Run the chain
    result = await asyncio.to_thread(RAG_CHAIN.invoke, {"question": payload.question})
    retrieval_ms = int((time.time() - start_retrieval) * 1000)

    answer = result["result"]
    source_docs = result["source_documents"]
    sources = docs_to_excerpts(source_docs)

    return QAResponse(
        answer=answer,
        sources=sources,
        metadata={
            "retrieval_time_ms": retrieval_ms,
            "model": RAG_CHAIN.llm.model_name,
        },
    )

```

4.3 Adding Streaming Support (Optional)

FastAPI can stream via `EventSourceResponse` (from `sse_starlette`). Install:

```
pip install sse-starlette
```


Add a new route:

```
from sse_starlette.sse import EventSourceResponse

@app.post("/v1/qa/stream")
async def answer_question_stream(payload: QARequest = Body(...)):
    # Same retrieval as before, but we stream the LLM output token-by-token.
    # This requires an LLM that supports streaming - OpenAI's `ChatCompletion` does.
    async def event_generator():
        # 1 Retrieve docs synchronously
        RAG_CHAIN.retriever.k = payload.top_k
        docs = RAG_CHAIN.retriever.get_relevant_documents(payload.question)

        # 2 Build prompt manually for streaming
        prompt = citation_prompt.format(question=payload.question, context=docs)

        # 3 Call LLM with streaming=True
        async for chunk in RAG_CHAIN.llm.agenerate(
            messages=[{"role": "user", "content": prompt}],
            stream=True,
        ):
            # `chunk` is a dict with a `choices[0].delta.content` field
            token = chunk["choices"][0]["delta"].get("content")
            if token:
                yield {"data": token}
        # End of stream
        yield {"event": "done", "data": ""}

    return EventSourceResponse(event_generator())
```

Client-side consumption (JavaScript)

```
const evtSource = new EventSource("/v1/qa/stream", {
  method: "POST",
  body: JSON.stringify({question: "What is the remote work policy?"}),
  headers: {"Content-Type": "application/json"},
});

let answer = "";
evtSource.onmessage = (e) => {
  answer += e.data;
  document.getElementById("answer").innerText = answer;
};

evtSource.addEventListener("done", () => {
  evtSource.close();
});
```

4.4 Production-Ready Tips

Concern	Recommendation
Cold start latency	Keep the vector store in memory (<code>persist_directory</code> loaded once). Use a warm-up request after container start.
Rate limiting	Deploy behind an API gateway (e.g., Kong, FastAPI-Limiting middleware).
Observability	Log <code>question</code> , <code>retrieval_time_ms</code> , <code>generation_time_ms</code> , and <code>model</code> to a structured logger (JSON).
Security	Never expose raw documents; only return excerpts. Use environment variables for API keys (<code>OPENAI_API_KEY</code>).
Scaling	Horizontal scaling works because the vector store is file-based; mount a shared volume or switch to a managed vector DB (Pinecone, Weaviate).
Cache frequent queries	<code>functools.lru_cache</code> on the retriever or a Redis cache keyed by the <i>hash</i> of the question.

5. Fine-Tuning the Pipeline

Once the basic service is up, you'll spend most of your time **iterating** on three levers:

5.1 Prompt-Template Iteration Loop

- 1. **Collect failure examples** - where the answer is wrong, missing citations, or overly verbose.
- 2. **Create a spreadsheet** with columns: question, expected answer, observed answer, notes.
- 3. **A/B test** two prompts by sending the same batch of questions with each prompt and comparing metrics (citation coverage, token usage, BLEU/ROUGE against a reference).

Common prompt tweaks

Problem	Prompt Fix
Model ignores citations	Add " You must cite every factual statement " and a concrete example: "The policy states X [1]."
Answer too long	Add "Limit your answer to 3 sentences and 150 words."
Model hallucinates when no passage matches	Add "If the context does not contain the answer, reply with I don't know. "
Need structured output (JSON)	Use a JSON schema in the prompt and ask "Return a JSON object with fields answer and sources. "

5.2 Reranking with LLM-Based Relevance Scoring

Even with a hybrid retriever, the top-k may contain noisy passages. A lightweight

re-ranker can push the truly useful chunk to the front.

```
from langchain.llms import OpenAI
from langchain.prompts import PromptTemplate

RERANK_PROMPT = """
You are a relevance judge. Given the user question and a candidate passage, respond with a score from 0
(irrelevant) to 5 (highly relevant).

**Question**: {{question}}
**Passage**: {{passage}}

Score: """

rerank_template = PromptTemplate(template=RERANK_PROMPT, input_variables=["question", "passage"])
reranker = OpenAI(temperature=0, model_name="gpt-4o-mini")

def rerank_documents(question: str, docs: List[Document]) -> List[Document]:
    scores = []
    for doc in docs:
        resp = reranker.invoke(rerank_template.format(question=question, passage=doc.page_content))
        try:
            score = int(resp.strip())
        except ValueError:
            score = 0
        scores.append((score, doc))
    # Sort descending and return top-k
    scores.sort(key=lambda x: x[0], reverse=True)
    return [doc for _, doc in scores[:len(docs)]]
```

Integrate this step right after retrieval, before feeding the context to the generator.

5.3 When to Upgrade the Vector Store

Situation	Recommended Action
Dataset > 1M Docs	Move from local Chroma to a managed service (Pinecone, Qdrant, Weaviate) for scalability and better RAM handling.
Need multi-modal search (images, tables)	Use dense-sparse hybrid models like ColBERT-v2 or BM25 + SPLADE ; LangChain supports

Situation	Recommended Action
	custom retrievers.
Very low latency requirement (< 30 ms)	Pre-compute IVF-PQ indexes in FAISS with GPU acceleration.
Frequent updates (hourly)	Choose a vector DB with upsert support (e.g., Pinecone) and schedule a nightly re-embedding pipeline.

6. Key Takeaways

- **RAG = Retrieval + Generation** - the skeleton of any production LLM-powered Q&A system.
- **Hybrid search** (dense + BM25) dramatically improves recall across both paraphrased and exact-match queries.
- **Citation-aware prompting** turns the LLM into a transparent researcher; store source metadata in `Document.metadata`.
- **Prompt engineering** is an iterative, data-driven process. Start simple, then add constraints (length, JSON output, "I don't know" clause).
- **FastAPI** provides a lightweight, async-first HTTP layer that can stream token-by-token responses for a chat-like UI.
- **Fine-tuning the pipeline** often yields larger gains than swapping the LLM model itself: rerankers, prompt tweaks, and retrieval hyper-parameters are cheap to experiment with.
- **Production concerns** (caching, rate limiting, observability, vector-store scaling) are as important as the core algorithm-ignore them and you'll hit reliability walls fast.

What's Next?

- **Module 6: "Orchestrating Multi-Tool Chains - Tools, Agents, and Memory"** - you'll learn how to let the LLM call external APIs (e.g., a calculator or a database) and retain conversational context across turns.
- **Hands-on Challenge:** Extend the RAG service to support **image-based documents** (e.g., scanned contracts) using OCR + multimodal embeddings.

Happy building! 🚀

Designing FastAPI Services for LLM Inference

"A model is only as useful as the API that delivers its answers."

- When you wrap a large language model (LLM) in a well-engineered FastAPI service, you turn raw inference power into a reusable, observable, and scalable product.

In this chapter you will learn how to turn the RAG pipelines you built in Modules 4-5 into production-ready **FastAPI** services that:

- Accept and validate rich request payloads with **Pydantic**.
- Run inference **asynchronously** and **stream tokens** back to the client (think "ChatGPT-style" feel).
- Off-load long-running jobs to **background tasks** so the API stays responsive.
- Publish **OpenAPI/Swagger** documentation automatically - no extra effort required.

By the end you will have a self-contained, testable FastAPI app that can be dropped into a Docker container, Kubernetes pod, or serverless function.

Table of Contents

Section	What you'll learn
1? Introduction	Why FastAPI is a natural fit for LLM inference, and how it builds on the retrieval-augmented pipelines you already built.
2? Core Concepts	Async request handling, token streaming, request validation, background tasks, and auto-generated docs.
3? Practical Application	Step-by-step code walk-through: a minimal LLM service, streaming endpoint, background job manager, testing, and deployment tips.
4? Key Takeaways	A concise cheat-sheet you can refer to when you start building your own services.

1? Introduction

FastAPI has taken the Python web-framework world by storm because it:

- **Leverages modern Python typing** - the same type hints you already use in LangChain and Pydantic.
- **Runs natively on `asyncio`** - perfect for I/O-heavy workloads like token streaming over HTTP.
- **Generates OpenAPI schemas automatically**, giving you a Swagger UI without any extra configuration.

When you built **Retrieval-Augmented Generation (RAG)** pipelines in the previous modules, you already had three logical pieces:

1. **Vector Store** - FAISS, Chroma, or Pinecone (the knowledge base).
2. **Retriever** - a component that turns a user query into a set of relevant documents.
3. **LLM** - the generative engine that consumes the retrieved context and produces an answer.

All three can be wrapped in a **FastAPI service** that:

- Receives a JSON payload (`query`, optional `metadata`, `stream: bool`).
- Validates it with a Pydantic model.
- Calls the retriever synchronously (fast DB lookup).
- Calls the LLM **asynchronously**, optionally streaming each token as it arrives.
- Returns either a single JSON response (non-streaming) or a **Server-Sent Events (SSE)** stream (streaming).

The result is a clean separation of concerns:

```
client <--HTTP--> FastAPI router <--async--> LangChain chain <--sync--> Vector store
```

With this architecture you can:

- **Scale inference horizontally** - each FastAPI worker is stateless and can be replicated behind a load balancer.
- **Add observability** - request/response schemas are visible in Swagger, and you can plug in middleware for logging or tracing.
- **Swap out components** - replace OpenAI's GPT-4 with a locally hosted Llama-2 model without touching the API layer.

2. Core Concepts

2.1 Asynchronous Endpoints & Token Streaming

FastAPI's async support comes from **Starlette**, the underlying ASGI framework. An endpoint defined with `async def` can:

- **Yield control** back to the event loop while waiting for I/O (e.g., an HTTP request to OpenAI's API or a local model's CUDA inference).
- **Stream data** to the client using Python's async generators.

Why stream tokens?

- Gives the user a "real-time" chat feel.
- Reduces perceived latency - the client sees the first token after ~200 ms instead of waiting for the entire completion.

The two most common streaming patterns with FastAPI are:

Pattern	Transport	How it works
Server-Sent Events (SSE)	<code>text/event-stream</code>	The server pushes a line-delimited stream of JSON objects. Browsers have native <code>EventSource</code> support.
WebSockets	<code>ws://</code> or <code>wss://</code>	Full-duplex channel; useful when you need bidirectional communication (e.g., user can interrupt generation).

In this chapter we'll focus on **SSE** because it works over plain HTTP and integrates nicely with the OpenAI `stream=True` flag.

2.2 Pydantic Request Validation

FastAPI automatically parses the request body into a **Pydantic model**. Benefits include:

- **Typed fields** (e.g., `query: str`, `top_k: int = 5`).
- **Validators** (`@validator` methods) for custom logic (e.g., max length, prohibited words).
- **Schema generation** - the model's JSON schema appears in the Swagger UI.

Typical request model for an LLM endpoint:

```
from pydantic import BaseModel, Field, validator
from typing import Optional, List

class LLMRequest(BaseModel):
    query: str = Field(..., description="User prompt that drives generation")
    top_k: int = Field(5, ge=1, le=20, description="Number of retrieved docs")
    stream: bool = Field(False, description="Whether to stream tokens via SSE")
    metadata: Optional[dict] = Field(
        None,
        description="Optional key-value pairs that the downstream chain can use"
    )

    @validator('query')
    def limit_query_length(cls, v):
        max_len = 1024
        if len(v) > max_len:
            raise ValueError(f'Query length exceeds {max_len} characters')
        return v
```

FastAPI will return a **422 Unprocessable Entity** response if validation fails, with a helpful JSON body describing each field's error.

2.3 Background Tasks for Long-Running Jobs

Even with async streaming, some LLM workloads are **CPU-bound** or **GPU-bound** and may take several seconds to minutes (e.g., large context windows, fine-tuned models). You don't want a single request to block the worker loop.

FastAPI provides a ``BackgroundTasks`` dependency that:

- Schedules a callable to run **after the response is sent**.
- Runs in the same event loop (no separate thread) - ideal for I/O, but you can wrap CPU-heavy work in a thread pool (`run_in_executor`).

Typical usage pattern:

```
from fastapi import BackgroundTasks

def store_result(job_id: str, result: str):
    # Persist result to DB, cache, or a message queue
    ...

@app.post("/generate")
async def generate(request: LLMRequest, background_tasks: BackgroundTasks):
    job_id = uuid4().hex
    # Fire-and-forget: start generation in background
    background_tasks.add_task(
        run_generation_job,
        job_id,
        request.query,
        request.top_k,
        request.metadata,
        stream=False # background jobs are non-streaming
    )
    return {"job_id": job_id, "status": "queued"}
```

Clients can later **poll** an endpoint (`GET /jobs/{job_id}`) to retrieve the finished result. This pattern is essential for batch jobs, document-level summarization, or any task that exceeds typical request-response timeouts.

2.4 Automatic OpenAPI Documentation

FastAPI introspects:

- **Path operations** (`@app.get`, `@app.post`, ...)
- **Pydantic models** used as request bodies or responses
- **Dependency signatures** (e.g., `BackgroundTasks`)

...and builds a **OpenAPI 3.1** schema automatically. Visiting `http://localhost:8000/docs` gives you a **Swagger UI**; `http://localhost:8000/redoc` gives a **ReDoc** view. You can enhance docs with:

- `summary` and `description` arguments on the route decorator.
- `response_model` to document the shape of the successful response.

- `responses={404: {"description": "Not found"}}` for error cases.

Example:

```
@app.post(
    "/generate",
    summary="Generate text from a prompt",
    description="Runs a retrieval-augmented generation pipeline. "
    "If `stream=True`, tokens are streamed via SSE.",
    response_model=GenerationResponse,
    responses={422: {"description": "Validation error"}}
)
async def generate(...):
    ...
```

Swagger will now show the **example payload** and the **response schema**, making your API self-documenting for front-end developers and third-party integrators.

3.2 Practical Application

Below we build a **complete FastAPI service** that:

- Loads a **FAISS vector store** (from Module 4).
- Wraps a **LangChain RAG chain** (retriever → LLM).
- Exposes two endpoints:
 1. **POST /chat** - synchronous but optionally streams tokens.
 2. **POST /jobs** - fire-and-forget background generation (returns a job ID).
- Provides a **job status endpoint** (GET /jobs/{job_id}).

3.1 Project Layout

```
llm_service/  
├── app/  
│   ├── __init__.py  
│   ├── main.py          # FastAPI entry point  
│   ├── models.py        # Pydantic request/response schemas  
│   ├── chain.py          # LangChain RAG pipeline construction  
│   ├── background.py     # Background task utilities  
│   └── store.py          # Simple in-memory job store (for demo)  
├── requirements.txt  
└── Dockerfile
```

Tip: Keep the LangChain chain creation outside the request handlers so it's instantiated once at startup - otherwise you'll reload the vector store on every request.

3.2 requirements.txt

```
fastapi[all]==0.110.0    # includes uvicorn, pydantic, starlette  
langchain==0.1.12  
faiss-cpu==1.7.4        # or faiss-gpu if you have CUDA  
openai==1.12.0          # optional, for OpenAI LLMs  
python-dotenv==1.0.0    # for env vars  
uvicorn[standard]==0.27.0
```

3.3 app/models.py - Pydantic Schemas

```

from pydantic import BaseModel, Field, validator
from typing import Optional, List, Dict, Any

class LLMRequest(BaseModel):
    query: str = Field(..., description="User prompt")
    top_k: int = Field(5, ge=1, le=20, description="Number of docs to retrieve")
    stream: bool = Field(False, description="Stream tokens via SSE")
    metadata: Optional[Dict[str, Any]] = Field(
        None,
        description="Arbitrary key-value pairs passed to downstream chain"
    )

    @validator('query')
    def limit_len(cls, v):
        max_len = 2048
        if len(v) > max_len:
            raise ValueError(f'Query exceeds {max_len} characters')
        return v

class GenerationChunk(BaseModel):
    """One token or chunk of text streamed to the client."""
    token: str
    index: int # position of token in the final output

class GenerationResponse(BaseModel):
    job_id: Optional[str] = Field(
        None,
        description="If generated in background, the identifier to poll later"
    )
    answer: Optional[str] = Field(
        None,
        description="Full generated text (when stream=False)"
    )
    streamed: bool = Field(
        False,
        description="True if the response was streamed via SSE"
    )

```

3.4 app/store.py - Simple In-Memory Job Store

Production note: Replace this with Redis, PostgreSQL, or a cloud datastore.

```
import threading
from typing import Dict, Optional
from dataclasses import dataclass, field

@dataclass
class JobResult:
    status: str = "queued"          # queued | running | completed | failed
    answer: Optional[str] = None
    error: Optional[str] = None

class JobStore:
    """Thread-safe in-memory store for demo purposes."""
    def __init__(self):
        self._store: Dict[str, JobResult] = {}
        self._lock = threading.Lock()

    def create(self, job_id: str) -> None:
        with self._lock:
            self._store[job_id] = JobResult(status="queued")

    def update(self, job_id: str, **kwargs) -> None:
        with self._lock:
            job = self._store.get(job_id)
            if not job:
                raise KeyError(job_id)
            for k, v in kwargs.items():
                setattr(job, k, v)

    def get(self, job_id: str) -> JobResult:
        with self._lock:
            if job_id not in self._store:
                raise KeyError(job_id)
            return self._store[job_id]

job_store = JobStore()
```

3.5 app/chain.py - Building the LangChain RAG Pipeline

```

import os
from pathlib import Path
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA
from langchain.schema import Document

# Load environment variables (e.g., OPENAI_API_KEY)
from dotenv import load_dotenv
load_dotenv()

# -----
# 1️⃣ Vector store initialization (FAISS)
# -----
def load_faiss_store(persist_dir: Path = Path("./data/faiss_index")) -> FAISS:
    """Load a persisted FAISS index; create a dummy one if missing."""
    if persist_dir.is_dir():
        print(f"Loading FAISS index from {persist_dir}")
        return FAISS.load_local(str(persist_dir), OpenAIEmbeddings())
    else:
        # For demo: create a tiny index with a few hard-coded docs
        print("FAISS index not found - creating a temporary one.")
        docs = [
            Document(page_content="FastAPI is a modern, fast web framework for building APIs with Python.", metadata={"source": "fastapi.org"}),
            Document(page_content="LangChain helps developers build LLM-powered applications by chaining together components.", metadata={"source": "langchain.com"}),
            Document(page_content="FAISS enables efficient similarity search on dense vectors.", metadata={"source": "faiss.ai"}),
        ]
        vect = FAISS.from_documents(docs, OpenAIEmbeddings())
        vect.save_local(str(persist_dir))
        return vect

# -----
# 2️⃣ LLM initialization (OpenAI or local)
# -----
def get_llm():
    """Return an LLM instance; we use OpenAI for the demo."""
    return OpenAI(model_name="gpt-4o-mini", temperature=0.2, streaming=False)

# -----

```

... (continued on next page)


```
# 3. RetrievalQA chain (retriever + LLM)
# -----
def build_rag_chain(top_k: int = 5):
    vect = load_faiss_store()
    retriever = vect.as_retriever(search_kwargs={"k": top_k})
    llm = get_llm()
    chain = RetrievalQA.from_chain_type(
        llm,
        chain_type="stuff", # "map_reduce", "refine", etc.
        retriever=retriever,
        return_source_documents=False
    )
    return chain
```

Why not create the chain inside the request handler? Because the vector store load can take seconds, and the LLM client performs a network handshake. Instantiating it once during startup keeps request latency low.

3.6 app/background.py - Background Generation Logic

```
import asyncio
from uuid import uuid4
from .chain import build_rag_chain
from .store import job_store

async def run_generation_job(
    job_id: str,
    query: str,
    top_k: int,
    metadata: dict | None,
    stream: bool = False,
) -> None:
    """
    Execute the RAG chain in the background.
    Updates the global JobStore with status & result.
    """
    try:
        job_store.update(job_id, status="running")
        # Build a fresh chain with the requested `top_k`
        chain = build_rag_chain(top_k=top_k)

        # LangChain's `invoke` returns the answer synchronously.
        # If you need true streaming, you would call `acall` with `streaming=True`.
        answer = await asyncio.to_thread(chain.run, {"question": query})
        job_store.update(job_id, status="completed", answer=answer)
    except Exception as exc:
        job_store.update(job_id, status="failed", error=str(exc))
```

Note: The `await asyncio.to_thread` wrapper moves the blocking `chain.run` call off the event loop, preventing the FastAPI worker from being blocked. For truly async LLMs (e.g., OpenAI's `acreate` with `stream=True`), you can call the async method directly.

3.7 app/main.py - FastAPI Application

```

import uuid
from fastapi import FastAPI, BackgroundTasks, Request, HTTPException, status
from fastapi.responses import StreamingResponse, JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from typing import AsyncGenerator

from .models import LLMRequest, GenerationResponse, GenerationChunk
from .chain import build_rag_chain
from .background import run_generation_job
from .store import job_store

app = FastAPI(
    title="LLM Inference Service",
    version="0.1.0",
    description="FastAPI wrapper around a LangChain Retrieval-Augmented Generation pipeline."
)

# Optional: allow a front-end on a different origin to call the API
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # tighten for production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# -----
# Helper: async generator that yields SSE events
# -----
async def sse_token_stream(chain, query: str) -> AsyncGenerator[bytes, None]:
    """
    Calls the LLM with `stream=True` and yields Server-Sent Events.
    Each event is a JSON line containing a token and its index.
    """
    # LangChain's `acall` with `streaming=True` returns an async generator of tokens.
    # Here we assume the underlying LLM client supports streaming.
    async for token in chain.acall({"question": query}, callbacks=None, stream=True):
        # token is a string fragment; we wrap it in a JSON payload.
        chunk = GenerationChunk(token=token, index=0) # index handling simplified
        # SSE spec: "data: <payload>\n\n"

```

... (continued on next page)

```

        yield f"data: {chunk.json()}\n\n".encode("utf-8")

# -----
# 1❏ chat endpoint - synchronous or streaming
# -----
@app.post(
    "/chat",
    response_model=GenerationResponse,
    summary="Generate answer (optionally streamed)",
    description="Runs the RAG chain. Set `stream=true` to receive Server-Sent Events.",
)
async def chat_endpoint(
    request: LLMRequest,
    background_tasks: BackgroundTasks,
    raw_request: Request,
):
    # FastAPI already validated `request` against LLMRequest schema.
    if request.stream:
        # Build a chain that supports streaming; we set `streaming=True` in the LLM.
        chain = build_rag_chain(top_k=request.top_k)

        # Return an SSE streaming response.
        # Note: FastAPI automatically sets `Content-Type: text/event-stream`.
        return StreamingResponse(
            sse_token_stream(chain, request.query),
            media_type="text/event-stream",
        )
    else:
        # Non-streaming: run generation synchronously (but off the event loop).
        chain = build_rag_chain(top_k=request.top_k)

        # Offload the blocking call to a thread pool.
        answer = await asyncio.to_thread(chain.run, {"question": request.query})
        return GenerationResponse(answer=answer, streamed=False)

# -----
# 2❏ jobs - fire-and-forget background generation
# -----
@app.post(
    "/jobs",

```

... (continued on next page)

```

        response_model=GenerationResponse,
        status_code=status.HTTP_202_ACCEPTED,
        summary="Queue a generation job",
        description="Accepts a prompt and returns a job_id. Use GET /jobs/{job_id} to poll for results."
    )
    async def create_job(
        request: LLMRequest,
        background_tasks: BackgroundTasks,
    ):
        job_id = uuid.uuid4().hex
        job_store.create(job_id)

        # Schedule the background coroutine.
        background_tasks.add_task(
            run_generation_job,
            job_id,
            request.query,
            request.top_k,
            request.metadata,
            stream=False,
        )
        return GenerationResponse(job_id=job_id, streamed=False)

# -----
# 3 GET /jobs/{job_id} - poll for status / result
# -----
@app.get(
    "/jobs/{job_id}",
    response_model=GenerationResponse,
    summary="Check job status",
    description="Returns the current status and, if completed, the generated answer."
)
async def get_job(job_id: str):
    try:
        job = job_store.get(job_id)
    except KeyError:
        raise HTTPException(status_code=404, detail="Job not found")

    if job.status == "completed":
        return GenerationResponse(answer=job.answer, streamed=False)

```

... (continued on next page)

```
elif job.status == "failed":
    raise HTTPException(status_code=500, detail=f"Job failed: {job.error}")
else:
    # queued or running
    return JSONResponse(
        content={"job_id": job_id, "status": job.status},
        status_code=202,
    )
```

What does this file teach you?

1. **FastAPI startup wiring** - the chain is built lazily inside each request but re-uses the same vector store.
2. **Conditional streaming** - if `request.stream` returns a `StreamingResponse` that yields **SSE** events.
3. **Background task definition** - `BackgroundTasks.add_task` queues a coroutine that updates the global job store.
4. **OpenAPI customization** - each route includes a `summary` and `description`; the response models automatically appear in Swagger.

3.8 Running the Service Locally

```
# 1 Create a virtual environment
python -m venv .venv
source .venv/bin/activate

# 2 Install dependencies
pip install -r requirements.txt

# 3 Set your OpenAI key (or any other LLM credentials)
export OPENAI_API_KEY="sk-..."

# 4 Launch the API
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Visit `http://localhost:8000/docs` - you'll see a fully-documented Swagger UI with the three endpoints.

Quick test with curl

Non-streaming request

```
curl -X POST "http://localhost:8000/chat" \
  -H "Content-Type: application/json" \
  -d '{"query":"What is FastAPI?","top_k":3,"stream":false}'
```

Streaming request (SSE)

```
curl -N -X POST "http://localhost:8000/chat" \
  -H "Content-Type: application/json" \
  -d '{"query":"Explain vector stores in 2 sentences","top_k":2,"stream":true}'
```

You'll see a series of lines like:

```
data: {"token":"Vector","index":0}
data: {"token":" stores","index":0}
...
```

Background job

```
# 1❏ Submit job
JOB_ID=$(curl -s -X POST "http://localhost:8000/jobs" \
  -H "Content-Type: application/json" \
  -d '{"query":"Summarize the LangChain docs","top_k":5,"stream":false}' \
  | jq -r .job_id)

# 2❏ Poll until completed
while true; do
  STATUS=$(curl -s "http://localhost:8000/jobs/$JOB_ID")
  echo "$STATUS"
  [[ "$STATUS" == *"answer"* ]] && break
  sleep 2
done
```

3.9 Testing the Service

FastAPI ships with `TestClient` (based on `httpx`) that lets you write unit tests without running a server.


```
# tests/test_api.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_chat_non_stream():
    payload = {"query": "What is FAISS?", "top_k": 2, "stream": False}
    resp = client.post("/chat", json=payload)
    assert resp.status_code == 200
    data = resp.json()
    assert "answer" in data
    assert "FAISS" in data["answer"]

def test_chat_stream():
    payload = {"query": "Say the word 'hello' three times.", "top_k": 1, "stream": True}
    with client.stream("POST", "/chat", json=payload) as resp:
        assert resp.status_code == 200
        tokens = []
        for line in resp.iter_lines():
            if line.startswith(b"data:"):
                json_payload = line[5:].strip()
                token_obj = json.loads(json_payload)
                tokens.append(token_obj["token"])
        assert "".join(tokens).strip() == "hello hello hello"
```

Run with `pytest -q`. These tests confirm both the **synchronous** and **streaming** paths work as expected.

3.10 Deploying to Production

3.10.1 Containerization

Dockerfile (minimal, multi-stage build):

```
# ---- Builder ----
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# ---- Runtime ----
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.12/site-packages /usr/local/lib/python3.12/site-packages
COPY app/ ./app/
ENV PORT=8000
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build & run:

```
docker build -t llm-service .
docker run -p 8000:8000 -e OPENAI_API_KEY=$OPENAI_API_KEY llm-service
```

3.10.2 Scaling with Gunicorn + Uvicorn Workers

For higher throughput you can launch multiple ASGI workers:

```
gunicorn app.main:app \
    -k uvicorn.workers.UvicornWorker \
    --workers 4 \
    --bind 0.0.0.0:8000
```

Each worker gets its own event loop; the vector store is loaded **per worker** (still cheap for FAISS). For **GPU-accelerated models**, you may need to **pin each worker to a specific GPU** using environment variables (`CUDA_VISIBLE_DEVICES`).

3.10.3 Observability

FastAPI integrates nicely with **OpenTelemetry**, **Prometheus**, and **Sentry**. Example

snippets (not exhaustive):

```
# app/monitoring.py
from prometheus_client import Counter, Histogram
from fastapi import Request

REQUEST_COUNT = Counter(
    "http_requests_total", "Total HTTP requests", ["method", "endpoint", "status"]
)
REQUEST_LATENCY = Histogram(
    "http_request_duration_seconds", "Request latency", ["endpoint"]
)

async def metrics_middleware(request: Request, call_next):
    method = request.method
    endpoint = request.url.path
    with REQUEST_LATENCY.labels(endpoint).time():
        response = await call_next(request)
    REQUEST_COUNT.labels(method, endpoint, response.status_code).inc()
    return response
```

Add the middleware to `app/main.py`:

```
app.middleware("http")(metrics_middleware)
```

Now Prometheus can scrape `/metrics` (expose via `PrometheusFastAPI` or a separate endpoint).

4 Key Takeaways

Concept	What you learned	How to apply it
FastAPI + asyncio	Async endpoints keep the event loop free while waiting for I/O (LLM calls, DB look-ups).	Declare <code>async def</code> route handlers; use <code>await</code> for async client calls or <code>asyncio.to_thread</code> for

❓ Concept	❓ What you learned	❓ How to apply it
		blocking code.
Token streaming (SSE)	Server-Sent Events let you push each generated token to the client, mimicking a chat UI.	Return <code>StreamingResponse</code> with an async generator that yields data: <code>{...}\n\n</code> .
Pydantic validation	Guarantees request shape, auto-generates OpenAPI schemas, and gives clear 422 errors.	Define request models (<code>LLMRequest</code>) with field constraints and custom validators.
Background tasks	Off-load long-running inference to a fire-and-forget job while returning a job ID.	Use <code>BackgroundTasks.add_task</code> and a persistent store (Redis, DB) to track status.
OpenAPI auto-docs	Swagger UI is generated without extra code; great for internal teams and external partners.	Add <code>summary</code> , <code>description</code> , <code>response_model</code> to route decorators; visit <code>/docs</code> .
Testing with TestClient	Unit test both sync and streaming endpoints without a live server.	Use <code>client.post</code> for normal responses; <code>client.stream</code> for SSE.
Deployment patterns	Containerize, run multiple workers, and instrument with Prometheus/Sentry for production.	Multi-stage Dockerfile; <code>gunicorn -k uvicorn.workers.UvicornWorker</code> ; middleware for metrics.

Quick Checklist for Your Next LLM Service

1. Define a clear request schema (`BaseModel`).
2. Instantiate heavy components once (vector store, embeddings, LLM).
3. Decide streaming vs. background based on latency requirements.
4. Wrap blocking calls with `asyncio.to_thread` or use truly async SDKs.
5. Add observability (metrics, logging, error tracking).
6. Write tests for both paths.
7. Containerize and spin up multiple workers before scaling to production.

You now have a **battle-tested blueprint** for turning any LangChain RAG pipeline-or a plain LLM call-into a robust, documented, and scalable FastAPI service. In the next module we'll explore **advanced orchestration** with Celery and message queues, enabling truly distributed inference pipelines that can handle thousands of concurrent requests. Happy coding!

Integrating LangChain with FastAPI: End-to-End Application

Module 7 of 9 - "FromPrototype to Production"

Introduction

In the first half of this book you learned how to **design, build, and evaluate** Retrieval-Augmented Generation (RAG) pipelines with LangChain. You also explored the fundamentals of exposing a language model behind a **FastAPI** service - a lightweight, async-first web framework that has become the de-facto standard for Python-based AI micro-services.

Now it's time to **bring those two worlds together**: we will wire a complete LangChain RAG chain into a FastAPI route, make the endpoint **concurrency-aware**, protect it with **rate-limiting**, and expose the most common LLM knobs (model name, temperature, top-k) as **query-string parameters**. Finally we will demonstrate how to **unit-test** the API layer and write a **simple integration test** that exercises the whole stack from HTTP request to LLM response.

By the end of this chapter you will have a **production-ready skeleton** that you can clone, extend, and deploy to any cloud platform (Docker, Kubernetes, or serverless). The code is deliberately modular: the LangChain chain lives in its own module, the FastAPI app lives in another, and the test suite lives in a third. This separation of concerns will make it easy to:

- Swap out a vector store or LLM without touching the API code.
- Scale the service horizontally while preserving request-level isolation.
- Add observability (logging, metrics, tracing) later with minimal friction.

Let's dive in.

Core Concepts

Concept	Why it matters for LLM-powered services	Quick TL;DR
Async-first design	LLM inference (especially remote APIs) is I/O-bound. Blocking the event loop wastes CPU and throttles throughput.	Use <code>async def</code> endpoints, <code>httpx.AsyncClient</code> , and async LangChain components when possible.
Dependency injection	FastAPI's dependency system lets you create per-request objects (e.g., a fresh chain, a DB session) and also global singletons (e.g., a shared HTTP client).	Declare a <code>Depends</code> function that builds the chain; FastAPI will call it for each request.
Rate-limiting	Public LLM APIs (OpenAI, Anthropic, etc.) enforce quotas; uncontrolled traffic can quickly hit limits and incur cost spikes.	Implement a lightweight token bucket using <code>aioredis</code> or an in-memory fallback for local dev.
Configurable inference parameters	Prompt engineering is only half the story; temperature, top-k, max tokens, etc., dramatically affect output quality and cost.	Expose them as optional query parameters with sensible defaults and validation via Pydantic.
Testing the API layer	Unit tests guarantee that request parsing, validation, and error handling work; integration tests guarantee the chain returns sensible answers.	Use <code>httpx.AsyncClient</code> with <code>TestClient</code> for fast, in-process testing.

Concept	Why it matters for LLM-powered services	Quick TL;DR
Observability hooks	Production systems need logs, metrics, and tracing to debug model drift or latency spikes.	The chapter shows where to plug in structlog and Prometheus exporters without cluttering core logic.

Key mental model: FastAPI = request router + dependency graph; LangChain = chain graph of LLM-aware components.

The bridge between them is a dependency that builds a chain instance per request and returns the answer synchronously (or asynchronously) to the HTTP layer.

Practical Application

Below you will find a **complete, runnable example** that we will develop step-by-step. The final directory layout looks like this:

```

❏ -app/
| ❏ __init__.py
| ❏ -main.py          # FastAPI entry point
| ❏ -api/
| | ❏ __init__.py
| |   └ rag.py        # HTTP route + dependencies
| ❏ -chains/
| | ❏ __init__.py
| |   └ rag_chain.py  # LangChain RAG pipeline
| ❏ -config/
| | ❏ __init__.py
| |   └ settings.py   # Pydantic settings (env vars, defaults)
|   └ utils/
|     ❏ __init__.py
|     └ rate_limiter.py # Simple async token bucket
❏ -tests/
| ❏ __init__.py
| ❏ -unit/
| |   └ test_rag_route.py
|   └ integration/
|     └ test_full_flow.py
❏ -requirements.txt
└ README.md

```

Note: The code uses Python 3.11 features (pattern matching, `typing.Self`) but works fine on 3.9+ with minor adjustments.

1❏ Setting the Stage - Project Dependencies

Add the following to `requirements.txt` (or use `poetry add`):


```
fastapi==0.110.0
uvicorn[standard]==0.27.0
httpx==0.27.0
langchain==0.1.13
langchain-community==0.0.30
pydantic==2.6.3
python-dotenv==1.0.1
aioredis==2.0.1          # optional, for production-grade rate limiting
structlog==24.1.0
prometheus-client==0.20.0
pytest==8.1.1
pytest-asyncio==0.23.6
```

Install with:

```
pip install -r requirements.txt
```

2.2 Centralized Settings

FastAPI works beautifully with **Pydantic Settings**. Create `app/config/settings.py`:

```

# app/config/settings.py
from pydantic import BaseSettings, Field, SecretStr, model_validator
from typing import Literal

class Settings(BaseSettings):
    # -----
    # LLM configuration
    # -----
    model_name: str = Field(
        "gpt-3.5-turbo",
        description="OpenAI model name (or any provider-specific identifier)."
    )
    temperature: float = Field(
        0.7,
        ge=0.0,
        le=2.0,
        description="Sampling temperature; higher = more creative."
    )
    top_k: int = Field(
        40,
        ge=1,
        description="Number of tokens to consider at each sampling step."
    )
    max_tokens: int = Field(
        512,
        ge=1,
        description="Maximum tokens in the model's completion."
    )
    # -----
    # Vector store / retriever settings
    # -----
    # For this demo we use a simple Chroma in-memory DB.
    chroma_persist_dir: str = Field(
        "./chroma_db",
        description="Directory where Chroma persists its collection."
    )
    # -----
    # Rate limiting
    # -----
    rate_limit_per_minute: int = Field(

```

... (continued on next page)

```

        30,
        description="Maximum requests per minute per client IP."
    )
    redis_url: str | None = Field(
        default=None,
        description="If set, use Redis for distributed rate limiting."
    )
    # -----
    # Misc
    # -----
    log_level: Literal["DEBUG", "INFO", "WARN", "ERROR"] = Field(
        "INFO", description="Application log level."
    )

class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"

    @model_validator(mode="after")
    def _check_redis(self):
        # Simple guard: if redis_url is provided, it must be a valid URI.
        if self.redis_url and not self.redis_url.startswith("redis://"):
            raise ValueError("redis_url must start with redis://")
        return self

```

Create a `.env` file (add to `.gitignore` in real projects):

```

# .env
OPENAI_API_KEY=sk-....
# Optional: REDIS_URL=redis://localhost:6379/0

```

Now you can import a **singleton** settings anywhere:

```

# app/__init__.py
from .config.settings import Settings

settings = Settings()

```

3? Building the LangChain RAG Pipeline

We keep the chain in its own module so that the FastAPI layer never needs to know **how** the answer is generated-only **that** it can call `run(query, **kwargs)`.

```
# app/chains/rag_chain.py
from __future__ import annotations

from typing import Any, Dict, List, Tuple
import os

from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import OpenAIEmbeddings
from langchain import PromptTemplate, LLMChain
from langchain.schema import Document
from langchain.llms import OpenAI
from langchain.retrievers import MultiQueryRetriever, BM25Retriever
from langchain.chains import RetrievalQA
from langchain.text_splitter import RecursiveCharacterTextSplitter

# -----
# Helper to (re)create the vector store - in a real project this would be
# a separate ingestion script. Here we embed a tiny static corpus.
# -----
def _initialize_vector_store(persist_dir: str) -> Chroma:
    # Only populate if empty
    if not os.path.isdir(persist_dir) or not os.listdir(persist_dir):
        docs = [
            Document(page_content="Python is a high-level, interpreted programming language."),
            Document(page_content="FastAPI is a modern, fast (high-performance) web framework for building APIs with Python 3.6+."),
            Document(page_content="LangChain provides composable abstractions for LLM-centric applications."),
            Document(page_content="Retrieval-Augmented Generation (RAG) combines a knowledge base with a language model."),
        ]
        splitter = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=20)
        chunks = splitter.split_documents(docs)

        embeddings = OpenAIEmbeddings()
        vectordb = Chroma.from_documents(
            documents=chunks,
            embedding=embeddings,
            persist_directory=persist_dir,
        )
        vectordb.persist()
    else:
        embeddings = OpenAIEmbeddings()
```

... (continued on next page)

```

        vectordb = Chroma(
            embedding_function=embeddings,
            persist_directory=persist_dir,
        )
    return vectordb

# -----
# Core RAG chain - returns a callable object with a .run() method.
# -----
class RAGChain:
    """
    A thin wrapper around LangChain's RetrievalQA that lets us inject
    model-level parameters at call time (temperature, top_k, max_tokens).
    """
    def __init__(self, *, model_name: str, temperature: float, top_k: int,
                 max_tokens: int, persist_dir: str):
        # 1 LLM
        self.llm = OpenAI(
            model_name=model_name,
            temperature=temperature,
            max_tokens=max_tokens,
            top_p=1.0, # default - we expose top_k via the retriever instead
        )

        # 2 Vector store & retriever
        vectordb = _initialize_vector_store(persist_dir)
        # Use a simple similarity retriever; you can swap for MMR, etc.
        self.retriever = vectordb.as_retriever(search_kwargs={"k": top_k})

        # 3 Prompt template - feel free to customize.
        PROMPT = PromptTemplate(
            template=(
                "You are a helpful AI assistant. Use the following pieces of context "
                "to answer the question at the end. If you don't know the answer, say "
                "\"I don't know\".\n\n"
                "Context:\n{context}\n\nQuestion: {question}\nAnswer:"
            ),
            input_variables=["context", "question"],
        )

```

... (continued on next page)

```

# 4 RetrievalQA chain
self.qa_chain = RetrievalQA.from_chain_type(
    llm=self.llm,
    retriever=self.retriever,
    chain_type="stuff", # "stuff" concatenates docs; other types exist.
    return_source_documents=True,
    chain_type_kwargs={"prompt": PROMPT},
)

def run(self, query: str, *, temperature: float | None = None,
        top_k: int | None = None, max_tokens: int | None = None) -> Dict[str, Any]:
    """
    Execute the chain with optional per-call overrides.
    """
    # Update per-call LLM params if supplied (LangChain 0.1.x allows .with() )
    if temperature is not None:
        self.llm.temperature = temperature
    if max_tokens is not None:
        self.llm.max_tokens = max_tokens
    if top_k is not None:
        self.retriever.search_kwargs["k"] = top_k

    # LangChain's RetrievalQA returns a dict: {"result": ..., "source_documents": ...}
    response = self.qa_chain({"query": query})
    return {
        "answer": response["result"],
        "sources": [
            {"page_content": doc.page_content, "metadata": doc.metadata}
            for doc in response["source_documents"]
        ],
    }

```

Key points to notice:

- The constructor **instantiates a single LLM** and **vector store** that can be reused across requests.
- The `run` method **mutates** only the lightweight attributes (temperature, top_k) - no need to rebuild the whole chain.
- The chain returns both the answer and the source documents, a best-practice for transparency.

4.2 Rate Limiter - Token Bucket Implementation

For local development we can keep everything **in-memory**. For production we'll optionally back the limiter with **Redis**, allowing multiple FastAPI workers to share the same quota.

```
# app/utils/rate_limiter.py
import asyncio
import time
from collections import defaultdict
from typing import Awaitable

try:
    import aioredis
except ImportError: # pragma: no cover
    aioredis = None # type: ignore

class InMemoryTokenBucket:
    """
    Simple per-IP token bucket stored in a dict.
    Not safe for multi-process, but perfect for dev / unit tests.
    """
    def __init__(self, capacity: int, refill_per_sec: float):
        self.capacity = capacity
        self.refill_per_sec = refill_per_sec
        self.tokens: dict[str, float] = defaultdict(lambda: capacity)
        self.last_refill: dict[str, float] = defaultdict(time.time)
        self.lock = asyncio.Lock()

    async def allow(self, key: str) -> bool:
        async with self.lock:
            now = time.time()
            elapsed = now - self.last_refill[key]
            # Refill tokens based on elapsed time
            refill = elapsed * self.refill_per_sec
            self.tokens[key] = min(self.capacity, self.tokens[key] + refill)
            self.last_refill[key] = now
            if self.tokens[key] >= 1:
                self.tokens[key] -= 1
                return True
            return False

class RedisTokenBucket:
    """
```


... (continued on next page)

```

Distributed token bucket using Redis INCR and EXPIRE.
Each key represents a client (IP) with a TTL of 60 seconds.
"""
def __init__(self, redis_url: str, capacity: int):
    if not aioredis:
        raise RuntimeError("aioredis is required for RedisTokenBucket")
    self.capacity = capacity
    self.redis = aioredis.from_url(redis_url, decode_responses=True)

    async def allow(self, key: str) -> bool:
        # Lua script guarantees atomicity: increment and check capacity.
        script = """
        local current = redis.call('INCR', KEYS[1])
        if current == 1 then
            redis.call('EXPIRE', KEYS[1], 60)
        end
        if current > tonumber(ARGV[1]) then
            return 0
        else
            return 1
        end
        """
        allowed = await self.redis.eval(script, 1, key, self.capacity)
        return bool(allowed)

```

The bucket **refills** at a rate derived from the per-minute limit:

```

# In the FastAPI dependency (see next section)
capacity = settings.rate_limit_per_minute
refill_per_sec = capacity / 60.0

```

5? FastAPI Layer - Routes, Dependencies, and Validation

Now we tie everything together in `app/api/rag.py`. This file defines:

- a **dependency that creates a `RAGChain` instance** (cached per-process).
- a **dependency that enforces rate limiting** based on the client IP.

- the **HTTP endpoint** that accepts query parameters, validates them, and returns a JSON payload.

```
# app/api/rag.py
from fastapi import APIRouter, Depends, HTTPException, Query, Request, status
from pydantic import BaseModel, Field, PositiveInt, conint, confloat
from typing import Annotated, Literal, Dict, Any

from ..config import settings
from ..chains.rag_chain import RAGChain
from ..utils.rate_limiter import InMemoryTokenBucket, RedisTokenBucket

router = APIRouter()

# -----
# Pydantic model for the response payload
# -----
class SourceDoc(BaseModel):
    page_content: str
    metadata: Dict[str, Any] = Field(default_factory=dict)

class RAGResponse(BaseModel):
    answer: str
    sources: list[SourceDoc]

# -----
# Dependency that returns a shared RAGChain (singleton per process)
# -----
def get_rag_chain() -> RAGChain:
    # The settings object is a singleton - we reuse it.
    return RAGChain(
        model_name=settings.model_name,
        temperature=settings.temperature,
        top_k=settings.top_k,
        max_tokens=settings.max_tokens,
        persist_dir=settings.chroma_persist_dir,
    )

# -----
# Rate-limiting dependency - raises 429 if limit exceeded
```

... (continued on next page)

```

# -----
async def rate_limiter(request: Request) -> None:
    client_ip = request.client.host
    # Choose bucket implementation based on config
    if settings.redis_url:
        bucket = RedisTokenBucket(settings.redis_url, settings.rate_limit_per_minute)
    else:
        # In-memory bucket; we keep a single instance per process.
        # FastAPI caches dependencies when `cache=True` (default for non-async).
        bucket = request.app.state.rate_bucket # type: ignore[attr-defined]

    allowed = await bucket.allow(client_ip)
    if not allowed:
        raise HTTPException(
            status_code=status.HTTP_429_TOO_MANY_REQUESTS,
            detail="Rate limit exceeded. Try again later."
        )

# -----
# Register a startup event to initialise the in-memory bucket
# -----
@router.on_event("startup")
async def startup_event():
    # Attach bucket to the app for reuse across requests
    capacity = settings.rate_limit_per_minute
    refill = capacity / 60.0
    bucket = InMemoryTokenBucket(capacity, refill)
    # `state` is a generic object you can stash anything on.
    router.app.state.rate_bucket = bucket # type: ignore[attr-defined]

# -----
# The actual endpoint - GET /rag?query=...&temperature=...&top_k=...
# -----
@router.get(
    "/rag",
    response_model=RAGResponse,
    summary="Ask a question using a Retrieval-Augmented Generation chain.",
    tags=["RAG"],

```

... (continued on next page)

```

)
async def ask_rag(
    request: Request,
    query: Annotated[
        str,
        Query(..., description="The user question to be answered by the LLM.")
    ],
    temperature: Annotated[
        confloat(ge=0.0, le=2.0) | None,
        Query(
            default=None,
            description="Sampling temperature (overrides global setting)."
        ),
    ] = None,
    top_k: Annotated[
        PositiveInt | None,
        Query(
            default=None,
            description="Number of retrieved chunks to feed the model."
        ),
    ] = None,
    max_tokens: Annotated[
        PositiveInt | None,
        Query(
            default=None,
            description="Maximum tokens in the model's completion."
        ),
    ] = None,
    rag_chain: RAGChain = Depends(get_rag_chain),
    _: None = Depends(rate_limiter), # Force rate-limiting before any logic
) -> RAGResponse:
    """
    **Workflow**

    1⌚ FastAPI validates query-string parameters (type, bounds).
    2⌚ `rate_limiter` ensures the client hasn't exceeded the per-minute quota.
    3⌚ The `RAGChain` is called with the user query and any override parameters.
    4⌚ The response is serialised to JSON using the Pydantic model above.
    """
    try:
        result = rag_chain.run(

```

... (continued on next page)

```

        query,
        temperature=temperature,
        top_k=top_k,
        max_tokens=max_tokens,
    )
except Exception as exc:
    # Convert any LangChain-level errors to a 500 with a helpful message.
    raise HTTPException(
        status_code=500,
        detail=f"Error while processing the request: {exc}",
    ) from exc

return RAGResponse(answer=result["answer"], sources=result["sources"])

```

Explanation of the FastAPI magic:

Piece	What it does
<code>Annotated[... , Query(...)]</code>	Declares a query-string param with validation (type, range, description).
<code>Depends(get_rag_chain)</code>	FastAPI injects the singleton <code>RAGChain</code> . The function is cached (same instance per process).
<code>Depends(rate_limiter)</code>	Runs before the endpoint body; raises 429 if the client is over its quota.
<code>request.app.state.rate_bucket</code>	A convenient place to store global objects (the in-memory bucket) that survive across requests.
<code>response_model=RAGResponse</code>	Guarantees the JSON shape and auto-generates OpenAPI docs.
<code>router.on_event("startup")</code>	Hook that runs when the app boots - perfect for initializing stateful resources.

6? Main Application Entrypoint

`app/main.py` pulls the router into a **FastAPI** instance and adds some optional middleware for logging and CORS (helpful during local testing).

```
# app/main.py
import structlog
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from .api.rag import router as rag_router
from .config import settings

# -----
# Structured logging with structlog - easy to swap for Loguru, etc.
# -----
structlog.configure(
    processors=[
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.JSONRenderer(),
    ],
    logger_factory=structlog.StdlibLoggerFactory(),
    wrapper_class=structlog.make_filtering_bound_logger(settings.log_level),
)

logger = structlog.get_logger(__name__)

def create_app() -> FastAPI:
    app = FastAPI(
        title="LangChain-FastAPI RAG Service",
        version="0.1.0",
        description=(
            "A minimal but production-ready example that wires a LangChain "
            "Retrieval-Augmented Generation pipeline into a FastAPI endpoint. "
            "It showcases async design, rate-limiting, and per-request LLM "
            "parameter overrides."
        ),
    )

    # -----
    # CORS - allow any origin for demo; tighten in prod.
    # -----
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"], # replace with specific domains in production
```

... (continued on next page)

```
        allow_credentials=True,
        allow_methods=["GET", "POST"],
        allow_headers=["*"],
    )

    # -----
    # Register the RAG router under the /api prefix
    # -----
    app.include_router(rag_router, prefix="/api")

    # -----
    # Simple health-check endpoint
    # -----
    @app.get("/health", tags=["Health"])
    async def health() -> dict[str, str]:
        return {"status": "ok"}

    # -----
    # Log each request (very lightweight)
    # -----
    @app.middleware("http")
    async def log_requests(request, call_next):
        logger.info("request_start", method=request.method, path=request.url.path)
        response = await call_next(request)
        logger.info(
            "request_end",
            method=request.method,
            path=request.url.path,
            status_code=response.status_code,
        )
        return response

    return app

app = create_app()
```

Run locally with **Uvicorn**:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Open <http://localhost:8000/docs> - the automatically generated **Swagger UI** will show

the `/api/rag` endpoint with all query parameters nicely documented.

7.2 Unit Tests - Verifying the API Layer

FastAPI ships with a **TestClient** that runs the ASGI app in-process, making unit tests fast and deterministic. We will test three aspects:

1. **Parameter validation** - ensure out-of-range values raise a 422.
2. **Rate limiting** - after exceeding the quota, a 429 is returned.
3. **Successful response** - the JSON payload matches the `RAGResponse` schema.

Create `tests/unit/test_rag_route.py`:

```

# tests/unit/test_rag_route.py
import asyncio
import pytest
from httpx import AsyncClient

from app.main import app
from app.config import settings

# -----
# Helper fixture that gives us a clean app instance per test
# -----
@pytest.fixture
def anyio_backend():
    return "asyncio"

@pytest.fixture
async def async_client():
    async with AsyncClient(app=app, base_url="http://testserver") as client:
        yield client

# -----
# 1Ⓜ Parameter validation
# -----
@pytest.mark.anyio
async def test_invalid_temperature(async_client: AsyncClient):
    response = await async_client.get(
        "/api/rag",
        params={"query": "What is Python?", "temperature": -0.5},
    )
    assert response.status_code == 422
    assert "temperature" in response.text

# -----
# 2Ⓜ Rate limiting - we set a tiny limit for the test
# -----
@pytest.mark.anyio
async def test_rate_limit(async_client: AsyncClient, monkeypatch):

```

... (continued on next page)

```

# Temporarily shrink the limit to 2 requests per minute for speed.
monkeypatch.setattr(settings, "rate_limit_per_minute", 2)

# First two calls should succeed.
for _ in range(2):
    resp = await async_client.get(
        "/api/rag",
        params={"query": "What is FastAPI?"},
    )
    assert resp.status_code == 200

# Third call must be rejected.
resp = await async_client.get(
    "/api/rag",
    params={"query": "What is LangChain?"},
)
assert resp.status_code == 429
assert "Rate limit exceeded" in resp.text

# -----
# 3 Happy path - check response shape
# -----
@pytest.mark.anyio
async def test_successful_rag(async_client: AsyncClient):
    resp = await async_client.get(
        "/api/rag",
        params={"query": "What is a vector store?"},
    )
    assert resp.status_code == 200
    data = resp.json()
    assert "answer" in data
    assert isinstance(data["answer"], str)
    assert "sources" in data
    assert isinstance(data["sources"], list)
    # At least one source document should be returned for this tiny corpus.
    assert len(data["sources"]) >= 1
    assert "page_content" in data["sources"][0]

```

Run the suite:

```
pytest -q
```

All tests should pass quickly because the entire stack runs in the same process, and the vector store is tiny.

8? Integration Test - End-to-End with Real LLM (Optional)

For a **full-stack sanity check** you may want to spin up the service in a Docker container and run a real request against the OpenAI API. This is optional for learners who lack API keys; the unit tests already guarantee correctness of request handling.

Create `tests/integration/test_full_flow.py`:

```
# tests/integration/test_full_flow.py
import pytest
import subprocess
import time
import httpx

# -----
# Spin up a Docker container with the app. Use a lightweight image.
# -----
@pytest.fixture(scope="module")
def docker_container():
    # Build the image (Dockerfile is simple - see sidebar below)
    subprocess.run(["docker", "build", "-t", "rag-service:dev", "."], check=True)

    # Run container exposing port 8000
    container = subprocess.Popen(
        ["docker", "run", "--rm", "-p", "8000:8000", "rag-service:dev"],
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
    )

    # Wait for the service to become healthy (simple poll)
    for _ in range(30):
        try:
            r = httpx.get("http://localhost:8000/health")
            if r.status_code == 200:
                break
        except httpx.ConnectError:
            pass
        time.sleep(1)
    else:
        container.terminate()
        pytest.fail("Service failed to start within timeout.")

    yield # test body runs here

    # Tear down
    container.terminate()
    container.wait()
```

... (continued on next page)

```
@pytest.mark.integration
def test_real_llm_response(docker_container):
    # This test will hit the real OpenAI endpoint - ensure the env var is set.
    resp = httpx.get(
        "http://localhost:8000/api/rag",
        params={"query": "Explain RAG in one sentence."},
        timeout=30,
    )
    assert resp.status_code == 200
    payload = resp.json()
    assert "answer" in payload
    # The answer should be a non-empty string.
    assert len(payload["answer"].strip()) > 0
```

Dockerfile (place at repository root):

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/
COPY .env . # optional - you can mount env vars at runtime instead.

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Run integration tests (requires Docker and a valid `OPENAI_API_KEY` in `.env`):

```
pytest -m integration -s
```

If everything is wired correctly you'll see a concise answer like:

```
"Retrieval-Augmented Generation (RAG) combines a knowledge base with a language model to produce grounded,
up-to-date responses."
```

9? Production-Ready Touches

The example we built is intentionally **minimal** so that learners can focus on the core concepts. Below is a checklist of additional production concerns you can adopt later:

Concern	Quick Implementation
HTTPS	Deploy behind a reverse proxy (NGINX, Traefik) with TLS termination.
Authentication	Add a FastAPI <code>Depends</code> that validates an API key or JWT token.
Distributed Rate Limiting	Switch the in-memory bucket to <code>RedisTokenBucket</code> (set <code>REDIS_URL</code>).
Observability	Export <code>prometheus_client</code> metrics (<code>request_latency_seconds</code> , <code>request_counter</code>).
Graceful Shutdown	Register lifespan events to close Redis connections and the LLM client.
Scaling	Package as a Docker image and run multiple replicas behind a load balancer; the Redis bucket ensures global quota enforcement.
Cold-Start Optimisation	Pre-load the vector store at startup (already done) and keep the LLM client warm.
CI/CD	Add GitHub Actions workflow that runs unit & integration tests and pushes the Docker image to a registry on merge.

10? Recap - From Idea to Deployable Service

Step	What you coded	Why it matters
------	----------------	----------------

Step	What you coded	Why it matters
Settings	<code>app/config/settings.py</code>	Centralizes configuration, enables env-based overrides.
RAG chain	<code>app/chains/rag_chain.py</code>	Encapsulates LangChain logic, isolates LLM/vector store details.
Rate limiter	<code>app/utils/rate_limiter.py</code>	Protects your quota and prevents DoS.
FastAPI route	<code>app/api/rag.py</code>	Exposes a clean HTTP contract, validates inputs, enforces limits.
App entrypoint	<code>app/main.py</code>	Sets up logging, middleware, health-check, and mounts the router.
Tests	<code>tests/unit/*</code> , <code>tests/integration/*</code>	Guarantees correctness, encourages safe refactoring.
Docker	<code>Dockerfile</code> + integration test	Turns the code into a portable artifact ready for cloud.

You now have a **full-stack scaffold** that can be extended in countless ways: add a **chat endpoint**, plug in **retrieval from Pinecone or Weaviate**, enable **streaming responses**, or integrate **LLM-based function calling**. The modular design ensures that each layer (settings, chain, API, tests) can evolve independently-a hallmark of robust AI engineering.

Key Takeaways

1. **LangChain and FastAPI complement each other** - LangChain handles the **knowledge-centric** workflow (retrieval, prompting, LLM orchestration), while FastAPI provides the **service-centric** concerns (routing, validation, concurrency).
2. **Async-first design is essential** for LLM APIs because most latency is I/O-

bound (network calls to OpenAI, vector store queries). Use `async def` endpoints and `async HTTP` clients (`httpx.AsyncClient`).

3. **Dependency injection** lets you create per-request objects (e.g., a fresh `RAGChain`) without sacrificing performance; FastAPI caches singleton dependencies automatically.

4. **Rate limiting can be simple or distributed** - the chapter shows an in-memory token bucket for local development and a Redis-backed bucket for production.

5. **Expose model knobs via query strings** and validate them with Pydantic. This gives API consumers fine-grained control while keeping defaults sane.

6. **Testing the API layer is straightforward** with FastAPI's `TestClient` (unit tests) and `httpx` against a running Docker container (integration).

7. **Observability hooks** (structured logging, Prometheus metrics) can be added without touching core business logic thanks to FastAPI middleware.

8. **The final scaffold is production-ready** - it can be containerized, horizontally scaled, and secured with minimal additional code.

By mastering the patterns in this chapter you've moved from "building a proof-of-concept RAG notebook" to "delivering a reliable, testable, and scalable LLM service". The next module will show how to **orchestrate multiple chains** (e.g., a classification front-end that routes requests to different RAG pipelines) and how to **deploy the service on a serverless platform** such as AWS Lambda or Google Cloud Run.

Happy coding, and may your prompts be ever relevant!

Security, Monitoring, and Observability in Production

*"A model is only as useful as the API that delivers its answers." -
FastAPI + LLM = production-ready intelligence*

In the previous modules you learned how to **design** a FastAPI service that wraps a

large language model (LLM) and **integrate** it with LangChain to build retrieval-augmented generation (RAG) pipelines. You now have a working prototype that can answer questions, fetch documents, and even stream responses.

The next logical step is to **harden** that prototype for the real world. Production-grade services must protect secrets, verify who can call them, and give you visibility into what's happening inside the system. This chapter walks you through the three pillars that make an LLM service reliable and trustworthy:

Pillar	What it solves	Typical tools
Security	Prevent unauthorized access, protect API keys & model credentials, enforce least-privilege policies	HashiCorp Vault, environment-based secrets, JWT, role-based access control (RBAC)
Monitoring	Track health, performance, and usage trends over time	Prometheus, Grafana, Alertmanager
Observability	Understand <i>why</i> a request behaved a certain way (traces, logs, metrics)	OpenTelemetry (OTel), structured logging, distributed tracing back-ends (Jaeger/Tempo)

By the end of this chapter you will be able to:

- Store LLM API keys securely using Vault or environment variables.
- Protect every endpoint with JWT-based authentication and fine-grained RBAC.
- Instrument FastAPI + LangChain calls with OpenTelemetry for end-to-end tracing.
- Export Prometheus-compatible metrics for latency, request volume, and error rates.
- Configure alerts that fire on latency spikes, high error percentages, or secret-rotation failures.

The code examples are deliberately **self-contained** so you can copy-paste them into a fresh project and run them locally with Docker Compose. Feel free to adapt the snippets to your own cloud provider or orchestration platform later on.

Table of Contents

1. [Introduction](#introduction)
 2. [Core Concepts](#core-concepts)
 - [Secret Management](#secret-management)
 - [Authentication & Authorization](#authentication--authorization)
 - [Logging, Tracing, and Metrics](#logging-tracing-and-metrics)
 - [Alerting & Incident Response](#alerting--incident-response)
 3. [Practical Application](#practical-application)
 - [Project Layout](#project-layout)
 - [Step-1: Secure Secrets with Vault & .env](#step-1-secure-secrets-with-vault--env)
 - [Step-2: Add JWT Auth & RBAC Middleware](#step-2-add-jwt-auth--rbac-middleware)
 - [Step-3: Instrument FastAPI & LangChain with OpenTelemetry](#step-3-instrument-fastapi--langchain-with-opentelemetry)
 - [Step-4: Expose Prometheus Metrics & Grafana Dashboards](#step-4-expose-prometheus-metrics--grafana-dashboards)
 - [Step-5: Wire Up Alertmanager for Latency & Error Alerts](#step-5-wire-up-alertmanager-for-latency--error-alerts)
 - [Running the Whole Stack with Docker-Compose](#running-the-whole-stack-with-docker-compose)
 4. [Key Takeaways](#key-takeaways)
-

Introduction

When you ship an LLM-backed API to production you quickly discover that **raw inference speed** is only one dimension of reliability. Real-world traffic brings:

- **Credential leakage risks** - API keys for OpenAI, Anthropic, or proprietary

models are valuable assets. A single leak can lead to runaway costs or data exfiltration.

- **Unauthorized usage** - Anyone with the endpoint URL can start sending prompts unless you enforce identity checks.
- **Unpredictable latency** - LLMs are often hosted in external clouds; network hiccups or model throttling can cause response times to vary dramatically.
- **Silent failures** - Errors deep inside LangChain (e.g., a failed vector store lookup) may surface as generic "500 Internal Server Error" without context.

By **embedding security, monitoring, and observability from day one**, you avoid costly retro-fits later and give your ops team the tooling they need to keep the service healthy.

The rest of this chapter builds on the FastAPI + LangChain service you built in Modules 6-7. We'll keep the same core logic (a `/chat` endpoint that streams LLM responses) but wrap it in production-grade scaffolding.

Core Concepts

Secret Management

Why it matters	Typical pitfalls	Recommended approach
Protect API keys (OpenAI, Cohere, etc.)	Hard-coding keys in source, committing them to Git, or exposing them via logs	Use HashiCorp Vault (or AWS Secrets Manager / GCP Secret Manager) and fall back to environment variables for simple local dev
Enable rotation without downtime	Manual copy-paste updates, stale keys lingering in memory	Load secrets at startup and re-load on SIGHUP or a timed watch; Vault's lease renewal automates rotation
Least-privilege access	Giving the app full admin rights to Vault	Create a Vault policy that only allows read on the path

Why it matters	Typical pitfalls	Recommended approach
		containing the LLM key

Quick Vault Primer

```
# Spin up a dev Vault server (no TLS, dev mode)
docker run -d --name vault -p 8200:8200 \
  -e 'VAULT_DEV_ROOT_TOKEN_ID=root' \
  vault:1.15.3 server -dev
```

- Store a secret:

```
export VAULT_ADDR='http://127.0.0.1:8200'
vault kv put secret/llm openai_key="sk-xxxxxxxxxxxxxxxxxxxx"
```

- Retrieve it from Python (using `hvac` client):

```
import hvac, os

def get_secret_from_vault(path: str, key: str) -> str:
    client = hvac.Client(url=os.getenv("VAULT_ADDR", "http://127.0.0.1:8200"),
                        token=os.getenv("VAULT_TOKEN", "root"))
    secret = client.secrets.kv.v2.read_secret_version(path=path)
    return secret["data"]["data"][key]
```

If you don't have Vault in your environment, simply create a `.env` file:

```
# .env (local dev)
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxx
```

FastAPI can load this file via `python-dotenv` or `pydantic.BaseSettings`.

Authentication & Authorization

JWT (JSON Web Tokens) are the de-facto standard for stateless API authentication. They let you:

- Verify a request's identity without a round-trip to a database.
- Embed **claims** (e.g., `role`, `exp`, `sub`) that downstream services can read for **RBAC** decisions.

JWT Flow Overview

1. **Login** - Client sends credentials (`username`, `password`).
2. **Token Issuance** - Server validates credentials, signs a JWT with a secret/private key, and returns it.
3. **Protected Call** - Client includes `Authorization: Bearer <jwt>` header.
4. **Verification** - FastAPI middleware decodes the token, checks signature & expiry, and injects a `User` object into the request context.

Role-Based Access Control (RBAC)

Define a simple mapping:

Role	Permissions
admin	Full access - can call <code>/admin/*</code> , manage secrets, view metrics.
developer	Can invoke <code>/chat</code> and <code>/rag</code> , but not admin endpoints.
viewer	Read-only access to <code>/metrics</code> and health checks.

FastAPI's **dependency injection** makes RBAC implementation clean:

```

from fastapi import Depends, HTTPException, Security, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from jose import JWTError, jwt
from typing import List

# Settings (in production store secret in Vault)
JWT_SECRET = os.getenv("JWT_SECRET", "super-secret-key")
ALGORITHM = "HS256"

security = HTTPBearer()

class User(BaseModel):
    username: str
    role: str
    scopes: List[str] = []

def decode_jwt(token: str) -> User:
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[ALGORITHM])
        return User(username=payload["sub"], role=payload["role"], scopes=payload.get("scopes", []))
    except JWTError as exc:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                             detail="Invalid token") from exc

def get_current_user(credentials: HTTPAuthorizationCredentials = Security(security)):
    return decode_jwt(credentials.credentials)

def require_role(required_role: str):
    def role_checker(user: User = Depends(get_current_user)):
        if user.role != required_role:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                detail=f"Requires role {required_role}")
        return user
    return role_checker

```

Use the `require_role` dependency on any endpoint:

```

@app.post("/admin/reload-secrets")
def reload_secrets(user: User = Depends(require_role("admin"))):
    # Trigger secret reload logic
    return {"msg": "Secrets reloaded"}

```

Logging, Tracing, and Metrics

Structured Logging

Plain `print` statements are noisy and hard to parse. Adopt **structured JSON logs** so that log aggregation platforms (e.g., Loki, Elasticsearch) can index fields:

```
import structlog

structlog.configure(
    processors=[
        structlog.processors.JSONRenderer()
    ]
)
log = structlog.get_logger()
```

Now you can log request IDs, user names, and latency:

```
log.info("incoming_request", endpoint="/chat", user=user.username, request_id=req_id)
```

OpenTelemetry (OTel)

OpenTelemetry provides a **vendor-agnostic** API for distributed tracing and metrics. In a FastAPI + LangChain stack you typically instrument:

Layer	What to capture
FastAPI	HTTP request/response spans, status codes, latency
LangChain	Calls to LLM providers, vector store lookups, chain execution
Database / Cache	Optional - e.g., Redis, PostgreSQL queries

Installation


```
pip install opentelemetry-sdk opentelemetry-instrumentation-fastapi \
           opentelemetry-instrumentation-requests \
           opentelemetry-exporter-otlp
```

FastAPI Instrumentation (auto-instrument)

```
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor

FastAPIInstrumentor().instrument_app(app)
RequestsInstrumentor().instrument()
```

Custom LangChain Span

LangChain does not ship with OTel instrumentation out-of-the-box, but you can wrap calls yourself:

```
from opentelemetry import trace
from opentelemetry.trace import SpanKind

tracer = trace.get_tracer(__name__)

def call_llm(prompt: str) -> str:
    with tracer.start_as_current_span("llm.inference", kind=SpanKind.CLIENT) as span:
        # Optionally add attributes
        span.set_attribute("model.provider", "openai")
        span.set_attribute("model.name", "gpt-4")
        response = openai.ChatCompletion.create(
            model="gpt-4",
            messages=[{"role": "user", "content": prompt}]
        )
        answer = response.choices[0].message["content"]
        span.set_attribute("response.tokens", response.usage.total_tokens)
    return answer
```

Exporters

For a local dev environment we'll use the **OTLP HTTP** exporter pointing at Jaeger (via the `jaegertracing/all-in-one` Docker image). Production deployments often ship traces to **Tempo**, **Datadog**, or **AWS X-Ray**.

```
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor

provider = TracerProvider()
processor = BatchSpanProcessor(OTLPSpanExporter(endpoint="http://jaeger:4318/v1/traces"))
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)
```

Prometheus Metrics

Prometheus scrapes HTTP endpoints exposing plain-text metrics. The `prometheus_fastapi_instrumentator` library automatically creates standard metrics (request count, latency, etc.) and lets you add **custom counters/gauges**.

```
pip install prometheus_fastapi_instrumentator
```

```
from prometheus_fastapi_instrumentator import Instrumentator

instrumentator = Instrumentator(
    should_group_status_codes=False,
    should_ignore_untemplated=False,
    should_respect_env_var=False,
    excluded_handlers=["/metrics", "/health"]
)

instrumentator.instrument(app).expose(app, endpoint="/metrics")
```

Custom LLM Metrics

```
from prometheus_client import Counter, Histogram

LLM_REQUESTS = Counter(
    "llm_requests_total",
    "Number of LLM inference requests",
    ["model_name", "status"]
)

LLM_LATENCY = Histogram(
    "llm_request_latency_seconds",
    "LLM inference latency",
    ["model_name"]
)

def call_llm(prompt: str) -> str:
    start = time.time()
    try:
        answer = ... # same as before
        LLM_REQUESTS.labels(model_name="gpt-4", status="success").inc()
        return answer
    except Exception:
        LLM_REQUESTS.labels(model_name="gpt-4", status="error").inc()
        raise
    finally:
        LLM_LATENCY.labels(model_name="gpt-4").observe(time.time() - start)
```

All metrics are now available at <http://localhost:8000/metrics> for Prometheus to scrape.

Alerting & Incident Response

Once metrics are flowing into Prometheus, you can fire alerts with **Alertmanager**. Typical production alerts for an LLM API include:

Alert	Condition	Action
LLMHighLatency	95th-percentile latency > 2 s for 5 min	Page on-call engineer, investigate upstream model throttling
LLMErrorRate	Error rate > 5 % over 1 min	Auto-scale or circuit-break;

Alert	Condition	Action
		notify ops
VaultSecretExpiry	Secret lease expiration < 1h	Rotate secret, trigger CI pipeline
JWTKeyRotation	JWT signing key older than 30 days	Rotate key, re-issue tokens

Alertmanager Configuration (simplified)

```
# alertmanager.yml
global:
  resolve_timeout: 5m
route:
  receiver: slack-notifications
receivers:
- name: slack-notifications
  slack_configs:
    - channel: "#llm-alerts"
      api_url: "${SLACK_WEBHOOK}"
      send_resolved: true
```

Prometheus Rule Example

```
# rules.yml
groups:
  - name: llm.rules
    rules:
      - alert: LLMHighLatency
        expr: histogram_quantile(0.95, sum(rate(llm_request_latency_seconds_bucket[5m])) by (le)) > 2
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "LLM latency > 2s"
          description: "95th percentile latency over the last 5 minutes is {{ $value }} seconds."

      - alert: LLMErrorRate
        expr: (sum(rate(llm_requests_total{status="error"}[1m])) / sum(rate(llm_requests_total[1m]))) >
0.05
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "LLM error rate > 5%"
          description: "Error rate over the last minute is {{ $value | humanizePercentage }}."
```

When an alert fires, the **Alertmanager** sends a message to Slack (or PagerDuty, email, etc.) and can also trigger **silences** or **inhibition rules** to avoid alert storms during planned maintenance.

Practical Application

Below you'll find a **complete, runnable example** that ties together all the concepts introduced so far. The project structure mirrors what you built in Modules 6-7, plus a few new files for security, observability, and infra.

Project Layout

```
llm-prod/
├── app/
│   ├── __init__.py
│   ├── main.py                # FastAPI entrypoint
│   └── api/
│       ├── __init__.py
│       ├── chat.py            # /chat endpoint (LangChain chain)
│       └── admin.py           # /admin endpoints (secret reload)
│   └── core/
│       ├── __init__.py
│       ├── security.py        # JWT utils, RBAC deps
│       ├── secrets.py         # Vault/.env loader
│       ├── observability.py   # OTel & Prometheus setup
│       └── models/
│           └── rag_chain.py    # LangChain RAG pipeline (unchanged)
├── Dockerfile
├── requirements.txt
├── .env.example
└── docker-compose.yml
```

Tip: Keep secret-related code under `core/` and keep your domain logic (`rag_chain.py`) pure and testable.

Step-1: Secure Secrets with Vault & .env

`core/secrets.py`

```

import os
from typing import Optional
import hvac
from pydantic import BaseSettings, Field

class Settings(BaseSettings):
    # Fallback to .env for local dev
    OPENAI_API_KEY: Optional[str] = Field(default=None, env="OPENAI_API_KEY")
    VAULT_ADDR: Optional[str] = Field(default=None, env="VAULT_ADDR")
    VAULT_TOKEN: Optional[str] = Field(default=None, env="VAULT_TOKEN")
    JWT_SECRET: Optional[str] = Field(default=None, env="JWT_SECRET")

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"

settings = Settings()

def load_openai_key() -> str:
    """Load the OpenAI key from Vault if configured, otherwise from env."""
    if settings.VAULT_ADDR and settings.VAULT_TOKEN:
        client = hvac.Client(url=settings.VAULT_ADDR, token=settings.VAULT_TOKEN)
        secret = client.secrets.kv.v2.read_secret_version(path="secret/llm")
        return secret["data"]["data"]["openai_key"]
    elif settings.OPENAI_API_KEY:
        return settings.OPENAI_API_KEY
    else:
        raise RuntimeError("OpenAI API key not configured")

```

When the container boots, `load_openai_key()` fetches the secret once and stores it in a module-level variable. You can expose a `/admin/reload-secrets` endpoint that re-calls this function on demand.

`.env.example`

```

# .env.example - copy to .env for local dev
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxx
JWT_SECRET=super-secret-key
# Vault configuration (optional)
VAULT_ADDR=http://vault:8200
VAULT_TOKEN=root

```

Step-2: Add JWT Auth & RBAC Middleware

`core/security.py`

```
import os
from datetime import datetime, timedelta
from typing import List

from fastapi import Depends, HTTPException, Security, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from jose import JWTError, jwt
from pydantic import BaseModel

from .secrets import settings

# JWT config - secret comes from Vault/.env
JWT_SECRET = settings.JWT_SECRET or "fallback-secret"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

security = HTTPBearer()

class TokenData(BaseModel):
    sub: str
    role: str
    exp: datetime

class User(BaseModel):
    username: str
    role: str

def create_access_token(*, username: str, role: str, expires_delta: timedelta = None) -> str:
    to_encode = {"sub": username, "role": role}
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, JWT_SECRET, algorithm=ALGORITHM)

def decode_token(token: str) -> User:
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[ALGORITHM])
        return User(username=payload["sub"], role=payload["role"])
    except JWTError as exc:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                             detail="Could not validate credentials") from exc
```

... (continued on next page)


```
def get_current_user(credentials: HTTPAuthorizationCredentials = Security(security)):
    return decode_token(credentials.credentials)

def require_role(required_role: str):
    def role_checker(user: User = Depends(get_current_user)):
        if user.role != required_role:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                detail=f"Requires role {required_role}")
        return user
    return role_checker
```

Login endpoint (in `api/admin.py`)

```
from fastapi import APIRouter, HTTPException, status
from pydantic import BaseModel
from ..core.security import create_access_token

router = APIRouter(prefix="/admin", tags=["admin"])

# In a real system you'd check a DB or LDAP. For demo we hard-code.
USERS_DB = {
    "alice": {"password": "wonderland", "role": "admin"},
    "bob": {"password": "builder", "role": "developer"},
    "eve": {"password": "eavesdrop", "role": "viewer"},
}

class LoginRequest(BaseModel):
    username: str
    password: str

@router.post("/login", response_model=dict)
def login(req: LoginRequest):
    user = USERS_DB.get(req.username)
    if not user or user["password"] != req.password:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                            detail="Invalid credentials")
    token = create_access_token(username=req.username, role=user["role"])
    return {"access_token": token, "token_type": "bearer"}
```

Now any endpoint can protect itself:

```
@router.post("/reload-secrets")
def reload_secrets(user: User = Depends(require_role("admin"))):
    # Trigger secret reload logic (e.g., call a function in core.secrets)
    from ..core.secrets import load_openai_key
    new_key = load_openai_key()
    # In a real app you would replace the client's auth header
    return {"msg": "Secrets reloaded", "new_key_fingerprint": new_key[:8]}
```

Step-3: Instrument FastAPI & LangChain with OpenTelemetry

`core/observability.py`

```

import os
from opentelemetry import trace
from opentelemetry.sdk.resources import SERVICE_NAME, Resource
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor
from prometheus_fastapi_instrumentator import Instrumentator

def init_tracing(app):
    """Configure OTEL TracerProvider and instrument FastAPI."""
    # Exporter to Jaeger (OTLP HTTP endpoint)
    jaeger_endpoint = os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT", "http://jaeger:4318/v1/traces")
    exporter = OTLPSpanExporter(endpoint=jaeger_endpoint)

    resource = Resource(attributes={
        SERVICE_NAME: "llm-fastapi-service"
    })
    provider = TracerProvider(resource=resource)
    processor = BatchSpanProcessor(exporter)
    provider.add_span_processor(processor)
    trace.set_tracer_provider(provider)

    # Auto-instrument FastAPI & outgoing HTTP (LangChain -> OpenAI)
    FastAPIInstrumentor().instrument_app(app)
    RequestsInstrumentor().instrument()

def init_metrics(app):
    """Add Prometheus metrics instrumentation and expose /metrics."""
    instrumentor = Instrumentator(
        should_group_status_codes=False,
        should_ignore_untemplated=False,
        should_respect_env_var=False,
        excluded_handlers=["/metrics", "/health"]
    )
    instrumentor.instrument(app).expose(app, endpoint="/metrics")

```

Wire it up in `main.py`

```
from fastapi import FastAPI
from .api import chat, admin
from .core.observability import init_tracing, init_metrics
from .core.secrets import load_openai_key

app = FastAPI(title="LLM Production Service", version="0.1.0")

# Register routers
app.include_router(chat.router)
app.include_router(admin.router)

# Initialize observability (run after FastAPI app creation)
init_tracing(app)
init_metrics(app)

# Load the LLM key at startup
@app.on_event("startup")
def startup_event():
    openai_key = load_openai_key()
    # Store globally (or in a dependency) for LangChain usage
    os.environ["OPENAI_API_KEY"] = openai_key
```

Wrap LangChain calls with a custom trace (in ``api/chat.py``)

```

from fastapi import APIRouter, Depends, HTTPException, Request
from fastapi.responses import StreamingResponse
from opentelemetry import trace
from ..core.security import require_role
from ..models.rag_chain import get_rag_chain

router = APIRouter(prefix="/chat", tags=["chat"])

tracer = trace.get_tracer(__name__)

@router.post("/stream")
async def chat_stream(prompt: str, request: Request, user = Depends(require_role("developer"))):
    """
    Stream LLM responses back to the client.
    The chain is instrumented with an OTel span.
    """
    chain = get_rag_chain()
    async def generator():
        with tracer.start_as_current_span("rag.chat") as span:
            span.set_attribute("user", user.username)
            span.set_attribute("prompt.length", len(prompt))
            # LangChain's `astream` yields partial tokens
            async for token in chain.astream({"question": prompt}):
                yield token["answer"]
    return StreamingResponse(generator(), media_type="text/event-stream")

```

Now every request to `/chat/stream` produces:

- **FastAPI HTTP span** (status, latency)
- **Custom `rag.chat` span** (user, prompt length)
- **Downstream HTTP spans** for each call to OpenAI (automatically captured by `RequestsInstrumentor`)

All spans flow to Jaeger, where you can visualise the end-to-end trace.

Step-4: Expose Prometheus Metrics & Grafana Dashboards

Prometheus Scrape Config (``prometheus.yml``)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "fastapi"
    static_configs:
      - targets: ["fastapi:8000"]    # FastAPI container name + port
```

Grafana Dashboard (JSON snippet)

You can import this JSON via Grafana UI → **Create** → **Import**.

```

{
  "dashboard": {
    "id": null,
    "title": "LLM Service Overview",
    "panels": [
      {
        "type": "graph",
        "title": "HTTP Request Latency (ms)",
        "targets": [
          {
            "expr": "histogram_quantile(0.95, sum(rate(http_server_request_duration_seconds_bucket[5m]))
by (le)) * 1000",
            "legendFormat": "95th percentile"
          }
        ],
        "datasource": "Prometheus"
      },
      {
        "type": "graph",
        "title": "LLM Inference Latency",
        "targets": [
          {
            "expr": "histogram_quantile(0.95, sum(rate(llm_request_latency_seconds_bucket[5m])) by (le))",
            "legendFormat": "LLM 95th pct"
          }
        ],
        "datasource": "Prometheus"
      },
      {
        "type": "stat",
        "title": "Error Rate",
        "targets": [
          {
            "expr": "sum(rate(http_server_requests_total{status=~\"5..\"}[1m])) /
sum(rate(http_server_requests_total[1m]))",
            "legendFormat": "5xx error rate"
          }
        ],
        "datasource": "Prometheus"
      },
      {
        "type": "table",

```

... (continued on next page)

```
"title": "Top Users by Request Count",
"targets": [
  {
    "expr": "sum by (user) (rate(http_server_requests_total[5m]))",
    "legendFormat": "{{user}}"
  }
],
"datasource": "Prometheus"
}
],
"schemaVersion": 30,
"version": 1
},
"overwrite": true
}
```

Running Grafana and Prometheus


```
# docker-compose.yml (excerpt)
services:
  fastapi:
    build: .
    ports:
      - "8000:8000"
    environment:
      - VAULT_ADDR=http://vault:8200
      - VAULT_TOKEN=root
    depends_on:
      - vault
      - jaeger

  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    depends_on:
      - prometheus
    volumes:
      - grafana-data:/var/lib/grafana

volumes:
  grafana-data:
```

Visit `http://localhost:3000` (admin/admin) → **Add data source** → **Prometheus**, URL `http://prometheus:9090`. Then import the dashboard JSON.

Step-5: Wire Up Alertmanager for Latency & Error Alerts

Alertmanager (``alertmanager.yml``)

```
global:
  resolve_timeout: 5m
  slack_api_url: ${SLACK_WEBHOOK} # set via env var in compose

route:
  receiver: slack-notifications
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 12h

receivers:
- name: slack-notifications
  slack_configs:
    - channel: "#llm-alerts"
      send_resolved: true
```

Prometheus with Alertmanager (`prometheus.yml` addition)

```
alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]
  rule_files:
    - "/etc/prometheus/rules.yml"
```

Compose addition

```
alertmanager:
  image: prom/alertmanager:latest
  ports:
    - "9093:9093"
  volumes:
    - ./alertmanager.yml:/etc/alertmanager/config.yml
  environment:
    - SLACK_WEBHOOK=https://hooks.slack.com/services/XXX/YYY/ZZZ
```

Now, when latency or error thresholds breach, a Slack message like:

```
[ALERT] LLMHighLatency
Severity: critical
Summary: LLM latency > 2s
Description: 95th percentile latency over the last 5 minutes is 2.31 seconds.
```

will appear in `#llm-alerts`. You can also configure **inhibition rules** to suppress the "Error Rate" alert when a "Service Down" alert is firing.

Running the Whole Stack with Docker-Compose

Full ``docker-compose.yml``

```
version: "3.9"
services:
  vault:
    image: hashicorp/vault:1.15
    environment:
      VAULT_DEV_ROOT_TOKEN_ID: root
    ports:
      - "8200:8200"
    command: server -dev -dev-root-token-id=root

  jaeger:
    image: jaegertracing/all-in-one:1.53
    ports:
      - "16686:16686" # UI
      - "4318:4318" # OTLP HTTP

  fastapi:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    environment:
      - VAULT_ADDR=http://vault:8200
      - VAULT_TOKEN=root
      - OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4318/v1/traces
    depends_on:
      - vault
      - jaeger

  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - ./rules.yml:/etc/prometheus/rules.yml
    ports:
      - "9090:9090"
    depends_on:
      - fastapi
```

... (continued on next page)

```
alertmanager:
  image: prom/alertmanager:latest
  volumes:
    - ./alertmanager.yml:/etc/alertmanager/config.yml
  ports:
    - "9093:9093"

grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
  depends_on:
    - prometheus
  volumes:
    - grafana-data:/var/lib/grafana

volumes:
  grafana-data:
```

Build & launch

```
docker compose up --build -d
```

- Verify services:
- FastAPI → <http://localhost:8000/docs> (Swagger UI)
- Jaeger → <http://localhost:16686> (trace viewer)
- Prometheus → <http://localhost:9090> (metrics)
- Grafana → <http://localhost:3000> (dashboards)
- Alertmanager → <http://localhost:9093>

Test the flow

```
# 1🔗 Get a JWT
curl -X POST http://localhost:8000/admin/login -H "Content-Type: application/json" \
  -d '{"username":"bob","password":"builder"}'

# 2🔗 Call the chat endpoint with streaming
TOKEN=$(curl -s -X POST http://localhost:8000/admin/login \
  -H "Content-Type: application/json" \
  -d '{"username":"bob","password":"builder"}' | jq -r .access_token)

curl -N -H "Authorization: Bearer $TOKEN" \
  -X POST http://localhost:8000/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Explain the difference between RAG and fine-tuning"}'
```

You should see a stream of tokens in the terminal. In Jaeger you'll see a trace titled `rag.chat` with child spans for each outbound OpenAI request. Prometheus will start counting the request and latency metrics, which appear instantly on the Grafana dashboard.

Key Takeaways

- **Never hard-code secrets** - use Vault (or cloud secret managers) and fall back to `.env` for local development.
- **Stateless JWT authentication** gives you fast, scalable auth; combine it with **role-based dependencies** to enforce fine-grained access.
- **Observability is a three-leg stool:**

Deployment, Scaling, and CI/CD Pipelines

"A model is only as useful as the API that delivers its answers." - FastAPI🔗 +🔗 LLM production-ready intelligence

In the previous modules you built a **LangChain-powered FastAPI service** that can

answer user queries, retrieve relevant documents, and even run inference on a GPU-accelerated LLM. You learned how to **structure the code**, **add validation**, **instrument logging**, and **secure the endpoints**.

Now it's time to move from *"it works on my laptop"* to *"it works for millions of users, automatically, safely, and cheaply."* This chapter walks you through the complete **deployment pipeline**:

- 1. **Containerizing** the FastAPI-LangChain app with a **multi-stage Docker build** (so you ship only what you need).
- 2. **Deploying** that container to three major cloud options - **AWS Elastic Container Service (ECS)**, **Google Kubernetes Engine (GKE)**, and **Azure Container Apps** - and showing the common patterns that make the switch painless.
- 3. Configuring **horizontal autoscaling** that reacts to request latency, CPU/GPU utilisation, and queue length.
- 4. Building a **CI/CD pipeline** (GitHub Actions or GitLab CI) that automatically tests, builds, pushes, and rolls out new versions.

By the end of this chapter you will have a **repeatable, production-grade workflow** that you can adapt to any LLM-backed service.

Introduction

From Prototype to Production

When you ran the FastAPI server locally, you probably used `uvicorn main:app --reload`. That command is great for rapid iteration but **not** for production:

Prototype	Production
Run on host OS - Python interpreter, dev-dependencies, source code.	Run in an isolated container - immutable runtime, minimal OS, reproducible builds.

Prototype	Production
Single-process - one worker, no graceful shutdown guarantees.	Multiple workers + process manager - gunicorn+uvicorn workers, health checks, graceful termination.
Manual deployment - copy files, ssh into a VM.	Automated deployment - IaC (Terraform/Helm), CI/CD, zero-downtime rollouts.
Static capacity - one VM, one GPU, manual scaling.	Dynamic scaling - autoscale CPU/GPU pods, request-based load balancing.
Ad-hoc testing - run a single pytest locally.	Continuous testing - unit, integration, contract tests on every PR.

The **goal** of this chapter is to replace the left column with the right, **without losing any of the functionality you built earlier**. Everything you've written - LangChain chains, prompt templates, FastAPI routers, Pydantic models - stays exactly the same; we just wrap it in a more robust delivery system.

Why Containers?

- **Portability** - The same image runs on your laptop, on an AWS Fargate node, or on a Google Cloud GPU VM.
- **Isolation** - Dependencies (PyTorch, CUDA, LangChain) don't clash with the host OS.
- **Security** - You can run containers with a non-root user, enforce read-only file systems, and limit capabilities.
- **Size & Speed** - Multi-stage builds let you ship a ~150MB runtime image instead of a 1.5GB development image.

Why Cloud-Native?

LLM inference is **resource-hungry** (GPU, high memory) but also **bursty** (traffic spikes during product launches). Cloud-native platforms give you:

- **Elastic compute** - spin up GPU nodes only when needed.
- **Managed services** - no need to patch the underlying OS or manage a load

balancer.

- **Observability integrations** - CloudWatch, Stackdriver, Azure Monitor.

Why CI/CD?

A modern LLM product evolves quickly: new prompt engineering, model upgrades, bug fixes. CI/ CD guarantees that **every change is validated and deployed automatically**, reducing human error and mean-time-to-recovery (MTTR).

Core Concepts

Below we unpack the technical building blocks you'll use throughout the practical section.

1. Docker Multi-Stage Builds

A **multi-stage Dockerfile** separates the **build environment** (with compilers, pip, Git) from the **runtime environment** (only the minimal OS and Python wheels).

Typical stages:

Stage	Purpose	Example Base Image
builder	Install dependencies, compile wheels (e.g., torch, torchvision).	python:3.11-slim-bullseye
runtime	Copy only the compiled wheels and source code. Run as non-root.	python:3.11-slim-bullseye (or nvidia/cuda for GPU)

Key advantages:

- **Smaller final image** - eliminates gcc, make, git, etc.
- **Cache-friendly** - Docker caches each stage; only code changes trigger rebuild

of the final layer.

2. Container Registries

- **Docker Hub** - public, free tier, rate-limited.
- **Amazon ECR** - integrated with IAM, supports lifecycle policies.
- **Google Artifact Registry** - native to GCP, supports regional repositories.
- **Azure Container Registry (ACR)** - supports Azure AD authentication.

Your CI pipeline will **push the image** to one of these registries; the cloud service will **pull** from it during deployment.

3. Cloud Deployment Models

Platform	Compute Model	GPU Support	Managed Service	Typical Use-Case
AWS ECS (Fargate)	Serverless containers (no EC2 management)	GPU via Fargate Spot with GPU (still in preview) or ECS on EC2 with GPU instances	Fully managed task scheduler, service discovery	Light-to-medium traffic, quick rollouts
Google Kubernetes Engine (GKE)	Managed K8s cluster	NVIDIA GPU nodes (A100, T4, etc.)	Full K8s API, autoscaling groups, node pools	Complex micro-service topologies, advanced autoscaling
Azure Container Apps	Serverless containers with Dapr integration	GPU via Azure Container Apps for Kubernetes (preview)	Built-in scaling, revision-based deployments	Event-driven workloads, low-ops environment

All three platforms share a **common denominator**: they run **Docker images**, expose **HTTP endpoints** via a load balancer, and support **horizontal pod autoscaling (HPA)** based on custom metrics.

4. Autoscaling Signals

LLM inference workloads are best scaled on **GPU utilisation** and **request latency**:

Metric	Source	Why it matters
CPU %	CloudWatch, Metrics Server	If CPU spikes, the container may be bottlenecked before the GPU.
GPU %	NVIDIA DCGM, Cloud Monitoring	Directly reflects model inference load.
Request latency (p90)	Prometheus, CloudWatch Logs Insights	Guarantees SLA; high latency triggers more replicas.
Queue length (e.g., RabbitMQ, SQS)	Custom exporter	When you front-end the API with a message queue, scaling based on queue depth smooths bursts.

In Kubernetes you can combine **HorizontalPodAutoscaler** (HPA) with **Custom Metrics Adapter** (e.g., Prometheus Adapter) to scale on GPU utilisation. In ECS you can use **Application Auto Scaling** with **target tracking** based on CloudWatch custom metrics.

5. CI/CD Fundamentals

Concept	GitHub Actions	GitLab CI
Workflow definition	<code>.github/workflows/*.yaml</code>	<code>.gitlab-ci.yml</code>
Secrets management	Repository → Settings → Secrets	Settings → CCI → Variables
Job matrix	<code>strategy.matrix</code>	<code>parallel:</code>
Artifact storage	<code>actions/upload-artifact</code>	<code>artifacts:</code>
Environment protection	<code>environment: + reviewers</code>	<code>environment: + protected</code>

Typical pipeline stages for an LLM-backed FastAPI service:

1. Lint & Static Analysis - `ruff`, `mypy`, `bandit`.
2. Unit & Integration Tests - `pytest`, with a **GPU-enabled runner** for model-

related tests (or a mocked inference layer).

3. **Build Docker Image** - multi-stage Dockerfile, tag with `git SHA` and `semver`.

4. **Push to Registry** - login via OIDC, push image.

5. **Deploy** - use **Terraform**, **Helm**, or **AWS CDK** to apply changes; optionally run **smoke tests** against the new revision.

Practical Application

Below is a **complete, end-to-end example** that you can copy-paste, adjust, and run. We'll assume the following project layout (the same you built in earlier modules):

```
└─ app/
  │ └─ __init__.py
  │ └─ main.py                # FastAPI entrypoint
  │ └─ api/
  │   │ └─ routes.py
  │   └─ core/
  │     │ └─ chain.py          # LangChain pipeline
  │     │ └─ config.py
  │     └─ models/
  │       └─ request.py
└─ tests/
  └─ test_api.py
└─ Dockerfile
└─ pyproject.toml
└─ requirements.txt
```

Note: If you used Poetry, replace `requirements.txt` with `poetry.lock` and adjust the build steps accordingly.

1. Docker Multi-Stage Build

Create a file named `Dockerfile` at the project root:

```
# -----
# Stage 1 - Builder
# -----

FROM python:3.11-slim-bullseye AS builder

# Install system build tools & CUDA libs (if you need GPU)
# For CPU-only inference you can skip the CUDA packages.
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        build-essential gcc git curl && \
    rm -rf /var/lib/apt/lists/*

# Create a non-root user for later stages
ARG USERNAME=appuser
ARG UID=1000
ARG GID=1000
RUN groupadd -g ${GID} ${USERNAME} && \
    useradd -m -u ${UID} -g ${GID} -s /bin/bash ${USERNAME}

# Set workdir
WORKDIR /opt/app

# Install Python build dependencies first (caching layer)
COPY pyproject.toml poetry.lock* ./
# If you use pip + requirements.txt, replace the line below:
# RUN pip install --upgrade pip && pip install -r requirements.txt
RUN pip install --upgrade pip && \
    pip install poetry && \
    poetry export -f requirements.txt --output requirements.txt && \
    pip install -r requirements.txt --no-cache-dir

# Copy source code
COPY . .

# -----
# Stage 2 - Runtime (CPU)
# -----

FROM python:3.11-slim-bullseye AS runtime

# If you need GPU, switch base image to NVIDIA CUDA:
```

... (continued on next page)

```
# FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04 AS runtime

# Create non-root user (same UID/GID as builder for file permissions)
ARG USERNAME=appuser
ARG UID=1000
ARG GID=1000
RUN groupadd -g ${GID} ${USERNAME} && \
    useradd -m -u ${UID} -g ${GID} -s /bin/bash ${USERNAME}

# Install OS runtime deps (e.g., libgomp for PyTorch)
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        libgomp1 && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /opt/app

# Copy only the compiled wheels and source from builder
COPY --from=builder /opt/app /opt/app

# Switch to non-root user
USER ${USERNAME}

# Expose the HTTP port FastAPI will listen on
EXPOSE 8080

# Use Gunicorn with Uvicorn workers for production
ENV GUNICORN_CMD="gunicorn -k uvicorn.workers.UvicornWorker \
    -w 4 -b 0.0.0.0:8080 \
    --log-level info \
    --access-logfile - \
    --error-logfile - \
    main:app"

CMD ["/bin/sh", "-c", "$GUNICORN_CMD"]
```

Explanation of Key Parts

Section	Why it matters
Builder stage	Installs heavy build tools, compiles wheels (especially torch with CUDA).

Section	Why it matters
Non-root user	Security best practice; containers never run as root.
Copy only needed files	Keeps runtime image tiny (≈150 MB).
Gunicorn + Uvicorn workers	Provides multiple worker processes, graceful reloads, and better CPU utilisation.
<code>EXPOSE 8080</code>	Aligns with the FastAPI default we used in previous modules (uvicorn main:app --port 8080).

Tip: If you plan to use different GPU drivers across cloud providers, keep the base image generic (`nvidia/cuda:12.1.0-runtime-ubuntu22.04`) and let the provider attach the appropriate driver at runtime.

2. Pushing the Image to a Registry

Below is a **GitHub Actions** snippet that builds the image and pushes it to **Amazon ECR**. You can swap the steps for GCP Artifact Registry or Azure Container Registry with minor tweaks.

Create `.github/workflows/deploy.yml`:

```
name: CI / CD - Build & Deploy

on:
  push:
    branches: [ main ]
  pull_request:
    types: [ opened, synchronize, reopened ]

env:
  IMAGE_NAME: llm-fastapi-service
  REGISTRY: ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.${ secrets.AWS_REGION }.amazonaws.com
  FULL_IMAGE: ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha }

jobs:
  # -----
  # 1️⃣ Lint & Test
  # -----
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install poetry
          poetry install

      - name: Lint (ruff)
        run: poetry run ruff check .

      - name: Type check (mypy)
        run: poetry run mypy app
```

... (continued on next page)


```

- name: Run pytest (incl. GPU tests)
  # If you need a GPU runner, use self-hosted or a matrix with a custom runner.
  run: poetry run pytest -vv

# -----
# 2 Build & Push Docker Image
# -----
build-and-push:
  needs: test
  runs-on: ubuntu-latest
  permissions:
    contents: read
    id-token: write # needed for OIDC login to ECR
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Log in to Amazon ECR
      id: ecr-login
      uses: aws-actions/amazon-ecr-login@v2
      with:
        mask-password: 'true'

    - name: Build and push Docker image
      env:
        IMAGE_TAG: ${ github.sha }
      run: |
        docker build -t ${ env.FULL_IMAGE } .
        docker push ${ env.FULL_IMAGE }

# -----
# 3 Deploy (Terraform)
# -----
deploy:
  needs: build-and-push
  runs-on: ubuntu-latest
  environment: production
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

```

... (continued on next page)

```
- name: Install Terraform
  uses: hashicorp/setup-terraform@v3
  with:
    terraform_version: "1.9.5"

- name: Terraform Init & Apply
  env:
    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
    AWS_DEFAULT_REGION: ${ secrets.AWS_REGION }
  run: |
    cd infra
    terraform init -backend-config="bucket=${ secrets.TF_STATE_BUCKET }"
    terraform apply -auto-approve \
      -var "image_uri=${ env.FULL_IMAGE }" \
      -var "service_name=${ env.IMAGE_NAME }"
```

What This Workflow Does

1. **Lint & test** - ensures code quality before any artifact is built.
2. **Docker build** - runs the multi-stage Dockerfile, tags the image with the git SHA.
3. **ECR login** - uses OIDC (no static credentials).
4. **Push** - uploads the image to the private ECR repository.
5. **Terraform apply** - updates the ECS service (or any cloud resource) with the new image URI.

Switching to GKE - Replace the `deploy` job with a `gcloud` step that runs `helm upgrade` or `kubectyl apply`. The Terraform example below works for GKE as well; you just need a different Terraform module.

3. IaC - Deploying to Three Cloud Providers

Below we present **minimal IaC** for each platform. All three accept an `image_uri`

variable that the CI pipeline injects.

3.1 AWS ECS (Fargate) - Terraform Module

File: `infra/ecs/main.tf`

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

variable "image_uri" {
  description = "Docker image URI from ECR"
  type        = string
}

variable "service_name" {
  description = "Name of the ECS service"
  type        = string
  default     = "llm-fastapi"
}

variable "cpu" {
  description = "CPU units (1024 = 1 vCPU)"
  default     = 2048
}

variable "memory" {
  description = "Memory in MiB"
  default     = 4096
}

# -----
# VPC & Subnets (use existing or create new)
# -----

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "private" {
  filter {
    name = "vpc-id"
  }
}
```

... (continued on next page)

```

        values = [data.aws_vpc.default.id]
    }
}

# -----
# IAM role for ECS task execution
# -----
resource "aws_iam_role" "ecs_task_execution" {
    name = "${var.service_name}-execution-role"

    assume_role_policy = jsonencode({
        Version = "2012-10-17"
        Statement = [{
            Action    = "sts:AssumeRole"
            Effect    = "Allow"
            Principal = { Service = "ecs-tasks.amazonaws.com" }
        }]
    })
}

resource "aws_iam_role_policy_attachment" "ecs_task_execution" {
    role      = aws_iam_role.ecs_task_execution.name
    policy_arn = "arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy"
}

# -----
# ECS Cluster
# -----
resource "aws_ecs_cluster" "this" {
    name = "${var.service_name}-cluster"
}

# -----
# Task Definition (Fargate)
# -----
resource "aws_ecs_task_definition" "this" {
    family              = var.service_name
    requires_compatibilities = ["FARGATE"]
    network_mode       = "awsvpc"
    cpu                 = var.cpu

```

... (continued on next page)

```

memory          = var.memory
execution_role_arn = aws_iam_role.ecs_task_execution.arn
container_definitions = jsonencode([
  {
    name          = var.service_name
    image         = var.image_uri
    essential     = true
    portMappings = [
      {
        containerPort = 8080
        protocol      = "tcp"
      }
    ]
    logConfiguration = {
      logDriver = "awslogs"
      options = {
        awslogs-group      = "/ecs/${var.service_name}"
        awslogs-region     = var.aws_region
        awslogs-stream-prefix = "ecs"
      }
    }
    healthCheck = {
      command      = ["CMD-SHELL", "curl -f http://localhost:8080/health || exit 1"]
      interval     = 30
      timeout      = 5
      retries      = 3
      startPeriod  = 10
    }
  }
])
}

# -----
# Service + Load Balancer (Application Load Balancer)
# -----
resource "aws_lb" "this" {
  name          = "${var.service_name}-alb"
  internal      = false
  load_balancer_type = "application"
  subnets      = data.aws_subnets.private.ids
  security_groups = [aws_security_group.alb.id]
}

resource "aws_security_group" "alb" {

```

... (continued on next page)

```
name      = "${var.service_name}-alb-sg"
description = "Allow inbound HTTP"
vpc_id    = data.aws_vpc.default.id

ingress {
  description = "HTTP from Internet"
  from_port  = 80
  to_port    = 80
  protocol   = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

egress {
  description = "All outbound"
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
}

resource "aws_lb_target_group" "ecs" {
  name      = "${var.service_name}-tg"
  port      = 8080
  protocol  = "HTTP"
  vpc_id    = data.aws_vpc.default.id
  target_type = "ip"
  health_check {
    path          = "/health"
    interval      = 30
    timeout       = 5
    healthy_threshold = 2
    unhealthy_threshold = 2
    matcher       = "200-299"
  }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.this.arn
  port              = "80"
```

... (continued on next page)

```
protocol          = "HTTP"

default_action {
  type            = "forward"
  target_group_arn = aws_lb_target_group.ecs.arn
}
}

resource "aws_ecs_service" "this" {
  name           = var.service_name
  cluster        = aws_ecs_cluster.this.id
  task_definition = aws_ecs_task_definition.this.arn
  desired_count  = 2
  launch_type    = "FARGATE"

  network_configuration {
    subnets          = data.aws_subnets.private.ids
    security_groups   = [aws_security_group.service.id]
    assign_public_ip  = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.ecs.arn
    container_name   = var.service_name
    container_port    = 8080
  }

  deployment_controller {
    type = "ECS"
  }

  # Autoscaling (see next section)
  lifecycle {
    ignore_changes = [desired_count] # Managed by autoscaling
  }
}

resource "aws_security_group" "service" {
  name           = "${var.service_name}-svc-sg"
  description    = "Allow inbound from ALB"
```

... (continued on next page)


```
vpc_id      = data.aws_vpc.default.id

ingress {
  description = "From ALB SG"
  from_port   = 8080
  to_port     = 8080
  protocol    = "tcp"
  security_groups = [aws_security_group.alb.id]
}

egress {
  description = "All outbound"
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
}
```

Key points

- **Health check** - Calls `/health` (you should already have a health endpoint from Module 8).
- **Desired count = 2** - Minimum two tasks for HA; autoscaling will adjust upward.
- **Fargate** - No EC2 management; you can add GPU support by switching to **EC2 launch type** and attaching an `g4dn.xlarge` or `p3.2xlarge` capacity provider.

Autoscaling on ECS

Add a separate file `infra/ecs/autoscaling.tf`:

```

resource "aws_appautoscaling_target" "ecs" {
  max_capacity      = 10
  min_capacity      = 2
  resource_id       = "service/${aws_ecs_cluster.this.name}/${aws_ecs_service.this.name}"
  scalable_dimension = "ecs:service:DesiredCount"
  service_namespace = "ecs"
}

# Scale out when CPU > 70% or when request latency > 500ms (custom metric)
resource "aws_cloudwatch_metric_alarm" "cpu_high" {
  alarm_name          = "${var.service_name}-cpu-high"
  comparison_operator = "GreaterThanOrEqualTo"
  evaluation_periods  = 2
  metric_name         = "CPUUtilization"
  namespace           = "AWS/ECS"
  period              = 60
  statistic            = "Average"
  threshold            = 70
  alarm_actions       = [aws_appautoscaling_policy.scale_out.arn]

  dimensions = {
    ClusterName = aws_ecs_cluster.this.name
    ServiceName = aws_ecs_service.this.name
  }
}

resource "aws_appautoscaling_policy" "scale_out" {
  name          = "${var.service_name}-scale-out"
  policy_type   = "StepScaling"
  resource_id   = aws_appautoscaling_target.ecs.resource_id
  scalable_dimension = aws_appautoscaling_target.ecs.scalable_dimension
  service_namespace = aws_appautoscaling_target.ecs.service_namespace

  step_scaling_policy_configuration {
    adjustment_type = "ChangeInCapacity"
    cooldown        = 60
    metric_aggregation_type = "Average"

    step_adjustment {
      metric_interval_lower_bound = 0
    }
  }
}

```

... (continued on next page)

```

        scaling_adjustment      = 1
    }
}
}

# Optional: Scale based on a custom CloudWatch metric that you push from the app.
# Example: `LLMInferenceLatency` (p95) emitted via OpenTelemetry or Prometheus exporter.
resource "aws_cloudwatch_metric_alarm" "latency_high" {
    alarm_name          = "${var.service_name}-latency-high"
    comparison_operator = "GreaterThanOrEqualToThreshold"
    evaluation_periods  = 2
    metric_name         = "LLMInferenceLatency"
    namespace           = "Custom/LLM"
    period              = 60
    statistic           = "p95"
    threshold           = 500    # milliseconds
    alarm_actions       = [aws_appautoscaling_policy.scale_out.arn]
}

```

GPU Autoscaling - For GPU-enabled ECS on EC2, you would create a capacity provider that references an Auto Scaling group of GPU instances (e.g., `p3.2xlarge`). The autoscaling policy then scales the capacity provider rather than the service count.

3.2 GKE - Terraform + Helm

Folder structure

```

infra/
├── gke/
│   ├── main.tf
│   ├── variables.tf
│   └── helm/
│       └── values.yaml

```

`infra/gke/main.tf` (creates the cluster and a Helm release):

```
terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
    helm = {
      source  = "hashicorp/helm"
      version = "~> 2.12"
    }
  }
}

provider "google" {
  project = var.gcp_project
  region  = var.gcp_region
}

resource "google_container_cluster" "primary" {
  name     = "llm-fastapi-cluster"
  location = var.gcp_region

  # Enable GPU nodes
  node_config {
    machine_type = "n1-standard-4"
    oauth_scopes = ["https://www.googleapis.com/auth/cloud-platform"]
    # Add GPU accelerator
    guest_accelerator {
      type  = "nvidia-tesla-t4"
      count = 1
    }
    # Install NVIDIA driver automatically
    metadata = {
      "disable-legacy-endpoints" = "true"
    }
  }
}

# Enable Workload Identity for secure GCR/ECR access
workload_identity_config {
  identity_namespace = "${var.gcp_project}.svc.id.goog"
```

... (continued on next page)

```

    }

    remove_default_node_pool = true
}

resource "google_container_node_pool" "gpu_pool" {
  name          = "gpu-pool"
  location      = var.gcp_region
  cluster       = google_container_cluster.primary.name
  node_count    = 2

  node_config {
    machine_type = "n1-standard-8"
    guest_accelerator {
      type  = "nvidia-tesla-t4"
      count = 1
    }
    oauth_scopes = ["https://www.googleapis.com/auth/cloud-platform"]
    metadata = {
      "disable-legacy-endpoints" = "true"
    }
    # Enable GPU driver installation via DaemonSet (NVIDIA device plugin)
    # The GKE node image already contains the driver installer.
  }
}

# -----
# Helm provider - install the FastAPI chart
# -----
provider "helm" {
  kubernetes {
    host                = google_container_cluster.primary.endpoint
    token               = data.google_client_config.default.access_token
    cluster_ca_certificate =
base64decode(google_container_cluster.primary.master_auth.0.cluster_ca_certificate)
  }
}

data "google_client_config" "default" {}

resource "helm_release" "fastapi" {

```

... (continued on next page)

```
name      = "llm-fastapi"
chart     = "./helm/fastapi"
namespace = "default"
version   = "0.1.0"

set {
  name = "image.repository"
  value = var.image_repository # e.g. "gcr.io/my-project/llm-fastapi"
}

set {
  name = "image.tag"
  value = var.image_tag # git SHA
}

set {
  name = "service.type"
  value = "LoadBalancer"
}

set {
  name = "resources.limits.cpu"
  value = "2000m"
}

set {
  name = "resources.limits.memory"
  value = "8Gi"
}

set {
  name = "resources.limits.nvidia.com/gpu"
  value = "1"
}

# Autoscaling
set {
  name = "autoscaling.enabled"
  value = "true"
}
```

... (continued on next page)

```
set {  
  name = "autoscaling.minReplicas"  
  value = "2"  
}  
set {  
  name = "autoscaling.maxReplicas"  
  value = "10"  
}  
set {  
  name = "autoscaling.targetCPUUtilizationPercentage"  
  value = "70"  
}  
}
```

`infra/gke/helm/fastapi/Chart.yaml` (minimal chart):

```
apiVersion: v2  
name: fastapi  
description: Helm chart for FastAPI + LangChain service  
type: application  
version: 0.1.0  
appVersion: "1.0"
```

`infra/gke/helm/fastapi/templates/deployment.yaml` (excerpt):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "fastapi.fullname" . }}
  labels:
    {{- include "fastapi.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount | default 2 }}
  selector:
    matchLabels:
      {{- include "fastapi.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "fastapi.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          envFrom:
            - secretRef:
                name: llm-fastapi-secrets
      resources:
        limits:
          cpu: {{ .Values.resources.limits.cpu }}
          memory: {{ .Values.resources.limits.memory }}
          nvidia.com/gpu: {{ .Values.resources.limits.nvidia.com/gpu | default "1" }}
      readinessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 5
        periodSeconds: 10
      livenessProbe:
        httpGet:
          path: /health
          port: 8080
```

... (continued on next page)


```
initialDelaySeconds: 30
periodSeconds: 15
```

`infra/gke/helm/fastapi/templates/hpa.yaml` (autoscaling):

```
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "fastapi.fullname" . }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ include "fastapi.fullname" . }}
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.autoscaling.targetCPUUtilizationPercentage }}
{{- end }}
```

Deploying from CI

Add a job to the GitHub Actions workflow that runs `helm upgrade --install` after the image is pushed:

```
- name: Deploy to GKE via Helm
env:
  IMAGE_REPO: ${ secrets.GCR_REPOSITORY }}
  IMAGE_TAG: ${ github.sha }}
  GKE_CLUSTER: llm-fastapi-cluster
  GKE_ZONE: ${ secrets.GKE_ZONE }}
  GCP_PROJECT: ${ secrets.GCP_PROJECT }}
run: |
  gcloud auth configure-docker ${ secrets.GCR_HOST }}
  gcloud container clusters get-credentials $GKE_CLUSTER --zone $GKE_ZONE --project $GCP_PROJECT
  helm upgrade --install llm-fastapi ./infra/gke/helm/fastapi \
    --set image.repository=$IMAGE_REPO \
    --set image.tag=$IMAGE_TAG \
    --namespace default \
    --wait
```

GPU-specific notes: GKE automatically installs the NVIDIA device plugin DaemonSet on nodes that have GPUs. The pod spec above requests `nvidia.com/gpu: 1`, which guarantees that the scheduler places the pod on a GPU node.

3.3 Azure Container Apps - Bicep + GitHub Actions

Azure Container Apps (ACA) offers a **serverless** container platform with built-in **KEDA-based autoscaling**. It works nicely for LLM services when you need **event-driven scaling** (e.g., scaling on queue depth).

```
`infra/aca/main.bicep`
```



... (continued on next page)

... (continued on next page)

... (continued on next page)

****Safety Red-Teaming****: Adversarial fine-tuning and evaluation aimed at exposing and mitigating disallowed or harmful model behaviors before release.

****System Prompt****: A hidden instruction that sets the assistant's persona, tone, or overarching behavior (e.g., "You are a helpful, concise AI").

****Temperature****: A sampling hyper-parameter controlling randomness; low values (≈ 0.0) produce deterministic output, while higher values ($\approx 0.7-0.9$) increase creativity.

****Token Budgeting****: The practice of accounting for the number of tokens consumed by system prompts, user input, and model output to stay within the model's context window and cost limits.

****Tokenization (Byte-Pair Encoding - BPE)****: The process of breaking raw text into sub-word tokens based on a learned vocabulary, balancing coverage and vocabulary size.

****Top-p (Nucleus Sampling)****: A sampling strategy that limits generation to the smallest set of tokens whose cumulative probability exceeds a threshold p , providing a controllable trade-off between diversity and quality.

****Zero-Shot Prompting****: Issuing a single instruction to the model without any exemplars, relying on the model's pre-training and instruction-tuning to perform the task.

****OpenAI API****: The cloud service that exposes models such as GPT-3.5-Turbo and GPT-4 via HTTP endpoints, handling tokenization, inference, and billing.

****Anthropic API****: The cloud service that provides access to Claude models, offering similar endpoint semantics to OpenAI but with distinct pricing and safety guarantees.

****Hugging Face Transformers****: An open-source library offering model architectures, tokenizers, and pipelines for running LLMs locally or on custom hardware.

****LangChain****: A Python framework that abstracts LLM calls, prompt templates, memory, and retrieval, enabling rapid construction of complex LLM-driven applications.

****FastAPI****: A high-performance Python web framework used to expose LLM-backed services (e.g., chat endpoints) as RESTful APIs.

****tiktoken****: A lightweight Python library for counting tokens according to OpenAI's tokenization scheme, essential for token budgeting.

****Bias Amplification****: The tendency of an LLM to reinforce or exaggerate societal biases present in its training data, requiring detection and mitigation strategies.

****Max Tokens****: The upper limit on the number of tokens the model may generate in a single response; setting it prevents runaway outputs and helps control costs.

IMPORTANT DISCLAIMER

AI-Generated Content Notice

This document has been entirely generated by artificial intelligence technology through the Pustakam Injin platform. While significant effort has been made to ensure accuracy and coherence, readers should be aware of the following important considerations:

- The content is produced by AI language models and may contain factual inaccuracies, outdated information, or logical inconsistencies.
- Information should be independently verified before being used for critical decisions, academic citations, or professional purposes.
- The AI may generate plausible-sounding but incorrect or fabricated information (known as "hallucinations").
- Views and opinions expressed do not necessarily reflect those of the creators, developers, or any affiliated organizations.
- This content should not be considered a substitute for professional advice in medical, legal, financial, or other specialized fields.

Intellectual Property & Usage

This document is provided "as-is" for informational and educational purposes. Users are encouraged to fact-check, cross-reference, and critically evaluate all content. The Pustakam Injin serves as a knowledge exploration tool and starting point for research, not as a definitive source of truth.

Quality Assurance

While the Pustakam Injin employs advanced AI models and formatting techniques to produce professional-quality documents, no warranty is made regarding completeness, reliability, or accuracy. Users assume full responsibility for how they use, interpret, and apply this content.

Generated by: **Pustakam Injin**

Date: February 23, 2026 at 08:11 PM

For questions or concerns about this content, please refer to the Pustakam Injin documentation or contact the platform administrator.